

Cuestionario técnico

OCP Java SE 17 Developer & Oracle SQL / PL-SQL

Banco completo de 500 preguntas — formato cuestionario impreso

Incluye teoría y práctica, organizadas por categoría, con espacio para marcar respuestas. Las respuestas y la retroalimentación técnica se encuentran al final de cada sección.

Sección	Categoría	Preguntas
Teoría	Java básico	40
Teoría	Programación orientada a objetos	49
Teoría	Streams y programación funcional	29
Teoría	JDBC	28
Teoría	Oracle SQL	18
Teoría	PL/SQL	22
Teoría	Optimización	17
Práctica	Java básico	60
Práctica	Programación orientada a objetos	51
Práctica	Streams y programación funcional	61
Práctica	JDBC	32
Práctica	Oracle SQL	42
Práctica	PL/SQL	38

Práctica	Optimización	13
----------	--------------	----

PARTE I — TEORÍA

Java básico (40 preguntas)

1. [peso 2] ¿Cuál es el rango de un byte en Java?
 - A) 0 a 255
 - B) -256 a 255
 - C) -127 a 128
 - D) -128 a 127
2. [peso 2] ¿Qué tipo de dato NO puede inferirse con var?
 - A) Una lambda asignada a una interfaz funcional
 - B) Un tipo genérico con diamante
 - C) Un arreglo inicializado
 - D) null sin cast
3. [peso 2] ¿Qué ocurre al asignar un int a un long sin cast?
 - A) Pérdida de precisión silenciosa
 - B) Se requiere autoboxing
 - C) Error de compilación
 - D) Conversión implícita
4. [peso 2] ¿Por qué String es inmutable en Java?
 - A) Para permitir herencia segura
 - B) Para reducir el uso de memoria en todos los casos
 - C) Por seguridad, caching de hashCode y uso seguro en el String pool
 - D) Porque implementa Serializable
5. [peso 3] En un text block ("""), ¿qué determina la indentación incidental que se elimina?
 - A) No se elimina indentación nunca
 - B) Solo la primera línea de contenido
 - C) Siempre se eliminan 4 espacios
 - D) La línea de cierre y la línea con menor indentación
6. [peso 2] ¿Qué operador realiza desplazamiento a la derecha sin signo?
 - A) <<
 - B) ~>
 - C) >>>
 - D) >>
7. [peso 2] ¿Cuál de estos es un identificador válido para una variable en Java 17?
 - A) precio\$total
 - B) _ solo no es válido como identificador desde Java 9
 - C) new, palabra reservada del lenguaje
 - D) 2precio, no puede iniciar con un dígito

8. [peso 3] ¿Cuándo es obligatorio yield en un switch expression?
- A) Nunca; yield siempre es opcional en cualquier caso
 - B) Cuando un caso usa bloque {} y debe devolver un valor
 - C) Siempre que se use la sintaxis de flecha ->
 - D) Solo es obligatorio dentro del caso default
9. [peso 2] ¿Qué clase debe usarse para cálculos monetarios exactos?
- A) double, por su precisión de punto flotante
 - B) Math, ya que expone redondeo configurable
 - C) float, más liviano que double
 - D) BigDecimal
10. [peso 3] ¿Cuál es la diferencia clave entre Period y Duration?
- A) Son intercambiables
 - B) Duration solo funciona con Instant
 - C) Period es immutable y Duration no
 - D) Period modela cantidad en fechas
11. [peso 2] ¿Qué método de String elimina espacios en blanco Unicode al inicio y final?
- A) clean()
 - B) strip()
 - C) chomp()
 - D) trim()
12. [peso 3] ¿Qué garantiza el contrato entre equals() y hashCode()?
- A) hashCodes distintos implican objetos distintos según equals
 - B) No existe relación obligatoria
 - C) hashCodes iguales implican objetos iguales
 - D) Objetos iguales según equals deben tener el mismo hashCode
13. [peso 2] ¿Cuál es el valor por defecto de una variable local int no inicializada?
- A) No tiene: usarla sin inicializar es error de compilación
 - B) 0, igual que en los campos de instancia
 - C) Basura de memoria heredada del stack
 - D) null, porque toda referencia arranca sin valor
14. [peso 2] ¿Qué palabra clave impide que una variable local sea reasignada?
- A) final
 - B) immutable
 - C) static
 - D) const
15. [peso 3] ¿Cuál es el orden de precedencia correcto (de mayor a menor)?
- A) || , && , !
 - B) && , ! , ||
 - C) ! , && , ||
 - D) ! , || , &&

16. [peso 2] ¿Qué estructura garantiza al menos una ejecución del cuerpo?
- A) for-each
 - B) do-while
 - C) while
 - D) for
17. [peso 3] ¿Qué imprime `System.out.printf("%.2f", 1234567.891)`?
- A) 1234567,89
 - B) 1,234,567.89
 - C) 1.234.567,89
 - D) 1234567.89
18. [peso 2] ¿Qué método convierte "42" en un int primitivo?
- A) `Integer.intValue`
 - B) `Integer.valueOf`
 - C) `Integer.parseInt`
 - D) `new Integer("42")`
19. [peso 3] ¿Qué literal numérico NO es válido en Java 17?
- A) `int a = 1_000_000;`
 - B) `int d = _1000;`
 - C) `double b = 1_000.5;`
 - D) `int c = 0b1010;`
20. [peso 3] ¿Cuál es la diferencia entre `StringBuilder` y `StringBuffer`?
- A) `StringBuilder` es inmutable
 - B) `StringBuffer` es más rápido
 - C) `StringBuilder` es sincronizado
 - D) `StringBuffer` es sincronizado
21. [peso 2] ¿Qué devuelve `"banana".indexOf('a', 2)`?
- A) 5
 - B) 1
 - C) -1
 - D) 3
22. [peso 3] ¿Qué pasa si dos casos de un switch clásico (con :) coinciden y falta break?
- A) Error de compilación por falta de break obligatorio
 - B) Solo se ejecuta el primero y el switch termina ahí
 - C) Fall-through: se ejecutan los casos siguientes hasta break o fin
 - D) Se ejecuta el default sin importar el case que coincidió
23. [peso 3] ¿Qué tipos NO son válidos en la expresión de un switch en Java 17 (sin preview)?
- A) `String` y `enum`, que sí son válidos en el switch
 - B) `long`, `float`, `double` y `boolean`
 - C) `int` y `char`, ambos válidos como discriminante
 - D) `Integer` y `Short`, válidos por unboxing automático

24. [peso 2] ¿Qué clase representa un instante en la línea de tiempo UTC?
- A) Clock
 - B) Instant
 - C) LocalDateTime
 - D) ZoneId
25. [peso 3] ¿Qué formato produce `DateTimeFormatter.ofPattern("MM/dd/yyyy")` con `LocalTime`?
- A) La fecha actual del sistema, ignorando el patrón
 - B) La hora en formato 12h, adaptando el patrón
 - C) 00/00/0000, con ceros por los campos ausentes
 - D) `UnsupportedTemporalTypeException` en runtime
26. [peso 2] ¿Cuál es el resultado de `(int) 9.99`?
- A) 9.0
 - B) 10
 - C) 9
 - D) Error de compilación
27. [peso 2] ¿Dónde puede usarse `var` en Java 17?
- A) En tipos de retorno
 - B) En parámetros de métodos
 - C) En variables locales con inicializador
 - D) En campos de instancia
28. [peso 2] ¿Qué método de `Arrays` busca en un arreglo ORDENADO?
- A) `Arrays.find`
 - B) `Arrays.binarySearch`
 - C) `Arrays.search`
 - D) `Arrays.locate`
29. [peso 3] ¿Qué imprime `System.out.println(new int[]{1,2}.equals(new int[]{1,2}))`... conceptualmente, por qué?
- A) `false`: los arreglos heredan `equals` de `Object`
 - B) Error de compilación
 - C) `true`: el compilador los interna
 - D) `true`: los arreglos sobrescriben `equals`
30. [peso 3] Con pattern matching, ¿cuándo NO está disponible la variable de patrón en el `else`?
- A) Nunca está disponible en `else`
 - B) Cuando la condición es `if`
 - C) Cuando se usa negación: `if (!(o instanceof Tipo t))`
 - D) Siempre está disponible
31. [peso 2] ¿Qué expresión evalúa el operador ternario?
- A) condición : valorSiTrue ? valorSiFalse
 - B) condición ? valorSiTrue : valorSiFalse
 - C) valorSiTrue ? condición : valorSiFalse
 - D) condición ?? valorSiTrue

32. [peso 3] ¿Qué tipo resulta de la expresión (true ? 1 : 2.0)?
- A) int
 - B) Object
 - C) Number
 - D) double
33. [peso 3] En un text block, ¿qué hace la secuencia \ al final de una línea?
- A) Inserta un salto de línea literal
 - B) No es válida
 - C) Suprime el salto de línea
 - D) Escapa el delimitador
34. [peso 2] ¿Qué paquete se importa implícitamente en todo archivo Java?
- A) java.util
 - B) java.io
 - C) java.lang
 - D) java.base
35. [peso 3] ¿Cuál es el orden correcto en un archivo .java?
- A) package, import, class
 - B) import, package, class
 - C) class, package, import
 - D) Indistinto
36. [peso 3] ¿Qué sucede con `int max = Integer.MAX_VALUE; max++`?
- A) Se satura en MAX_VALUE sin generar overflow
 - B) ArithmeticException al superar el máximo representable
 - C) Overflow silencioso: pasa a Integer.MIN_VALUE
 - D) Se promueve automáticamente a long antes de sumar
37. [peso 2] ¿Qué método de String verifica contenido ignorando mayúsculas?
- A) compareTo, aunque distingue mayúsculas de minúsculas
 - B) equalsIgnoreCase
 - C) equals, que si distingue mayúsculas
 - D) matches, pensado para expresiones regulares
38. [peso 3] ¿Qué devuelve `"abc".repeat(0)`?
- A) "abc"
 - B) Cadena vacía ""
 - C) null
 - D) IllegalArgumentException
39. [peso 2] ¿Cuál es el tipo de retorno de `String.charAt(int)`?
- A) Character
 - B) char
 - C) int
 - D) String

40. [peso 3] ¿Qué hace String.valueOf(null) directamente en una llamada?

- A) NullPointerException en runtime
- B) Error de compilación por ambigüedad
- C) Devuelve cadena vacía
- D) Imprime "null"

Programación orientada a objetos (49 preguntas)

41. [peso 2] ¿Qué pilar de la OOP oculta el estado interno tras métodos de acceso?

- A) Herencia
- B) Polimorfismo
- C) Abstracción
- D) Encapsulamiento

42. [peso 3] ¿Qué regla aplica al tipo de retorno al sobrescribir un método?

- A) Puede ser covariante
- B) Puede ser un supertipo
- C) Puede ser cualquier tipo
- D) Debe ser idéntico siempre

43. [peso 3] Al sobrescribir, ¿qué puede hacer el método hijo con las excepciones checked?

- A) Declarar las mismas, subtipos, menos, o ninguna
- B) Solo puede agregar RuntimeException... y checked nuevas
- C) Debe declarar exactamente las mismas
- D) Declarar cualquier excepción nueva

44. [peso 2] ¿Qué modificador de acceso permite visibilidad en el mismo paquete y subclases?

- A) default (package-private)
- B) public
- C) protected
- D) private

45. [peso 3] ¿Qué debe ser la PRIMERA sentencia de un constructor?

- A) super() o this(), explícito o implícito
- B) Cualquier sentencia, siempre que compile
- C) Una asignación de campos antes que todo
- D) return, si el constructor termina temprano

46. [peso 2] ¿Puede una clase abstracta tener constructor?

- A) Solo si el constructor se declara public explícitamente
- B) Solo si la clase no declara ningún método abstracto
- C) No, nunca: las clases abstractas no admiten constructores
- D) Sí, lo invocan los constructores de las subclases

47. [peso 3] ¿Qué combinación de modificadores es ilegal en un método?
- A) private static
 - B) public static
 - C) abstract final
 - D) protected final
48. [peso 3] En Java 17, ¿qué tipos de miembros puede tener una interfaz?
- A) Abstractos y campos de instancia mutables
 - B) Solo default y static, sin abstractos
 - C) Solo métodos abstractos, como antes de Java 8
 - D) Abstractos, default, static, private y constantes
49. [peso 3] Si una clase hereda un método concreto de una superclase y un default idéntico de una interfaz, ¿cuál gana?
- A) El más específico en orden de declaración
 - B) El default de la interfaz
 - C) Error de compilación
 - D) El de la superclase
50. [peso 2] ¿Qué genera automáticamente un record?
- A) Un builder interno para construcción fluida
 - B) Solo el constructor canónico, sin nada más
 - C) Constructor canónico, accessors, equals, hashCode y toString
 - D) Setters y getters para cada componente
51. [peso 3] ¿Qué NO puede hacer un record?
- A) Declarar constructores adicionales
 - B) Extender otra clase
 - C) Declarar métodos estáticos
 - D) Implementar interfaces
52. [peso 3] ¿Qué exige sealed sobre sus subclases permitidas?
- A) Que sean final, sealed o non-sealed
 - B) Nada adicional
 - C) Que estén en otro módulo
 - D) Que sean abstractas
53. [peso 3] Si las subclases de una clase sealed están en el mismo archivo, ¿es obligatoria la cláusula permits?
- A) No, el compilador las infiere
 - B) Solo si hay más de dos
 - C) Sí, siempre
 - D) Solo en módulos
54. [peso 3] ¿Cuándo se ejecuta el constructor de un enum?
- A) Cada vez que se referencia una constante
 - B) Una vez por constante, al cargar la clase
 - C) Al llamar values()
 - D) Nunca: los enums no tienen constructor

55. [peso 3] ¿Qué variables locales puede capturar una clase anónima o lambda?
- A) Solo final o efectivamente final
 - B) Solo variables de tipo primitivo, nunca objetos
 - C) Cualquiera, sin restricciones de reasignación
 - D) Solo variables static de la clase contenedora
56. [peso 2] ¿Qué relación modela mejor la composición frente a la herencia?
- A) "usa-un" temporal
 - B) "tiene-un" (has-a)
 - C) No hay diferencia
 - D) "es-un" (is-a)
57. [peso 3] ¿Qué imprime getClass() en una referencia polimórfica Padre p = new Hijo()?
- A) class Padre
 - B) class Object
 - C) Depende del compilador
 - D) class Hijo
58. [peso 3] ¿Qué diferencia una sobrecarga (overload) de una sobreescritura (override)?
- A) Override solo aplica a métodos static
 - B) Overload: misma firma en clases distintas; override: firmas distintas
 - C) Son sinónimos
 - D) Overload: mismo nombre con parámetros distintos en la misma clase
59. [peso 3] ¿Cuál es el orden de inicialización al instanciar una subclase?
- A) Instancia del hijo, constructor hijo y recién después el padre completo
 - B) Constructor hijo, luego constructor padre, luego los estáticos
 - C) Aleatorio según la JVM y el recolector de basura activo
 - D) Estáticos del padre, estáticos del hijo, instancia del padre, constructor padre, instancia del hijo, constructor hijo
60. [peso 2] ¿Qué método de Object debe sobreescribirse junto con equals()?
- A) clone()
 - B) toString()
 - C) finalize()
 - D) hashCode()
61. [peso 3] ¿Qué garantiza la anotación @Override?
- A) Error de compilación si no se sobreescribe realmente un método del supertipo
 - B) Que el método pase a ser final automáticamente
 - C) Visibilidad public forzada sobre el método anotado
 - D) Mejor rendimiento en tiempo de ejecución
62. [peso 2] ¿Qué es una clase final?
- A) Una clase sin constructores
 - B) Una clase inmutable
 - C) Una clase abstracta terminada
 - D) Una clase que no puede ser extendida

63. [peso 4] ¿Qué requisitos definen una clase inmutable correcta?
- A) Implementar Cloneable para copiar el estado interno
 - B) Campos private final, sin setters, clase final y copias defensivas de campos mutables
 - C) Usar exclusivamente tipos primitivos en los campos
 - D) Solo declarar la clase final basta para lograrlo
64. [peso 3] ¿Qué hace @FunctionalInterface?
- A) Genera la implementación de la lambda en tiempo de compilación
 - B) Permite declarar múltiples métodos abstractos sin error
 - C) Convierte cualquier interfaz en funcional automáticamente
 - D) Verifica en compilación que exista exactamente un método abstracto
65. [peso 3] En genéricos, ¿qué significa List<? extends Number>?
- A) Lista de superclases de Number, como Object
 - B) Lista solo de Number exacto, sin subtipos permitidos
 - C) Lista de un subtipo desconocido de Number: se puede leer como Number, no agregar
 - D) Lista que acepta agregar cualquier subtipo de Number
66. [peso 4] Según PECS, ¿qué firma es correcta para copiar de src a dst?
- A) copy(List<? extends T> src, List<? super T> dst)
 - B) copy(List<T> src, List<Object> dst)
 - C) copy(List<? super T> src, List<? extends T> dst)
 - D) copy(List<?> src, List<?> dst)
67. [peso 3] ¿Qué es el type erasure?
- A) Eliminación de tipos no usados por el GC
 - B) Los parámetros genéricos se eliminan en compilación
 - C) Un error de casting
 - D) La conversión de genéricos a arrays
68. [peso 2] ¿Qué categoría de excepción NO exige try/catch ni throws?
- A) Unchecked (RuntimeException y Error)
 - B) Todas las excepciones lo exigen sin excepción
 - C) IOException, por derivar de Exception
 - D) Checked, ya que el compilador las verifica
69. [peso 3] En try-with-resources con dos recursos, ¿en qué orden se cierran?
- A) En orden inverso a la declaración
 - B) En orden de declaración
 - C) Solo se cierra el último
 - D) Aleatorio
70. [peso 4] Si el try lanza una excepción y close() lanza otra, ¿qué recibe el catch?
- A) La de close(); la del try se pierde
 - B) La del try; la de close() queda como suppressed
 - C) Ambas se pierden
 - D) Una excepción combinada nueva

71. [peso 3] ¿Qué permite el multi-catch catch (IOException | SQLException e)?
- A) Capturar solo subtipos entre sí en el mismo catch
 - B) Manejar excepciones sin relación de herencia en un solo bloque
 - C) Relanzar la excepción sin necesidad de declararla
 - D) Capturar también instancias de Error en el mismo bloque
72. [peso 2] ¿Cuál de estas es una excepción checked?
- A) ClassCastException
 - B) FileNotFoundException
 - C) ArithmeticException
 - D) NullPointerException
73. [peso 3] ¿Cuál es el propósito del módulo (module-info.java)?
- A) Reemplazar los paquetes
 - B) Definir el main
 - C) Declarar dependencias
 - D) Configurar el classpath
74. [peso 3] En module-info.java, ¿qué diferencia hay entre requires y requires transitive?
- A) transitive solo aplica en tiempo de ejecución, no de compilación
 - B) transitive re-exporta la dependencia a quienes dependen de este módulo
 - C) Ninguna: ambas cláusulas se comportan igual siempre
 - D) requires es opcional y puede omitirse sin efecto
75. [peso 3] ¿Qué directiva permite reflexión profunda sobre un paquete sin exportarlo?
- A) opens
 - B) uses
 - C) exports
 - D) provides
76. [peso 3] ¿Puede un método static acceder directamente a un campo de instancia?
- A) Sí, siempre
 - B) Solo si el campo es public
 - C) No: no existe this en contexto estático
 - D) Solo dentro de la misma clase
77. [peso 2] ¿Qué interfaz debe implementar una clase para ser usada en for-each?
- A) Enumeration
 - B) Collection
 - C) Iterator
 - D) Iterable
78. [peso 3] ¿Qué colección mantiene el orden de INSERCIÓN al iterar?
- A) Hashtable
 - B) TreeMap
 - C) LinkedHashMap
 - D) HashMap

79. [peso 3] ¿Qué estructura ofrece búsqueda ordenada con métodos como floorKey y ceilingKey?
- A) LinkedList, gracias a su orden de inserción
 - B) ArrayDeque, por su acceso en ambos extremos
 - C) TreeMap (NavigableMap)
 - D) HashMap, mediante iteración ordenada de buckets
80. [peso 3] ¿Qué diferencia a Comparable de Comparator?
- A) Comparable define el orden natural en la propia clase
 - B) Son idénticas en propósito y uso
 - C) Comparable permite declarar múltiples órdenes
 - D) Comparator solo funciona con tipos primitivos
81. [peso 2] ¿Qué lanza Collections.unmodifiableList(list).add(x)?
- A) IllegalStateException
 - B) Nada, agrega el elemento
 - C) UnsupportedOperationException
 - D) ConcurrentModificationException
82. [peso 3] ¿Qué estructura es más adecuada para una cola LIFO (pila) moderna?
- A) LinkedList con synchronized
 - B) ArrayDeque
 - C) PriorityQueue
 - D) Stack
83. [peso 4] ¿Por qué es peligroso mutar un objeto usado como clave en un HashMap?
- A) No hay riesgo, porque HashMap recalcula el hash al buscar
 - B) Cambia su hashCode y la entrada queda 'perdida' en el bucket original
 - C) Se duplica la clave automáticamente dentro del mismo bucket
 - D) El map lo detecta en tiempo real y lanza una excepción
84. [peso 2] ¿Qué patrón de diseño usa un constructor privado y una instancia única estática?
- A) Singleton
 - B) Prototype
 - C) Builder
 - D) Factory
85. [peso 3] ¿Qué problema resuelve el patrón Builder?
- A) Gestionar conexiones a bases de datos de forma seg.
 - B) Resolver la herencia múltiple de constructores
 - C) Serialización eficiente de objetos complejos
 - D) Construcción legible de objetos con muchos parámetros opcionales
86. [peso 3] ¿Qué aporta una jerarquía sealed a un switch por patrones (pattern matching)?
- A) Herencia múltiple entre los subtipos sellados
 - B) Mayor velocidad de ejecución en tiempo de compilación
 - C) Serialización automática de cada subtipo permitido
 - D) Exhaustividad verificada por el compilador sin default

87. [peso 3] ¿Cuál de las cuatro formas de referencia a método NO existe?

- A) instancia::métodoDeInstancia
- B) Clase::this
- C) Clase::métodoEstático
- D) Clase::new

88. [peso 2] ¿Qué imprime Object.toString() por defecto?

- A) NombreDeClase@hashHexadecimal
- B) La dirección física de memoria
- C) El contenido de los campos
- D) null

89. [peso 3] ¿Puede un constructor llamar a this() y super() a la vez?

- A) Sí, en ese orden: primero this() y luego super()
- B) Solo en clases abstractas se permite combinarlas
- C) No: solo una de las dos y debe ser la primera sentencia
- D) Sí, en cualquier orden que el programador elija

Streams y programación funcional (29 preguntas)

90. [peso 2] ¿Cuál es la característica principal de las operaciones intermedias de un Stream?

- A) Son perezosas: no se ejecutan hasta la operación terminal
- B) Se ejecutan inmediatamente al declararlas en el pipeline
- C) Cierran el stream apenas se invocan una vez
- D) Devuelven colecciones en lugar de otro Stream

91. [peso 3] ¿Qué diferencia hay entre map y flatMap?

- A) flatMap es para primitivos
- B) map aplana y flatMap anida
- C) Ninguna
- D) map transforma 1→1

92. [peso 3] ¿Qué devuelve findFirst() sobre un stream vacío?

- A) NoSuchElementException
- B) 0
- C) Optional.empty()
- D) null

93. [peso 2] ¿Qué lanza Optional.get() sobre un Optional vacío?

- A) NoSuchElementException
- B) NullPointerException
- C) IllegalStateException
- D) null

94. [peso 3] ¿Qué devuelve allMatch sobre un stream VACÍO?
- A) false
 - B) Optional.empty
 - C) true (verdad vacía)
 - D) NoSuchElementException
95. [peso 3] ¿Qué requisito tiene reduce(identity, accumulator) para paralelismo correcto?
- A) Que la operación devuelva un Optional en vez de un valor
 - B) Que el stream se procese siempre de forma secuencial
 - C) Que el acumulador sea asociativo y la identidad neutra
 - D) Que el acumulador esté marcado como synchronized
96. [peso 3] ¿Qué colector cuenta elementos por grupo?
- A) summingInt solamente
 - B) partitioningBy
 - C) groupingBy(f, counting())
 - D) toMap con count
97. [peso 3] ¿Qué diferencia a IntStream de Stream<Integer>?
- A) IntStream opera con primitivos sin boxing y ofrece sum/average/max especializados
 - B) Stream<Integer> es más rápido por el cache de Integer
 - C) Ninguna, son completamente intercambiables en la API
 - D) IntStream no admite la operación map en su pipeline
98. [peso 2] ¿Qué interfaz funcional recibe un argumento y devuelve boolean?
- A) Function<T, Boolean>
 - B) Predicate<T>
 - C) Consumer<T>
 - D) Supplier<Boolean>
99. [peso 2] ¿Qué interfaz funcional NO recibe argumentos y devuelve un valor?
- A) Supplier<T>
 - B) Function<T,R>
 - C) Consumer<T>
 - D) Runnable
100. [peso 3] ¿Qué interfaz funcional corresponde a (String, String) -> Integer?
- A) BinaryOperator<String>
 - B) BiConsumer<String, Integer>
 - C) Function<String, Integer>
 - D) BiFunction<String, String, Integer>
101. [peso 2] ¿Qué interfaz funcional usa forEach de un Stream?
- A) Consumer
 - B) Predicate
 - C) Supplier
 - D) Function

102. [peso 3] ¿Cuándo conviene usar `parallelStream()`?
- A) Cuando se necesita preservar un orden estricto de salida
 - B) Siempre: el paralelismo es más rápido en cualquier caso
 - C) En streams pequeños que hacen operaciones de I/O
 - D) Con colecciones grandes, operaciones costosas por elemento, sin estado compartido mutable
103. [peso 3] ¿Qué operación intermedia es 'stateful' y puede penalizar el paralelismo?
- A) `map`
 - B) `peek`
 - C) `filter`
 - D) `sorted`
104. [peso 3] ¿Qué hace `Collectors.teeing` (Java 12)?
- A) Duplica cada elemento antes de recolectarlo
 - B) Divide el stream original en dos streams independientes
 - C) Une dos streams en uno solo antes de recolectar
 - D) Aplica dos colectores al mismo stream y combina ambos resultados con una función
105. [peso 3] ¿Qué diferencia hay entre `Stream.generate` y `Stream.iterate`?
- A) `generate` ordena los elementos que produce
 - B) `iterate` es siempre un stream finito por definición
 - C) `generate` usa un `Supplier` sin estado previo
 - D) Ninguna, ambos métodos son intercambiables
106. [peso 2] ¿Qué operación terminal devuelve siempre `long`?
- A) `max()`
 - B) `sum()`
 - C) `reduce()`
 - D) `count()`
107. [peso 3] ¿Por qué `peek()` no debe usarse para lógica de negocio?
- A) Está pensada para debugging: el JIT/stream puede omitirla si el resultado no la necesita
 - B) No compila si se combina con lambdas que capturan variables
 - C) Cierra por completo el stream apenas se invoca una sola vez
 - D) Es simplemente más lento que `map` en cualquier pipeline
108. [peso 3] ¿Qué colector calcula el promedio de un campo numérico?
- A) `Collectors.averagingInt`
 - B) `Collectors.counting`
 - C) `Collectors.summingInt`
 - D) `IntStream.average`
109. [peso 2] ¿Qué método crea un `Stream` desde un arreglo?
- A) `arr.stream()`
 - B) `Stream.array(arr)`
 - C) `Arrays.stream(arr)`
 - D) `Stream.from(arr)`

110. [peso 3] ¿Qué característica pierde un stream generado desde un HashSet?
- A) Posibilidad de paralelismo
 - B) Operaciones terminales
 - C) Encounter order garantizado
 - D) Lazy evaluation
111. [peso 3] ¿Cuál es la firma funcional de Consumer<T>.andThen?
- A) Convierte el Consumer en una Function
 - B) Devuelve un nuevo Consumer que ejecuta ambos en secuencia
 - C) Devuelve únicamente el resultado del segundo Consumer
 - D) No existe el método andThen en Consumer
112. [peso 3] ¿Qué interfaz representa un operador binario del mismo tipo, como Integer::sum?
- A) BinaryOperator<Integer>
 - B) BiFunction<Integer,Integer,Integer>
 - C) IntBinaryOperator... ambas B y C son válidas
 - D) UnaryOperator<Integer>
113. [peso 3] ¿Cuándo usar Collectors.toUnmodifiableList() frente a toList()?
- A) Son completamente idénticos en comportamiento
 - B) toUnmodifiableList garantiza inmutabilidad
 - C) toUnmodifiableList es la única que permite valores null
 - D) toList ya es immutable por contrato desde Java 16
114. [peso 2] ¿Qué operación transforma Stream<T> en Stream<R>?
- A) map
 - B) filter
 - C) peek
 - D) distinct
115. [peso 3] ¿Qué hace mapMulti (Java 16) frente a flatMap?
- A) Es idéntico a flatMap en todos los casos de uso
 - B) Solo funciona con tipos primitivos como IntStream
 - C) Aplana dos niveles de anidamiento en una sola llamada
 - D) Reemplaza el stream intermedio por un Consumer, evitando crear un Stream por elemento
116. [peso 3] En un Comparator, ¿qué hace nullsFirst?
- A) Convierte cada null en una cadena vacía para comparar
 - B) Envuelve otro comparador tratando null como menor que cualquier valor
 - C) Lanza NullPointerException apenas encuentra un null
 - D) Elimina los elementos null antes de comparar
117. [peso 3] ¿Qué sucede al llamar stream() sobre una colección y luego modificar la colección ANTES de la terminal?
- A) Siempre se ve el estado original de la colección
 - B) No compila si la colección se modifica antes de la terminal
 - C) El stream es 'late-binding': generalmente refleja los cambios hechos antes de la terminal
 - D) Lanza siempre ConcurrentModificationException al iterar

118. [peso 2] ¿Qué método de Optional ejecuta una acción solo si hay valor?

- A) ifPresent
- B) orElse
- C) filter
- D) get

JDBC (28 preguntas)

119. [peso 2] ¿Qué interfaz de JDBC representa la conexión física a la base de datos?

- A) Driver
- B) Session
- C) DataSource
- D) Connection

120. [peso 3] ¿Por qué se prefiere DataSource sobre DriverManager en producción?

- A) Es una API más nueva que DriverManager
- B) DriverManager simplemente no funciona con Oracle
- C) DataSource no necesita URL para conectarse a la BD
- D) Permite pooling de conexiones, configuración externa y transacciones distribuidas

121. [peso 2] ¿Cuál es el formato correcto de una URL JDBC?

- A) oracle://host/db
- B) db:jdbc://host
- C) jdbc:vendor:detalles
- D) http://host:1521/db

122. [peso 3] ¿Dónde apunta el cursor de un ResultSet recién creado?

- A) Antes de la primera fila
- B) En la última fila
- C) Es indefinido
- D) En la primera fila

123. [peso 3] ¿Qué devuelve el método execute() de PreparedStatement?

- A) ResultSet siempre, sin importar el tipo de sentencia
- B) boolean: true si el resultado es un ResultSet, false si es un update count
- C) void, ya que el resultado se consulta aparte
- D) int con el número de filas afectadas por el update

124. [peso 3] ¿Cuál es el estado por defecto de auto-commit en una Connection JDBC?

- A) No existe el concepto de auto-commit en JDBC
- B) Depende del driver JDBC que se esté utilizando
- C) Desactivado: hay que confirmar cada sentencia a mano
- D) Activado: cada sentencia se confirma automáticamente

125. [peso 3] ¿Qué garantiza try-with-resources con Connection, PreparedStatement y ResultSet?
- A) Solo garantiza el auto-commit de la transacción
 - B) Mejor rendimiento general de las consultas ejecutadas
 - C) El cierre automático en orden inverso, incluso ante excepciones
 - D) Reintentos automáticos ante fallos de conexión
126. [peso 3] ¿Qué interfaz JDBC sirve para invocar procedimientos almacenados?
- A) PreparedStatement
 - B) CallableStatement
 - C) StoredProcedure
 - D) ProcedureStatement
127. [peso 3] ¿Qué tipo de ResultSet permite moverse hacia atrás con previous()?
- A) CONCUR_UPDATABLE, que solo controla la escritura
 - B) TYPE_SCROLL_INSENSITIVE o TYPE_SCROLL_SENSITIVE
 - C) Cualquiera de los tipos admite moverse hacia atrás
 - D) TYPE_FORWARD_ONLY, el tipo por defecto del ResultSet
128. [peso 2] ¿Qué excepción lanzan casi todas las operaciones JDBC?
- A) RuntimeException
 - B) IOException
 - C) SQLException
 - D) DataAccessException
129. [peso 3] ¿Qué hace Connection.setReadOnly(true)?
- A) Bloquea la tabla mientras dure la transacción
 - B) Impide ejecutar cualquier SELECT sobre la conexión
 - C) Da una PISTA al driver/BD para optimizar
 - D) Cierra la conexión inmediatamente
130. [peso 4] ¿Qué nivel de aislamiento evita lecturas sucias pero permite lecturas no repetibles?
- A) READ_UNCOMMITTED
 - B) READ_COMMITTED
 - C) REPEATABLE_READ
 - D) SERIALIZABLE
131. [peso 3] ¿Para qué sirve DatabaseMetaData?
- A) Consultar capacidades del driver/BD: tablas, versiones, soporte de features
 - B) Describir la estructura del ResultSet actual
 - C) Optimizar automáticamente las consultas ejecutadas
 - D) Configurar los parámetros del pool de conexiones
132. [peso 3] ¿Qué ventaja de rendimiento aporta PreparedStatement frente a Statement?
- A) Usa menos memoria en el cliente Java
 - B) La BD parsea y compila el plan una vez y lo reutiliza con distintos parámetros
 - C) Ninguna ventaja real frente a Statement clásico
 - D) Evita el uso de transacciones en la sesión

133. [peso 2] ¿Sigue siendo necesario `Class.forName("oracle.jdbc.OracleDriver")` en JDBC 4+?
- A) Sí, siempre es obligatorio antes de conectar
 - B) Solo es necesario específicamente con Oracle
 - C) Solo se requiere en aplicaciones web desplegadas
 - D) No: los drivers se autoregistran mediante META-INF/services
134. [peso 3] ¿Qué método de `PreparedStatement` asigna NULL a un parámetro?
- A) `setNull(1, Types.VARCHAR)`
 - B) `skip(1)`
 - C) `setString(1, null)` únicamente
 - D) `setEmpty(1)`
135. [peso 3] ¿Qué clase de `java.sql` corresponde a un `TIMESTAMP` con precisión de nanosegundos?
- A) `java.util.Date`
 - B) `java.sql.Time`
 - C) `java.sql.Date`
 - D) `java.sql.Timestamp`
136. [peso 3] ¿Qué hace `setFetchSize(100)` en un `Statement`?
- A) Crea 100 conexiones nuevas hacia la base de datos
 - B) Sugiere al driver traer filas del servidor en bloques de 100
 - C) Limita la consulta a traer un máximo de 100 filas
 - D) Es equivalente a aplicar `ROWNUM <= 100` en el SQL
137. [peso 3] ¿Qué diferencia hay entre `setMaxRows(n)` y `setFetchSize(n)`?
- A) Ambos limitan el total de filas de igual manera
 - B) Ninguna diferencia real entre los dos métodos
 - C) `maxRows` LIMITA el total de filas del `ResultSet`
 - D) `fetchSize` limita filas; `maxRows` solo el buffer
138. [peso 3] ¿Qué representa un `RowSet` frente a un `ResultSet`?
- A) Un tipo particular de índice sobre la tabla
 - B) Exactamente lo mismo, con otro nombre
 - C) Un `ResultSet` extendido, a menudo desconectado
 - D) Una vista materializada sobre la consulta
139. [peso 2] ¿Qué patrón de diseño implementa `DriverManager` al elegir el driver según la URL?
- A) Observer
 - B) Singleton
 - C) Factory
 - D) Decorator
140. [peso 4] En Oracle JDBC, ¿qué logra la propiedad `oracle.jdbc.implicitStatementCacheSize`?
- A) Aumenta el tamaño del fetch por defecto del driver
 - B) Limita el número de filas devueltas por consulta
 - C) Activa el cifrado SSL en la conexión JDBC
 - D) Cachea `PreparedStatement`s cerrados para reutilizar cursores y evitar re-parseo

141. [peso 3] ¿Qué método permite comprobar si una conexión sigue válida?
- A) conn.isValid(timeoutSeconds)
 - B) conn.test()
 - C) conn.isClosed() es suficiente
 - D) conn.ping()
142. [peso 3] ¿Qué ocurre al llamar dos veces executeQuery() sobre el mismo PreparedStatement?
- A) El primer ResultSet se cierra implícitamente al ejecutar de nuevo
 - B) Se mantienen ambos ResultSet abiertos simultáneamente
 - C) Lanza SQLException al intentar ejecutar de nuevo
 - D) Se duplican las filas devueltas en el segundo ResultSet
143. [peso 3] ¿Qué aporta un pool como HikariCP frente a abrir conexiones directas?
- A) Cifrado automático del tráfico de red hacia la base de datos
 - B) Solo agrega registros de logging adicionales sobre cada conexión
 - C) Cache de resultados de las últimas consultas ejecutadas por el pool
 - D) Reutilización de conexiones, límites de concurrencia, validación y métricas: menor latencia y protección de la BD
144. [peso 4] ¿Por qué un parámetro ? NO puede sustituir el nombre de una tabla?
- A) Es una limitación arbitraria del estándar JDBC
 - B) Sí puede sustituir el nombre de una tabla
 - C) Solo aplica esta restricción en bases de datos Oracle
 - D) Los parámetros son valores de datos ligados tras compilar el plan
145. [peso 3] ¿Qué significa que el driver Oracle 'thin' sea tipo 4?
- A) Es 100% Java y habla directamente el protocolo de red de la BD, sin librerías nativas
 - B) Requiere tener instalado un cliente OCI nativo
 - C) Usa el puente ODBC para comunicarse con la BD
 - D) Solo funciona en sistemas operativos Linux
146. [peso 2] ¿Qué paquete contiene las interfaces principales de JDBC?
- A) java.db
 - B) oracle.sql
 - C) java.sql
 - D) javax.jdbc

Oracle SQL (18 preguntas)

147. [peso 2] ¿En qué orden lógico procesa Oracle las cláusulas de un SELECT?
- A) WHERE, FROM, SELECT, ORDER BY
 - B) FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY
 - C) SELECT, FROM, WHERE, GROUP BY
 - D) FROM, SELECT, WHERE, HAVING

148. [peso 3] ¿Qué diferencia a NVL2(expr, a, b) de NVL(expr, a)?
- A) NVL2 solo acepta argumentos numéricos
 - B) NVL2 devuelve a si expr NO es NULL y b si es NULL
 - C) Son completamente iguales en comportamiento
 - D) NVL2 compara dos expresiones entre sí
149. [peso 3] ¿Qué diferencia hay entre COUNT(*) y COUNT(columna)?
- A) COUNT(*) es más lento siempre
 - B) COUNT(*) cuenta filas
 - C) COUNT(columna) incluye NULLs
 - D) Ninguna
150. [peso 3] ¿Cuál es la diferencia entre WHERE y HAVING?
- A) HAVING es más rápido que WHERE en general
 - B) WHERE solo admite el operador AND
 - C) Son sinónimos intercambiables en Oracle
 - D) WHERE filtra filas antes de agrupar
151. [peso 3] ¿Qué operador devuelve las filas de la primera consulta que NO están en la segunda?
- A) INTERSECT
 - B) DIFFERENCE
 - C) EXCEPT DISTINCT
 - D) MINUS
152. [peso 3] ¿Qué tipo de subconsulta puede referirse a columnas de la consulta externa?
- A) Inline view
 - B) Correlacionada
 - C) Ninguna
 - D) Escalar solamente
153. [peso 3] ¿Qué cláusula moderna de Oracle 12c+ reemplaza el patrón ROWNUM para paginación?
- A) TOP n, igual que en SQL Server
 - B) OFFSET n ROWS FETCH NEXT m ROWS ONLY
 - C) SAMPLE, pensada para muestreo aleatorio
 - D) LIMIT ... OFFSET, como en MySQL
154. [peso 3] ¿Qué diferencia hay entre TRUNC(fecha, 'MM') y ROUND(fecha, 'MM')?
- A) ROUND siempre elimina la parte de la hora
 - B) Ninguna, ambas funciones truncan igual
 - C) TRUNC va al día 1 del mes actual
 - D) TRUNC redondea al mes más cercano
155. [peso 2] ¿Qué constraint garantiza que una columna referencie una clave de otra tabla?
- A) CHECK
 - B) UNIQUE
 - C) NOT NULL
 - D) FOREIGN KEY

156. [peso 3] ¿Qué sentencia combina INSERT y UPDATE según exista o no la fila?

- A) UPSERT
- B) INSERT OR UPDATE
- C) REPLACE
- D) MERGE

157. [peso 3] ¿Qué ventaja tienen las funciones analíticas sobre GROUP BY?

- A) Devuelven agregados SIN colapsar las filas: cada fila conserva su detalle junto al cálculo de ventana
- B) No pueden combinarse con una cláusula WHERE en la misma consulta
- C) Son más lentas de ejecutar, aunque producen resultados más exactos
- D) Reemplazan por completo la necesidad de mantener índices

158. [peso 3] ¿Qué es una CTE (WITH clause) en Oracle?

- A) Una tabla temporal permanente
- B) Una subconsulta nombrada
- C) Un tipo de índice
- D) Un sinónimo

159. [peso 3] ¿Qué diferencia hay entre una vista y una vista materializada?

- A) La vista es solo una consulta almacenada
- B) La vista guarda físicamente los datos
- C) Ninguna diferencia entre ambos conceptos
- D) La materializada no puede indexarse nunca

160. [peso 3] ¿Qué diferencia hay entre NATURAL JOIN y JOIN ... USING?

- A) NATURAL JOIN exige además una cláusula ON explícita
- B) USING se aplica de forma implícita sin declararlo
- C) Son sintácticamente idénticas en Oracle
- D) NATURAL une por TODAS las columnas homónimas automáticamente

161. [peso 2] ¿Qué comando revierte una transacción no confirmada?

- A) UNDO
- B) CANCEL
- C) ROLLBACK
- D) REVERT

162. [peso 3] ¿Qué tipo de dato de Oracle almacena texto de longitud variable hasta 4000 bytes (por defecto)?

- A) VARCHAR2
- B) CHAR
- C) CLOB solamente
- D) STRING

163. [peso 3] ¿Qué hace GROUP BY ROLLUP(a, b)?

- A) El producto cartesiano de agrupaciones
- B) Ordena los grupos
- C) Solo agrupa por a y b
- D) Genera los grupos

164. [peso 3] ¿Qué objeto genera valores numéricos únicos y secuenciales?

- A) INDEX
- B) SYNONYM
- C) TRIGGER
- D) SEQUENCE

PL/SQL (22 preguntas)

165. [peso 2] ¿Cuál es la estructura mínima de un bloque anónimo PL/SQL?

- A) DECLARE...END;
- B) BEGIN...END;
- C) BEGIN...EXCEPTION...END;
- D) PROCEDURE...END;

166. [peso 3] ¿Qué declara v_sal emp.sal%TYPE?

- A) Una variable con el MISMO tipo de la columna emp.sal
- B) Una constante cuyo valor no puede cambiar
- C) Un cursor apuntando a la tabla emp
- D) Una fila completa con todas las columnas de emp

167. [peso 3] ¿Qué regla aplica a WHEN OTHERS en un manejador de excepciones?

- A) Debe ser el ÚLTIMO manejador del bloque
- B) Puede ir en cualquier posición
- C) Debe ser el primero
- D) Es obligatorio

168. [peso 4] ¿Qué hace PRAGMA EXCEPTION_INIT?

- A) Crea una excepción completamente nueva en Oracle
- B) Desactiva el manejo de excepciones en el bloque
- C) Inicializa variables antes de lanzar la excepción
- D) Asocia una excepción declarada por el usuario a un código de error Oracle

169. [peso 3] ¿Qué diferencia hay entre un cursor implícito y uno explícito?

- A) Ninguna diferencia real entre ambos tipos
- B) El explícito es siempre más rápido de ejecutar
- C) El implícito lo gestiona Oracle para cada SELECT INTO/DML
- D) El implícito no existe como concepto en PL/SQL

170. [peso 3] ¿Qué diferencia un procedimiento de una función en PL/SQL?

- A) Las funciones no pueden aceptar parámetros de entrada
- B) Ninguna diferencia real entre ambos en PL/SQL
- C) La función DEBE devolver un valor con RETURN y puede usarse en SQL
- D) El procedimiento se ejecuta más rápido siempre

171. [peso 4] ¿Qué característica tiene un parámetro OUT al entrar al procedimiento?
- A) Conserva el valor del llamador
 - B) Es una constante
 - C) Es de solo lectura
 - D) Entra como NULL
172. [peso 3] ¿Qué contiene la especificación (spec) de un package frente a su body?
- A) Solo comentarios
 - B) Los triggers
 - C) El código completo
 - D) La interfaz pública
173. [peso 4] ¿Qué provoca el error ORA-04091 (mutating table)?
- A) Un trigger de fila que consulta o modifica LA MISMA tabla que disparó el trigger
 - B) Un cursor que quedó abierto sin cerrarse
 - C) Un interbloqueo entre dos sesiones concurrentes
 - D) Falta de permisos suficientes sobre la tabla
174. [peso 3] ¿Qué son :OLD y :NEW en un trigger de fila?
- A) Variables globales del paquete que las declara
 - B) Savepoints creados automáticamente por el trigger
 - C) Pseudoregistros con los valores previos y nuevos de la fila
 - D) Cursores abiertos implícitamente por Oracle
175. [peso 4] ¿Qué hace la cláusula LIMIT en BULK COLLECT?
- A) Ordena los resultados antes de recolectarlos
 - B) No tiene ningún efecto sobre el rendimiento
 - C) Controla cuántas filas se traen por FETCH, acotando el consumo de memoria PGA
 - D) Limita el total de filas que la consulta puede procesar
176. [peso 3] ¿Qué es un REF CURSOR y para qué se usa?
- A) Un tipo especial de índice sobre la tabla
 - B) Un cursor estático que apunta a una sola consulta
 - C) Un puntero a un cursor que puede abrirse con distintas consultas y pasarse entre programas
 - D) Una vista materializada sobre el resultado
177. [peso 4] ¿Qué hace PRAGMA AUTONOMOUS_TRANSACTION?
- A) Desactiva por completo la generación de redo en el bloque actual
 - B) Ejecuta el bloque en una transacción independiente que puede hacer COMMIT sin afectar la transacción principal
 - C) Acelera la ejecución de la transacción actual reduciendo el I/O
 - D) Impide que se pueda hacer rollback en el resto de la sesión
178. [peso 3] ¿Qué diferencia hay entre NUMBER y PLS_INTEGER para contadores?
- A) Ninguna diferencia relevante entre ambos tipos
 - B) PLS_INTEGER acepta valores decimales igual que NUMBER
 - C) NUMBER siempre es más rápido en cualquier operación
 - D) PLS_INTEGER usa aritmética de hardware y menos memoria: más rápido en bucles

179. [peso 3] ¿Cuándo cambia un objeto PL/SQL a estado INVALID?
- A) Al hacer rollback de la transacción actual
 - B) Solo si el objeto se borra explícitamente
 - C) Cuando cambia un objeto del que depende
 - D) Nunca, una vez compilado permanece válido
180. [peso 4] ¿Qué aporta RESULT_CACHE a una función?
- A) Solo funciona si además la función está marcada como DETERMINISTIC
 - B) Duplica el tamaño de memoria PGA asignada a la sesión actual
 - C) Un cache local válido únicamente dentro de la sesión que lo generó
 - D) Oracle guarda los resultados en la shared pool y los comparte ENTRE sesiones, invalidando al cambiar las tablas dependientes
181. [peso 3] ¿Puede una columna de tabla ser BOOLEAN en Oracle 19c?
- A) No: BOOLEAN es solo de PL/SQL en esa versión
 - B) Solo puede usarse en tablas temporales
 - C) Solo es posible declarándolo con un CHECK
 - D) Sí, desde la versión 19c está disponible
182. [peso 3] ¿Qué hace el paquete DBMS_SCHEDULER?
- A) Genera secuencias
 - B) Administra memoria
 - C) Programa y gestiona trabajos
 - D) Optimiza consultas
183. [peso 4] ¿Qué significan los niveles de PRAGMA en funciones para SQL: WNDS, RNDS?
- A) Definen distintos tipos de trigger disponibles en Oracle
 - B) Son simplemente nombres alternativos de índices del diccionario
 - C) Restricciones de pureza históricas: Write/Read No Database State, usadas con RESTRICT_REFERENCES en packages
 - D) Controlan los distintos modos de apertura de un cursor explícito
184. [peso 3] ¿Qué privilegio se necesita para ejecutar un procedimiento de otro esquema?
- A) SELECT sobre el procedimiento del otro esquema
 - B) EXECUTE sobre el procedimiento
 - C) DBA, el rol administrativo completo
 - D) CREATE PROCEDURE en el esquema propio
185. [peso 4] ¿Qué cambia AUTHID CURRENT_USER en un procedimiento?
- A) No cambia nada respecto al comportamiento por defecto
 - B) Se ejecuta con los privilegios y resolución de nombres del INVOCADOR, no del dueño
 - C) Convierte automáticamente al procedimiento en autónomo
 - D) Requiere privilegios de DBA para poder compilarse
186. [peso 3] ¿Para qué sirve DBMS_UTILITY.FORMAT_ERROR_BACKTRACE?
- A) Formatea números dentro de un bloque PL/SQL
 - B) Reintenta automáticamente la operación fallida
 - C) Cambia el idioma en que se muestran los errores
 - D) Obtener la LÍNEA exacta donde se originó la excepción, complementando SQLERRM

Optimización (17 preguntas)

187. [peso 3] ¿Por qué esta condición impide usar el índice sobre hiredate?

- A) Aplicar una función a la columna indexada
- B) Los índices no funcionan con fechas
- C) El índice solo sirve en PKs
- D) Falta un hint

188. [peso 3] ¿Qué muestra EXPLAIN PLAN / DBMS_XPLAN.DISPLAY?

- A) El tiempo real que tardó la consulta en ejecutarse
- B) Los locks que están activos sobre la tabla
- C) El plan de ejecución ESTIMADO: operaciones, orden, cardinalidades y costos previstos
- D) Muestra los datos reales devueltos por la consulta

189. [peso 4] ¿Cuándo un FULL TABLE SCAN es MEJOR que un acceso por índice?

- A) Solo cuando la tabla es muy pequeña y cabe entera en el buffer cache, únicamente entonces
- B) Cuando existe un índice único sobre la columna filtrada en el WHERE
- C) Cuando la consulta lee un porcentaje alto de la tabla: las lecturas multibloque secuenciales superan a millones de accesos aleatorios por rowid
- D) Nunca es mejor que un acceso indexado, sin importar cuántas filas se devuelvan

190. [peso 4] ¿Qué método de join conviene para unir dos tablas GRANDES sin índices útiles por una igualdad?

- A) HASH JOIN
- B) NESTED LOOPS
- C) MERGE JOIN CARTESIAN
- D) Cualquiera da igual

191. [peso 3] ¿Qué recolecta DBMS_STATS.GATHER_TABLE_STATS y por qué importa?

- A) Privilegios
- B) Estadísticas
- C) Backups
- D) Índices nuevos

192. [peso 3] ¿Qué índice soporta esta consulta: WHERE deptno = 10 ORDER BY sal?

- A) Dos índices separados siempre
- B) Índice en (sal, deptno)
- C) Índice compuesto en
- D) Índice en (sal)

193. [peso 4] ¿Puede usarse un índice en (a, b) para WHERE b = 5 sin condición sobre a?

- A) Solo con hint FULL
- B) No con range scan normal
- C) Nunca
- D) Sí, igual que con a

194. [peso 4] ¿Qué es un índice bitmap y dónde conviene?
- A) Mapas de bits por valor: eficiente en columnas de BAJA cardinalidad en entornos de LECTURA (DWH); pésimo con DML concurrente
 - B) El tipo de índice que Oracle crea por defecto al declarar una clave primaria
 - C) Un índice pensado específicamente para almacenar imágenes y archivos binarios
 - D) Solo puede usarse sobre claves primarias declaradas como UNIQUE
195. [peso 3] ¿Qué diferencia EXISTS de IN en subconsultas, en términos de optimización?
- A) El optimizador moderno suele transformarlas al mismo plan
 - B) IN nunca puede aprovechar un índice disponible
 - C) EXISTS bloquea las filas que va evaluando
 - D) EXISTS siempre supera en rendimiento a IN
196. [peso 4] ¿Qué logra el particionamiento por rango de fechas en una tabla de millones de filas?
- A) Partition pruning: las consultas por fecha leen SOLO las particiones relevantes, y el mantenimiento se hace por partición
 - B) Comprime automáticamente los datos de todas las particiones sin configuración
 - C) Genera índices más pequeños únicamente, sin cambiar el plan de acceso
 - D) Duplica los datos entre las distintas particiones para redundancia
197. [peso 3] ¿Qué mide la 'selectividad' de un predicado?
- A) El número total de columnas de la tabla
 - B) La cantidad de memoria usada por la consulta
 - C) La velocidad de lectura del disco duro
 - D) La fracción de filas que devuelve: predicados muy selectivos favorecen índices
198. [peso 4] ¿Qué es un covering index (índice cubridor)?
- A) Un índice que contiene TODAS las columnas que la consulta necesita: se responde sin acceder a la tabla
 - B) Un índice que cifra el contenido de sus columnas por seguridad
 - C) Un tipo particular de índice bitmap orientado a lectura masiva
 - D) Un índice construido sobre absolutamente todas las columnas de la tabla
199. [peso 3] ¿Qué caracteriza a una consulta 'sargable'?
- A) Que sus predicados permiten aprovechar índices: columna limpia comparada con expresión, sin funciones sobre la columna
 - B) Que la consulta use SELECT * en vez de listar columnas específicas
 - C) Que la consulta incluya hints de optimización explícitos en el texto
 - D) Que la consulta incluya obligatoriamente una cláusula GROUP BY
200. [peso 4] ¿Qué indica una espera alta en 'db file sequential read'?
- A) Lecturas de UN bloque
 - B) Lecturas multibloque de full scans
 - C) Bloqueos de filas
 - D) Problemas de red

201. [peso 3] ¿Qué hace el hint /*+ INDEX(e ix_emp_dept) */?

- A) Sugiere (fuerza si es válido) al optimizador usar ese índice para esa tabla
- B) Recolecta estadísticas actualizadas de la tabla
- C) Crea físicamente el índice mencionado en la tabla
- D) Solo documenta la intención de la consulta

202. [peso 4] ¿Qué causa el error ORA-01555 (snapshot too old)?

- A) Demasiados índices creados sobre la tabla consultada
- B) Una consulta larga necesita versiones de undo ya sobrescritas para mantener su lectura consistente
- C) Una clave foránea que quedó en estado inválido tras un ALTER
- D) Falta de espacio disponible en el disco para el tablespace

203. [peso 3] ¿Por qué conviene paginar resultados grandes hacia la aplicación?

- A) Oracle exige por configuración paginar toda consulta que sea grande
- B) Los ResultSet de JDBC tienen un límite fijo de 1000 filas por diseño
- C) Es una cuestión puramente estética que no afecta el rendimiento real
- D) Limita memoria en cliente y servidor, reduce latencia percibida y evita transferir filas que el usuario nunca verá

PARTE II — PRÁCTICA (CÓDIGO)

Java básico (60 preguntas)

204. [peso 3] ¿Qué imprime este código?

```
int x = 5;  
int y = x++ + ++x;  
System.out.println(y);
```

- A) 11
- B) 10
- C) 12
- D) No compila

205. [peso 3] ¿Cuál es el resultado?

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

- A) 15
- B) 3345
- C) 12345
- D) 339

206. [peso 4] ¿Qué imprime?

```
double d = 0.1 + 0.2;  
System.out.println(d == 0.3);
```

- A) Excepción en runtime
- B) Error de compilación
- C) false
- D) true

207. [peso 3] ¿Qué imprime?

```
String a = "Java";  
String b = "Ja" + "va";  
System.out.println(a == b);
```

- A) false
- B) No compila
- C) Depende de la JVM
- D) true

208. [peso 4] ¿Qué imprime?

```
String a = "Java";  
String b = new String("Java").intern();  
System.out.println(a == b);
```

- A) false
- B) NullPointerException
- C) No compila
- D) true

209. [peso 3] ¿Qué imprime?

```
StringBuilder sb = new StringBuilder("radar");  
sb.insert(2, "io").delete(0, 1);  
System.out.println(sb);
```

- A) raiodar
- B) adar
- C) aiodar
- D) aioadar

210. [peso 3] ¿Cuál es el resultado?

```
var list = List.of(1, 2, 3);  
list.add(4);  
System.out.println(list);
```

- A) [1, 2, 3, 4] tras la llamada a add
- B) UnsupportedOperationException en runtime
- C) [1, 2, 3] sin cambios, ya que add se ignora
- D) No compila porque List.of no tiene el método add

211. [peso 3] ¿Qué imprime?

```
int[] arr = {1, 2, 3};  
int[] copy = arr;  
copy[0] = 99;  
System.out.println(arr[0]);
```

- A) 1
- B) 0
- C) ArrayIndexOutOfBoundsException
- D) 99

212. [peso 3] ¿Qué imprime?

```
System.out.println(-7 % 3);
```

- A) -1
- B) 2
- C) 1
- D) -2

213. [peso 4] ¿Cuál es el resultado?

```
Integer a = 127, b = 127;  
Integer c = 128, d = 128;  
System.out.println((a == b) + " " + (c == d));
```

- A) false false
- B) false true
- C) true false
- D) true true

214. [peso 3] ¿Qué imprime este switch expression?

```
int day = 3;  
String type = switch (day) {  
    case 1, 7 -> "Weekend";  
    case 2, 3, 4, 5, 6 -> "Weekday";  
    default -> "Invalid";  
};  
System.out.println(type);
```

- A) Weekend
- B) No compila
- C) Invalid
- D) Weekday

215. [peso 3] ¿Cuántas veces se imprime "Hi"?

```
for (int i = 0; i < 3; i++)  
    for (int j = 3; j > i; j--)  
        System.out.println("Hi");
```

- A) 5
- B) 6
- C) 9
- D) 3

216. [peso 4] ¿Qué imprime?

```
OUTER: for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) continue OUTER;  
        System.out.print(i + " " + j + " ");  
    }  
}
```

- A) 11 12 21 22 31 32
- B) 11 21 31
- C) 11 21 31 12 22 32
- D) No compila

217. [peso 4] ¿Qué imprime?

```
var bd1 = new BigDecimal("1.0");  
var bd2 = new BigDecimal("1.00");  
System.out.println(bd1.equals(bd2) + " " + (bd1.compareTo(bd2) == 0));
```

- A) true true
- B) true false
- C) false true
- D) false false

218. [peso 3] ¿Qué imprime?

```
LocalDate date = LocalDate.of(2024, 1, 31);  
System.out.println(date.plusMonths(1));
```

- A) 2024-03-02
- B) 2024-02-31
- C) 2024-03-03
- D) 2024-02-29

219. [peso 4] ¿Qué imprime?

```
LocalDateTime ldt = LocalDateTime.of(2024, 3, 10, 2, 30);  
ZonedDateTime zdt = ldt.atZone(ZoneId.of("America/New_York"));  
System.out.println(zdt.getHour());
```

- A) 1
- B) 2
- C) 3
- D) Lanza DateTimeException

220. [peso 3] ¿Qué imprime?

```
String s = "hello";  
s.toUpperCase();  
s.concat(" world");  
System.out.println(s);
```

- A) HELLO world
- B) hello world
- C) hello
- D) HELLO

221. [peso 3] ¿Cuál es el resultado?

```
System.out.println("abcdef".substring(2, 4));
```

- A) cd
- B) cde
- C) bcd
- D) de

222. [peso 4] ¿Qué imprime?

```
char c = 'a';  
c += 2;  
System.out.println(c);  
int i = 'a' + 2;  
System.out.println(i);
```

- A) No compila
- B) c y c
- C) c y 99
- D) 99 y 99

223. [peso 3] ¿Qué imprime?

```
boolean b = false;  
if (b = true)  
    System.out.println("A");  
else  
    System.out.println("B");
```

- A) A
- B) Excepción en runtime
- C) B
- D) No compila

224. [peso 4] ¿Qué imprime?

```
short s = 10;  
s = s + 5; // línea A  
s += 5;    // línea B  
System.out.println(s);
```

- A) No compila por la línea A
- B) No compila por la línea B, que sí requeriría cast
- C) 20, tras sumar ambas líneas sin problema
- D) 15, ignorando la primera asignación

225. [peso 3] ¿Qué imprime?

```
int x = 10;  
x = x++;  
System.out.println(x);
```

- A) 11
- B) No compila
- C) 10
- D) 9

226. [peso 3] ¿Cuál es el resultado?

```
final int[] nums = {1, 2, 3};  
nums[0] = 99;  
nums = new int[5];  
System.out.println(nums[0]);
```

- A) No compila por `nums[0] = 99`, ya que el arreglo es final
- B) No compila por `nums = new int[5]`
- C) 0, porque el arreglo se reinicia igual
- D) 99, conservando el valor asignado antes

227. [peso 3] ¿Qué imprime?

```
System.out.println(10 / 4);  
System.out.println(10 / 4.0);
```

- A) 2.5 y 2.5
- B) 2.5 y 2
- C) 2 y 2.5
- D) 2 y 2

228. [peso 4] ¿Qué imprime?

```
System.out.println(10 / 0.0);  
System.out.println(10 % 0.0 == 10 % 0.0);
```

- A) `ArithmeticException`
- B) Infinity y false
- C) Infinity y true
- D) NaN y false

229. [peso 4] ¿Qué imprime?

```
int i = 0;  
boolean r = (i++ > 0) && (++i > 0);  
System.out.println(i);
```

- A) 2
- B) 1
- C) 0
- D) 3

230. [peso 3] ¿Qué imprime?

```
int i = 0;  
boolean r = (i++ > 0) | (++i > 0);  
System.out.println(i);
```

- A) 1
- B) No compila
- C) 0
- D) 2

231. [peso 3] ¿Qué imprime?

```
String[] arr = {"a", "b", "c"};
for (String s : arr) {
    s = s.toUpperCase();
}
System.out.println(String.join("", arr));
```

- A) aBC
- B) abc
- C) ABC
- D) No compila

232. [peso 4] ¿Qué imprime?

```
var nums = new java.util.ArrayList<>(List.of(1, 2, 3, 4));
for (Integer n : nums) {
    if (n == 2) nums.remove(n);
}
```

- A) No compila
- B) Elimina el 2 sin problema
- C) ConcurrentModificationException
- D) IndexOutOfBoundsException

233. [peso 3] ¿Qué imprime?

```
String s = null;
System.out.println("Valor: " + s);
```

- A) Valor:
- B) No compila
- C) Valor: null
- D) NullPointerException

234. [peso 4] ¿Qué imprime?

```
Integer i = null;
int j = i;
System.out.println(j);
```

- A) NullPointerException en runtime
- B) 0, como cualquier int no inicializado
- C) null, propagado desde el Integer original
- D) No compila por mezclar Integer con int

235. [peso 3] ¿Qué imprime?

```
System.out.println(Integer.parseInt("1010", 2));
```

- A) 1010
- B) NumberFormatException
- C) 10
- D) 2

236. [peso 3] ¿Qué imprime?

```
System.out.println(0x1F);  
System.out.println(017);
```

- A) 31 y 17.0
- B) 1F y 17
- C) 31 y 15
- D) 31 y 17

237. [peso 4] ¿Qué imprime?

```
String s = "a";  
for (int i = 0; i < 3; i++)  
    s += "b";  
System.out.println(s);
```

- A) No compila por reasignar una variable String final
- B) abbb — reutilizando el mismo objeto en cada vuelta
- C) ab, si el bucle solo se ejecutara una vez
- D) abbb — pero crea múltiples objetos String intermedios

238. [peso 3] ¿Qué imprime?

```
System.out.println("Hello".chars().count());
```

- A) Error: chars() no existe
- B) 5
- C) 4
- D) 6

239. [peso 4] ¿Qué imprime?

```
var sb = new StringBuilder("abc");  
sb.reverse().append(sb.length());  
System.out.println(sb);
```

- A) cba4
- B) 3cba
- C) abc3
- D) cba3

240. [peso 4] ¿Qué imprime?

```
int val = 2;  
switch (val) {  
    case 1: System.out.print("A");  
    case 2: System.out.print("B");  
    case 3: System.out.print("C"); break;  
    default: System.out.print("D");  
}
```

- A) BCD
- B) BC
- C) BD
- D) B

241. [peso 3] ¿Qué imprime?

```
LocalDate d1 = LocalDate.of(2024, 1, 1);  
LocalDate d2 = LocalDate.of(2024, 3, 15);  
Period p = Period.between(d1, d2);  
System.out.println(p.getMonths() + " " + p.getDays());
```

- A) 0 74
- B) 3 15
- C) 2 15
- D) 2 14

242. [peso 3] ¿Qué imprime?

```
var dt = LocalDate.of(2024, 2, 29);  
System.out.println(dt.plusYears(1));
```

- A) DateTimeException
- B) 2025-02-29
- C) 2025-03-01
- D) 2025-02-28

243. [peso 4] ¿Qué imprime?

```
double d = 1e308 * 10;  
float f = (float) 3.6e38;  
System.out.println(d + " " + f);
```

- A) Overflow: ArithmeticException
- B) NaN NaN
- C) 1e309 3.6e38
- D) Infinity Infinity

244. [peso 3] ¿Qué imprime?

```
System.out.println(Math.round(-2.5));  
System.out.println(Math.round(2.5));
```

- A) -3 y 2
- B) -2 y 2
- C) -3 y 3
- D) -2 y 3

245. [peso 3] ¿Cuál de estas declaraciones NO compila?

```
var a = 10;           // 1  
var b = null;         // 2  
var c = new int[3];   // 3  
var d = "text";       // 4
```

- A) 4
- B) 2
- C) 1
- D) 3

246. [peso 4] ¿Qué imprime?

```
int[] a = {5, 10, 15};  
System.out.println(java.util.Arrays.binarySearch(a, 8));
```

- A) -1
- B) 1
- C) -2
- D) 0

247. [peso 3] ¿Qué imprime?

```
int[][] m = new int[2][];  
m[0] = new int[]{1, 2};  
m[1] = new int[]{3, 4, 5};  
System.out.println(m[1][2] + m[0].length);
```

- A) ArrayIndexOutOfBoundsException
- B) 7
- C) 5
- D) 6

248. [peso 3] ¿Qué imprime?

```
int[] x = {3, 1, 2};  
int[] y = {3, 1, 2};  
System.out.println(x == y);  
System.out.println(java.util.Arrays.equals(x, y));
```

- A) true true
- B) true false
- C) false false
- D) false true

249. [peso 4] ¿Qué imprime?

```
Object o = Integer.valueOf(42);  
if (o instanceof Number n && n.intValue() > 40)  
    System.out.println("Big " + n);  
else  
    System.out.println("Small");
```

- A) ClassCastException
- B) No compila: n fuera de alcance
- C) Big 42
- D) Small

250. [peso 3] ¿Qué imprime?

```
System.out.println("1" + 2 * 3);
```

- A) 9
- B) 123
- C) 16
- D) 7

251. [peso 4] ¿Qué imprime?

```
byte b = (byte) 130;  
System.out.println(b);
```

- A) 126
- B) Error de compilación
- C) -126
- D) 130

252. [peso 4] ¿Qué imprime?

```
int a = 5;  
System.out.println(true ? a++ : a--);  
System.out.println(a);
```

- A) 5 y 5
- B) 5 y 6
- C) 6 y 5
- D) 6 y 6

253. [peso 3] ¿Qué imprime?

```
String txt = ""  
    Hola  
    Mundo"";  
System.out.println(txt.lines().count());
```

- A) 2
- B) No compila
- C) 1
- D) 3

254. [peso 3] ¿Qué imprime?

```
System.out.println("A,B,,C,".split(",").length);
```

- A) 5
- B) 4
- C) 3
- D) 2

255. [peso 4] ¿Qué imprime?

```
Integer x = 5;  
long y = 5;  
System.out.println(x.equals(y));
```

- A) false
- B) No compila
- C) ClassCastException
- D) true

256. [peso 3] ¿Qué imprime?

```
System.out.println(Boolean.parseBoolean("TRUE"));
System.out.println(Boolean.parseBoolean("yes"));
```

- A) NumberFormatException
- B) true false
- C) false false
- D) true true

257. [peso 4] ¿Qué imprime?

```
char c = 'A';
int shifted = c << 1;
System.out.println(shifted);
```

- A) 'B'
- B) 130
- C) No compila
- D) 65

258. [peso 3] ¿Qué imprime?

```
System.out.println(Math.floorDiv(-7, 2));
System.out.println(-7 / 2);
```

- A) -4 y -4
- B) -3 y -4
- C) -4 y -3
- D) -3 y -3

259. [peso 3] ¿Qué imprime?

```
System.out.println("apple".compareTo("banana") < 0);
```

- A) false
- B) Error
- C) 0
- D) true

260. [peso 4] ¿Qué imprime?

```
String s = "  java  ";
System.out.println "[" + s.strip().indent(2) + "]" );
```

- A) [java]
- B) [java\n]
- C) [java]
- D) [java]

261. [peso 3] ¿Qué imprime?

```
StringBuilder sb = new StringBuilder();  
sb.append(1).append('a').append(true).append(2.0);  
System.out.println(sb);
```

- A) Error de compilación
- B) 1a12.0
- C) 1atrue2.0
- D) 1atrue2

262. [peso 4] ¿Qué imprime?

```
long ms = 24 * 60 * 60 * 1000 * 30;  
System.out.println(ms);
```

- A) 2592000000
- B) Error de compilación
- C) 2.592E9
- D) -1702967296

263. [peso 3] ¿Qué imprime?

```
var d = Duration.ofDays(1);  
var p = Period.ofDays(1);  
System.out.println(d.toHours() + " " + p.getDays());
```

- A) 24 24
- B) 1 1
- C) No compila
- D) 24 1

Programación orientada a objetos (51 preguntas)

264. [peso 3] ¿Qué imprime?

```
class A { void hi() { System.out.print("A"); } }  
class B extends A { void hi() { System.out.print("B"); } }  
// ...  
A obj = new B();  
obj.hi();
```

- A) A
- B) AB
- C) No compila
- D) B

265. [peso 4] ¿Qué imprime?

```
class A { static void hi() { System.out.print("A"); } }
class B extends A { static void hi() { System.out.print("B"); } }
// ...
A obj = new B();
obj.hi();
```

- A) No compila
- B) B
- C) Excepción
- D) A

266. [peso 4] ¿Qué imprime?

```
class A { int x = 1; }
class B extends A { int x = 2; }
// ...
A obj = new B();
System.out.println(obj.x);
```

- A) 0
- B) No compila
- C) 2
- D) 1

267. [peso 4] ¿Qué imprime?

```
class Padre {
    Padre() { System.out.print("P"); init(); }
    void init() { System.out.print("pi"); }
}
class Hijo extends Padre {
    int x = 5;
    void init() { System.out.print("h" + x); }
}
// ...
new Hijo();
```

- A) Ppi
- B) Ph0
- C) Ph5
- D) No compila

268. [peso 3] ¿Cuál es el resultado?

```
public class Item {
    private int id;
    public Item(int id) { id = id; }
    public int getId() { return id; }
}
// ...
System.out.println(new Item(7).getId());
```

- A) 0
- B) 7
- C) NullPointerException
- D) No compila

269. [peso 4] ¿Por qué NO compila?

```
class Base {
    Base(int x) { }
}
class Derived extends Base {
    Derived() { System.out.println("D"); }
}
```

- A) Base debe ser public para permitir la herencia
- B) El compilador inserta super() pero Base no tiene constructor sin argumentos
- C) Sí compila, porque Java genera un constructor vacío igual
- D) Derived debe declararse abstract para heredar de Base

270. [peso 3] ¿Qué imprime?

```
abstract class Shape {
    abstract double area();
    void print() { System.out.println(area()); }
}
class Square extends Shape {
    double area() { return 4.0; }
}
// ...
new Square().print();
```

- A) AbstractMethodError
- B) 4.0
- C) 0.0
- D) No compila

271. [peso 4] ¿Qué imprime?

```
interface A { default String who() { return "A"; } }
interface B { default String who() { return "B"; } }
class C implements A, B {
    public String who() { return A.super.who() + B.super.who(); }
}
// ...
System.out.println(new C().who());
```

- A) No compila
- B) AB
- C) A
- D) B

272. [peso 3] ¿Cuál es el resultado?

```
interface Movable {
    int SPEED = 10;
}
class Car implements Movable {
    void boost() { SPEED = 20; }
}
```

- A) Compila y SPEED pasa a valer 20 en Car
- B) No compila: SPEED es implícitamente final
- C) Compila, pero con un warning del compilador
- D) Excepción en tiempo de ejecución al asignar

273. [peso 4] ¿Qué imprime?

```
record Point(int x, int y) {  
    Point {  
        if (x < 0) x = 0;  
    }  
}  
// ...  
System.out.println(new Point(-5, 3).x());
```

- A) 0
- B) No compila: x es final
- C) -5
- D) 3

274. [peso 3] ¿Qué imprime?

```
record User(String name) {}  
// ...  
var u1 = new User("Ana");  
var u2 = new User("Ana");  
System.out.println(u1 == u2);  
System.out.println(u1.equals(u2));
```

- A) true true
- B) true false
- C) false false
- D) false true

275. [peso 4] ¿Cuál es el resultado?

```
sealed interface Shape permits Circle, Square {}  
record Circle(double r) implements Shape {}  
record Square(double side) implements Shape {}  
record Triangle(double b) implements Shape {}
```

- A) No compila: Triangle no está en permits
- B) Compila con warning
- C) Compila
- D) No compila: los records no implementan sealed interfaces

276. [peso 3] ¿Qué imprime?

```
enum Level {  
    LOW, MID, HIGH;  
}  
// ...  
System.out.println(Level.valueOf("MID").ordinal());
```

- A) MID
- B) 0
- C) 2
- D) 1

277. [peso 4] ¿Qué imprime?

```
enum Op {  
    ADD { int apply(int a, int b) { return a + b; } },  
    MUL { int apply(int a, int b) { return a * b; } };  
    abstract int apply(int a, int b);  
}  
// ...  
System.out.println(Op.MUL.apply(3, 4));
```

- A) 12
- B) IllegalStateException
- C) 7
- D) No compila

278. [peso 3] ¿Cuál es el resultado?

```
class Outer {  
    private int x = 10;  
    class Inner {  
        int getX() { return x; }  
    }  
}  
// ...  
Outer o = new Outer();  
Outer.Inner i = o.new Inner();  
System.out.println(i.getX());
```

- A) 0
- B) No compila
- C) NullPointerException
- D) 10

279. [peso 4] ¿Qué imprime?

```
class Animal {}  
class Dog extends Animal {}  
// ...  
Animal a = new Animal();  
Dog d = (Dog) a;  
System.out.println("ok");
```

- A) ClassCastException en runtime
- B) No compila por el cast entre tipos relacionados
- C) ok, porque el cast siempre es válido en compilación
- D) null, ya que Dog nunca se llega a construir

280. [peso 3] ¿Qué imprime?

```
Object o = "text";  
if (o instanceof Integer i)  
    System.out.println(i + 1);  
else  
    System.out.println("no match");
```

- A) ClassCastException
- B) no match
- C) 1
- D) No compila

281. [peso 4] ¿Qué método se invoca?

```
void call(Object o) { System.out.print("Object"); }  
void call(String s) { System.out.print("String"); }  
void call(Integer i) { System.out.print("Integer"); }  
// ...  
call(null);
```

- A) String, por ser el tipo más específico
- B) Integer, por autoboxing del argumento null
- C) Object, como sobrecarga más genérica
- D) No compila: llamada ambigua

282. [peso 4] ¿Qué imprime?

```
void go(int i) { System.out.print("int"); }  
void go(long l) { System.out.print("long"); }  
void go(Integer i) { System.out.print("Integer"); }  
void go(int... i) { System.out.print("varargs"); }  
// ...  
short s = 5;  
go(s);
```

- A) Integer
- B) varargs
- C) int
- D) long

283. [peso 5] ¿Qué imprime?

```
class A {  
    static { System.out.print("1"); }  
    { System.out.print("2"); }  
    A() { System.out.print("3"); }  
}  
class B extends A {  
    static { System.out.print("4"); }  
    { System.out.print("5"); }  
    B() { System.out.print("6"); }  
}  
// ...  
new B(); new B();
```

- A) 142356 142356
- B) 145236 145236
- C) 142356 2356
- D) 145236 2356

284. [peso 4] ¿Este equals es una sobreescritura válida?

```
class Point {
    int x;
    public boolean equals(Point p) {
        return this.x == p.x;
    }
}
```

- A) No: es una sobrecarga
- B) Sí, pero solo con @Override
- C) Sí, sobreescrive Object.equals
- D) No compila

285. [peso 3] ¿Cuál es el resultado?

```
class Parent {
    protected void greet() {}
}
class Child extends Parent {
    private void greet() {}
}
```

- A) Compila sin ninguna advertencia del compilador
- B) Compila, pero con un warning de acceso
- C) No compila: no se puede reducir la visibilidad al sobreescibir
- D) Depende del paquete donde se declare Child

286. [peso 4] ¿Por qué esta clase NO es immutable?

```
final class Config {
    private final List<String> keys;
    Config(List<String> keys) { this.keys = keys; }
    List<String> getKeys() { return keys; }
}
```

- A) Porque a la clase le falta un setter explícito
- B) Sí es immutable, ya que el campo keys es final
- C) Porque el llamador conserva/recibe la referencia y puede mutar la lista
- D) Porque List no puede declararse final en Java

287. [peso 3] ¿Qué imprime?

```
interface Greeter {
    String greet(String name);
}
// ...
Greeter g = name -> "Hola " + name;
System.out.println(g.greet("Eva"));
```

- A) Hola name
- B) Hola Eva
- C) name
- D) No compila: falta implementar la interfaz

288. [peso 4] ¿Compila esta interfaz funcional?

```
@FunctionalInterface
interface Worker {
    void work();
    boolean equals(Object o);
    default void rest() {}
}
```

- A) No: tiene dos métodos abstractos distintos
- B) No: falta declarar el modificador static
- C) No: default no se permite junto a @FunctionalInterface
- D) Sí: equals coincide con un método public de Object y no cuenta

289. [peso 4] ¿Cuál es el resultado?

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;
```

- A) Excepción en runtime
- B) Compila: Integer es subtipo de Number
- C) Compila con cast implícito
- D) No compila: los genéricos son invariantes

290. [peso 5] ¿Qué imprime?

```
Object[] arr = new Integer[2];
arr[0] = "hello";
```

- A) Compila y ejecuta sin ningún problema
- B) No compila por asignar String a un arreglo de Integer
- C) ArrayStoreException en runtime
- D) NullPointerException al acceder al índice 0

291. [peso 3] ¿Qué imprime?

```
static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}
// ...
System.out.println(max("pear", "apple"));
```

- A) apple
- B) pear
- C) No compila
- D) ClassCastException

292. [peso 4] ¿Qué imprime?

```
static String test() {  
    try {  
        return "try";  
    } finally {  
        System.out.print("finally-");  
    }  
}  
// ...  
System.out.print(test());
```

- A) finally-
- B) try-finally
- C) finally-try
- D) try

293. [peso 5] ¿Qué imprime?

```
static int get() {  
    try {  
        return 1;  
    } finally {  
        return 2;  
    }  
}  
// ...  
System.out.println(get());
```

- A) 12
- B) No compila
- C) 2
- D) 1

294. [peso 4] ¿Qué requisito debe cumplir un recurso de try-with-resources?

```
try (var r = obtenerRecurso()) {  
    r.usar();  
}
```

- A) Implementar AutoCloseable
- B) Extender Resource
- C) Tener un método close() cualquiera
- D) Implementar Closeable únicamente

295. [peso 3] ¿Qué imprime?

```
try {  
    throw new IllegalArgumentException("bad");  
} catch (RuntimeException e) {  
    System.out.print("RE-");  
}  
catch (Exception e) {  
    System.out.print("E-");  
}
```

- A) RE-
- B) E-
- C) No compila
- D) RE-E-

296. [peso 4] ¿Cuál es el resultado?

```
try {  
    // código que no lanza IOException  
    System.out.print("ok");  
} catch (java.io.IOException e) {  
    System.out.print("io");  
}
```

- A) ok, porque el driver ignora el catch innecesario
- B) io, porque IOException sí puede ocurrir en el I/O interno
- C) Compila con un warning sobre excepción no verificada
- D) No compila: IOException nunca se lanza en el try

297. [peso 4] ¿Qué imprime?

```
class Res implements AutoCloseable {  
    String id;  
    Res(String id) { this.id = id; }  
    public void close() { System.out.print(id); }  
}  
// ...  
try (var a = new Res("A"); var b = new Res("B")) {  
    System.out.print("T");  
}
```

- A) BAT
- B) TAB
- C) ABT
- D) TBA

298. [peso 3] ¿Qué imprime?

```
class Counter {  
    static int count = 0;  
    Counter() { count++; }  
}  
// ...  
new Counter(); new Counter(); new Counter();  
System.out.println(Counter.count);
```

- A) 0
- B) 3
- C) No compila
- D) 1

299. [peso 4] ¿Qué imprime?

```
class Box<T> {  
    private T item;  
    void set(T item) { this.item = item; }  
    T get() { return item; }  
}  
// ...  
Box box = new Box();  
box.set("text");  
Integer i = (Integer) box.get();
```

- A) No compila por el uso de un raw type
- B) text, devuelto tal cual sin verificar el tipo
- C) null, porque el raw type descarta el valor
- D) ClassCastException en runtime

300. [peso 4] ¿Qué imprime?

```
var map = new HashMap<String, Integer>();  
map.put("a", 1);  
map.merge("a", 10, Integer::sum);  
map.merge("b", 10, Integer::sum);  
System.out.println(map);
```

- A) {a=11, b=null}
- B) NullPointerException
- C) {a=10, b=10}
- D) {a=11, b=10}

301. [peso 3] ¿Qué imprime?

```
var map = new HashMap<String, Integer>();  
System.out.println(map.getDefault("x", 0));  
map.putIfAbsent("x", 5);  
map.putIfAbsent("x", 9);  
System.out.println(map.get("x"));
```

- A) null y 9
- B) null y 5
- C) 0 y 5
- D) 0 y 9

302. [peso 4] ¿Cuál es el resultado?

```
var set = new TreeSet<String>();  
set.add("b");  
set.add(null);
```

- A) No compila al intentar agregar un null
- B) NullPointerException en runtime
- C) null se ignora silenciosamente al agregarlo
- D) El set contiene null y b sin inconvenientes

303. [peso 4] ¿Qué imprime?

```
record Item(String name) {}  
var set = new HashSet<Item>();  
set.add(new Item("pen"));  
System.out.println(set.contains(new Item("pen")));
```

- A) false
- B) NullPointerException
- C) No compila
- D) true

304. [peso 4] ¿Qué imprime?

```
record P(String name, int age) {}  
var list = new ArrayList<>(List.of(  
    new P("Ana", 30), new P("Bo", 25), new P("Ana", 25)));  
list.sort(Comparator.comparing(P::name)  
    .thenComparing(P::age, Comparator.reverseOrder()));  
System.out.println(list.get(0).age());
```

- A) 25
- B) No compila
- C) Depende del hash
- D) 30

305. [peso 4] ¿Qué imprime?

```
var original = new ArrayList<>(List.of(1, 2));  
var copia = List.copyOf(original);  
var vista = Collections.unmodifiableList(original);  
original.add(3);  
System.out.println(copia.size() + " " + vista.size());
```

- A) 3 3
- B) 3 2
- C) 2 3
- D) 2 2

306. [peso 4] ¿Qué imprime?

```
Queue<Integer> q = new PriorityQueue<>(Comparator.reverseOrder());  
q.offer(3); q.offer(1); q.offer(2);  
System.out.print(q.poll());  
System.out.print(q.poll());
```

- A) 13
- B) 32
- C) 31
- D) 12

307. [peso 3] ¿Qué imprime?

```
var deque = new ArrayDeque<Integer>();
deque.push(1);
deque.push(2);
deque.offer(3);
System.out.println(deque.peek());
```

- A) null
- B) 3
- C) 2
- D) 1

308. [peso 3] ¿Qué imprime?

```
var list = new ArrayList<Integer>(List.of(10, 20, 30));
list.remove(1);
System.out.println(list);
```

- A) [20, 30]
- B) [10, 20]
- C) [10, 30]
- D) [10, 20, 30]

309. [peso 4] ¿Qué imprime?

```
class Lazy {
    private static Lazy inst;
    private Lazy() { System.out.print("init-"); }
    static Lazy get() {
        if (inst == null) inst = new Lazy();
        return inst;
    }
}
// ...
Lazy.get(); Lazy.get();
```

- A) init-init-
- B) Nada
- C) No compila
- D) init-

310. [peso 3] ¿Qué imprime?

```
sealed interface Animal permits Cat, Dog {}
record Cat() implements Animal {}
record Dog() implements Animal {}
// ...
Animal a = new Dog();
String s = a instanceof Cat ? "gato" : "perro";
System.out.println(s);
```

- A) gato
- B) perro
- C) No compila
- D) null

311. [peso 4] ¿Cuál es el resultado?

```
class Outer {  
    static class Nested {  
        int val = 7;  
    }  
}  
// ...  
var n = new Outer.Nested();  
System.out.println(n.val);
```

- A) NullPointerException
- B) 0
- C) No compila: falta instancia de Outer
- D) 7

312. [peso 4] ¿Qué imprime?

```
interface Task { void run(); }  
// ...  
int count = 0;  
Task t = () -> System.out.println(count);  
count++;  
t.run();
```

- A) 0, el valor capturado antes del incremento
- B) 1, reflejando el incremento posterior
- C) Nada, la lambda nunca llega a ejecutarse
- D) No compila

313. [peso 4] ¿Qué referencia a método equivale a (s1, s2) -> s1.compareTo(s2)?

```
Comparator<String> c = ???;
```

- A) Comparator::compare
- B) String::compareTo
- C) String::compare
- D) s1::compareTo

314. [peso 5] ¿Qué imprime?

```
class A {  
    A() { this(5); System.out.print("a"); }  
    A(int x) { System.out.print("b"); }  
}  
// ...  
new A();
```

- A) a
- B) ab
- C) Recursión infinita
- D) ba

315. [peso 3] ¿Qué imprime?

```
Stream.of(1, 2, 3)
    .map(n -> {
        System.out.print(n);
        return n * 2;
    });
```

- A) Nada
- B) 246
- C) 123
- D) No compila

316. [peso 3] ¿Qué imprime?

```
var r = Stream.of("a", "bb", "ccc")
    .filter(s -> s.length() > 1)
    .map(String::toUpperCase)
    .toList();
System.out.println(r);
```

- A) [CCC]
- B) [BB, CCC]
- C) [bb, ccc]
- D) [A, BB, CCC]

317. [peso 4] ¿Qué imprime?

```
var s = Stream.of(1, 2, 3);
s.count();
s.findFirst();
```

- A) IllegalStateException: stream ya operado o cerrado
- B) 1, porque el stream conserva su estado entre llamadas
- C) Optional[1], igual que en la primera invocación
- D) 3, porque count() reinicia el cursor del stream

318. [peso 4] ¿Qué imprime?

```
var r = Stream.of(List.of(1, 2), List.of(3, 4))
    .flatMap(List::stream)
    .reduce(0, Integer::sum);
System.out.println(r);
```

- A) [1, 2, 3, 4]
- B) No compila
- C) 10
- D) 4

319. [peso 3] ¿Qué imprime?

```
var r = IntStream.rangeClosed(1, 5)
    .filter(n -> n % 2 == 0)
    .sum();
System.out.println(r);
```

- A) 2
- B) 6
- C) 9
- D) 15

320. [peso 4] ¿Qué imprime?

```
Optional<String> opt = Optional.empty();
System.out.println(opt.orElseGet(() -> "A"));
System.out.println(opt.map(String::toUpperCase).orElse("B"));
```

- A) A y B
- B) Excepción en la primera línea
- C) null y null
- D) A y NoSuchElementException

321. [peso 4] ¿Cuál es la diferencia de comportamiento?

```
opt.orElse(caro());
opt.orElseGet(() -> caro());
```

- A) orElse SIEMPRE ejecuta caro()
- B) orElse devuelve Optional
- C) orElseGet es más lento
- D) Ninguna

322. [peso 3] ¿Qué imprime?

```
var r = Stream.of("b", "a", "c")
    .sorted(Comparator.reverseOrder())
    .findFirst()
    .orElse("z");
System.out.println(r);
```

- A) z
- B) b
- C) a
- D) c

323. [peso 4] ¿Qué imprime?

```
var r = Stream.of(1, 2, 3, 4)
    .peek(System.out::print)
    .anyMatch(n -> n > 2);
System.out.println(" " + r);
```

- A) 3 true
- B) 123 true
- C) 12 false
- D) 1234 true

324. [peso 4] ¿Qué imprime?

```
var r = Stream.iterate(1, n -> n * 2)
    .limit(5)
    .toList();
System.out.println(r);
```

- A) Bucle infinito
- B) [1, 2, 4, 8, 16]
- C) [2, 4, 8, 16, 32]
- D) [1, 2, 3, 4, 5]

325. [peso 3] ¿Qué imprime?

```
var r = Stream.iterate(1, n -> n < 20, n -> n * 3)
    .toList();
System.out.println(r);
```

- A) [1, 3, 9]
- B) [3, 9, 27]
- C) Bucle infinito
- D) [1, 3, 9, 27]

326. [peso 4] ¿Qué imprime?

```
var r = Stream.of("a", "b", "c")
    .reduce("", (x, y) -> y + x);
System.out.println(r);
```

- A) cba
- B) abc
- C) bca
- D) acb

327. [peso 3] ¿Qué imprime?

```
var r = Stream.of("apple", "banana", "cherry")
    .collect(Collectors.joining(", ", "[", "]"));
System.out.println(r);
```

- A) [apple][banana][cherry]
- B) [applebananacherry]
- C) [apple, banana, cherry]
- D) apple, banana, cherry

328. [peso 4] ¿Qué contiene el mapa resultante?

```
var r = Stream.of("apple", "avocado", "banana")
    .collect(Collectors.groupingBy(s -> s.charAt(0)));
```

- A) {apple=a, avocado=a, banana=b}
- B) No compila
- C) {a=[apple, avocado], b=[banana]}
- D) {a=apple, b=banana}

329. [peso 4] ¿Qué imprime?

```
var r = Stream.of(1, 2, 3, 4, 5)
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
System.out.println(r.get(true));
```

- A) [1, 3, 5]
- B) [true, false]
- C) [2, 4]
- D) {false=[1,3,5]}

330. [peso 5] ¿Cuál es el resultado?

```
var r = Stream.of("a", "b", "a")
    .collect(Collectors.toMap(s -> s, String::length));
```

- A) {a=1, b=1}, sobrescribiendo la clave repetida
- B) {a=2, b=1}, sumando las longitudes repetidas
- C) {a=[1,1], b=[1]}, agrupando los valores
- D) IllegalStateException: clave duplicada

331. [peso 4] ¿Qué imprime?

```
var r = Stream.of("aa", "b", "ccc")
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.mapping(String::toUpperCase, Collectors.toList())));
System.out.println(r);
```

- A) {B=1, AA=2, CCC=3}
- B) {1=[B], 2=[AA], 3=[CCC]}
- C) No compila
- D) {1=B, 2=AA, 3=CCC}

332. [peso 3] ¿Qué imprime?

```
var avg = IntStream.of(2, 4, 6)
    .average()
    .getAsDouble();
System.out.println(avg);
```

- A) OptionalDouble[4.0]
- B) 4
- C) 4.0
- D) 12.0

333. [peso 4] ¿Qué imprime?

```
var stats = IntStream.rangeClosed(1, 4).summaryStatistics();
System.out.println(stats.getMax() + "-" + stats.getSum());
```

- A) 4-6
- B) 10-4
- C) 3-6
- D) 4-10

334. [peso 3] ¿Qué método convierte Stream<String> en IntStream de longitudes?

```
Stream.of("a", "bb").???(String::length)
```

- A) map
- B) flatMapToInt
- C) mapToInt
- D) toInt

335. [peso 3] ¿Qué imprime?

```
Predicate<Integer> pos = n -> n > 0;  
Predicate<Integer> par = n -> n % 2 == 0;  
System.out.println(pos.and(par.negate()).test(3));
```

- A) true
- B) No compila
- C) false
- D) null

336. [peso 3] ¿Qué imprime?

```
UnaryOperator<String> up = String::toUpperCase;  
Function<String, Integer> len = String::length;  
System.out.println(up.andThen(len).apply("java"));
```

- A) java4
- B) No compila
- C) 4
- D) JAVA

337. [peso 4] ¿Qué imprime?

```
Function<Integer, Integer> f = x -> x + 1;  
Function<Integer, Integer> g = x -> x * 2;  
System.out.println(f.compose(g).apply(3));  
System.out.println(f.andThen(g).apply(3));
```

- A) 8 y 8
- B) 7 y 7
- C) 8 y 7
- D) 7 y 8

338. [peso 4] ¿Qué imprime?

```
var r = Stream.of(3, 1, 4, 1, 5)  
    .distinct()  
    .sorted()  
    .skip(1)  
    .limit(2)  
    .toList();  
System.out.println(r);
```

- A) [3, 5]
- B) [4, 5]
- C) [3, 4]
- D) [1, 3]

339. [peso 3] ¿Qué imprime?

```
var r = "hola mundo".chars()  
    .filter(Character::isLetter)  
    .count();  
System.out.println(r);
```

- A) 8
- B) 9
- C) 10
- D) No compila

340. [peso 5] ¿Qué problema tiene este código?

```
List<Integer> result = new ArrayList<>();  
IntStream.range(0, 10_000)  
    .parallel()  
    .forEach(result::add);
```

- A) No compila: forEach no acepta method references
- B) ArrayList no es thread-safe: resultados perdidos o excepciones intermitentes
- C) Siempre lanza ConcurrentModificationException al ejecutarse
- D) Ninguno: ArrayList soporta escritura concurrente seg.

341. [peso 4] ¿Qué garantiza forEachOrdered en un stream paralelo?

- A) Ejecución en un solo hilo
- B) Mayor velocidad
- C) Nada distinto de forEach
- D) El orden de encuentro

342. [peso 4] ¿Qué imprime?

```
var r = List.of("a", "b", "c").parallelStream()  
    .reduce("", String::concat);  
System.out.println(r);
```

- A) Siempre abc
- B) No compila
- C) cba
- D) Puede variar entre ejecuciones

343. [peso 3] ¿Qué imprime?

```
var r = Stream.concat(  
    Stream.of(1, 2),  
    Stream.of(3))  
    .mapToInt(Integer::intValue)  
    .max();  
System.out.println(r.getAsInt());
```

- A) 3
- B) 2
- C) OptionalInt.empty
- D) No compila

344. [peso 4] ¿Qué imprime?

```
var map = Map.of("a", 1, "b", 2);
var r = map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder()))
    .map(Map.Entry::getKey)
    .collect(Collectors.joining());
System.out.println(r);
```

- A) ba
- B) ab
- C) b
- D) 21

345. [peso 4] ¿Qué imprime?

```
var r = Stream.of(1, 2, 3, 4)
    .collect(Collectors.teeing(
        Collectors.counting(),
        Collectors.summingInt(n -> n),
        (count, sum) -> sum + "/" + count));
System.out.println(r);
```

- A) 4/10
- B) [10, 4]
- C) 10/4
- D) 2.5

346. [peso 3] ¿Qué imprime?

```
var r = Stream.of("x", "y")
    .map(s -> s.repeat(2))
    .toArray(String[]::new);
System.out.println(r[1]);
```

- A) No compila
- B) yy
- C) xx
- D) xy

347. [peso 4] ¿Qué imprime?

```
var r = Stream.of("uno", "dos", "tres")
    .takeWhile(s -> s.length() == 3)
    .toList();
System.out.println(r);
```

- A) [tres]
- B) [uno, dos]
- C) [uno, dos, tres]
- D) []

348. [peso 4] ¿Qué imprime?

```
var r = Stream.of(1, 2, 5, 2, 1)
    .dropWhile(n -> n < 3)
    .toList();
System.out.println(r);
```

- A) []
- B) [5]
- C) [5, 2, 1]
- D) [1, 2, 2, 1]

349. [peso 4] ¿Qué imprime?

```
Optional<Integer> r = Stream.of(5, 3, 8)
    .min(Comparator.naturalOrder());
r.ifPresentOrElse(
    v -> System.out.println("min=" + v),
    () -> System.out.println("vacío"));
```

- A) Optional[3]
- B) vacío
- C) min=3
- D) min=5

350. [peso 3] ¿Qué imprime?

```
var r = List.of("a", "b", "c").stream()
    .map(s -> s + "!")
    .skip(2)
    .findAny();
System.out.println(r.orElse("none"));
```

- A) a!
- B) none
- C) c!
- D) b!

351. [peso 5] ¿Cuál es el resultado?

```
var r = Stream.of("a", "b")
    .sorted((s1, s2) -> s1.length() - s2.length())
    .collect(Collectors.toMap(s -> s, s -> s, (a, b) -> a, TreeMap::new));
System.out.println(r.getClass().getSimpleName());
```

- A) TreeMap
- B) HashMap
- C) LinkedHashMap
- D) No compila

352. [peso 4] ¿Qué imprime?

```
long c = Stream.of("a", "b", "c")
    .peek(System.out::print)
    .count();
System.out.println(" " + c);
```

- A) abc 3 siempre, sin importar el resto del pipeline
- B) Error de compilación por el orden de las operaciones
- C) c 3, porque limit corta el conteo también
- D) Puede imprimir solo '3': peek es omitible si count no necesita recorrer

353. [peso 3] ¿Qué imprime?

```
var nums = List.of(1, 2, 3, 4, 5, 6);
var r = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();
System.out.println(r);
```

- A) 56
- B) 14
- C) 91
- D) 20

354. [peso 4] ¿Qué imprime?

```
record Emp(String dept, double sal) {}
var r = Stream.of(new Emp("IT", 1000), new Emp("IT", 3000), new Emp("HR", 2000))
    .collect(Collectors.groupingBy(Emp::dept,
        Collectors.averagingDouble(Emp::sal)));
System.out.println(r.get("IT"));
```

- A) 4000.0
- B) [1000.0, 3000.0]
- C) 1000.0
- D) 2000.0

355. [peso 4] ¿Qué imprime?

```
var r = Stream.of("java", "sql", "api")
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        List::size));
System.out.println(r);
```

- A) List
- B) 3
- C) No compila
- D) [java, sql, api]

356. [peso 3] ¿Qué imprime?

```
int[] nums = {1, 2, 3};  
System.out.println(Arrays.stream(nums).boxed().toList());
```

- A) [1, 2, 3]
- B) IntStream
- C) No compila
- D) 1 2 3

357. [peso 4] ¿Qué imprime?

```
var r = Stream.of("a,b", "c,d")  
    .flatMap(s -> Arrays.stream(s.split(",")))  
    .count();  
System.out.println(r);
```

- A) 2
- B) 3
- C) 4
- D) 1

358. [peso 4] ¿Qué imprime?

```
var opt = Optional.of("data")  
    .filter(s -> s.length() > 10);  
System.out.println(opt.isPresent());
```

- A) false
- B) No compila
- C) true
- D) NoSuchElementException

359. [peso 4] ¿Qué imprime?

```
Optional<Optional<String>> nested = Optional.of(Optional.of("x"));  
var flat = nested.flatMap(o -> o);  
System.out.println(flat.get());
```

- A) Optional[Optional[x]]
- B) x
- C) Optional[x]
- D) NoSuchElementException

360. [peso 3] ¿Qué imprime?

```
BiFunction<Integer, Integer, Integer> f = (a, b) -> a * b;  
Function<Integer, String> g = n -> "R:" + n;  
System.out.println(f.andThen(g).apply(3, 4));
```

- A) 12
- B) R:34
- C) No compila
- D) R:12

361. [peso 4] ¿Qué imprime?

```
var words = List.of("stream", "map", "filter");  
var r = words.stream()  
    .collect(Collectors.toMap(  
        String::length, s -> s, (a, b) -> a + "|" + b));  
System.out.println(r.get(6));
```

- A) stream|filter
- B) filter
- C) IllegalStateException
- D) stream

362. [peso 3] ¿Qué imprime?

```
System.out.println(  
    Stream.empty().findFirst().orElse("default"));
```

- A) Optional.empty
- B) NoSuchElementException
- C) null
- D) default

363. [peso 4] ¿Qué imprime?

```
var r = Stream.of("b", "a", "c")  
    .sorted()  
    .peek(s -> System.out.print(s))  
    .limit(2)  
    .toList();
```

- A) ab
- B) abc
- C) ba
- D) bac

364. [peso 4] ¿Cuál es el resultado?

```
var r = Stream.of("a", null, "b")  
    .toList();
```

- A) [a, null, b]
- B) No compila
- C) [a, b]
- D) NullPointerException

365. [peso 4] ¿Qué imprime?

```
var r = IntStream.range(0, 5)  
    .mapToObj(i -> i % 2 == 0 ? "P" : "I")  
    .collect(Collectors.joining());  
System.out.println(r);
```

- A) IPIPI
- B) PIPIP
- C) PPPII
- D) PIPI

366. [peso 5] ¿Qué imprime este pipeline con infinito ordenado?

```
var r = Stream.iterate(5, n -> n - 1)
    .filter(n -> n < 3)
    .findFirst();
System.out.println(r.get());
```

- A) NoSuchElementException
- B) 5
- C) Bucle infinito
- D) 2

367. [peso 4] ¿Qué imprime?

```
var m = Stream.of("apple", "kiwi", "fig")
    .collect(Collectors.groupingBy(String::length,
        TreeMap::new, Collectors.toList()));
System.out.println(m.firstKey());
```

- A) 4
- B) fig
- C) 5
- D) 3

368. [peso 3] ¿Qué imprime?

```
var r = Stream.of("one", "two", "three")
    .max(Comparator.comparingInt(String::length)
        .orElse("-"));
System.out.println(r);
```

- A) one, el primero encontrado en el stream
- B) -, porque el stream llega vacío al comparador
- C) two, por tener longitud intermedia
- D) three

369. [peso 4] ¿Qué imprime?

```
var counts = Stream.of("a", "b", "a", "c", "a")
    .collect(Collectors.groupingBy(s -> s, Collectors.counting()));
System.out.println(counts.get("a"));
```

- A) 2
- B) 3L pero como Integer
- C) 3
- D) [a, a, a]

370. [peso 4] ¿Qué imprime?

```
var list = new ArrayList<>(Arrays.asList("b", null, "a"));
list.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
System.out.println(list);
```

- A) [null, a, b]
- B) [a, b, null]
- C) [null, b, a]
- D) NullPointerException

371. [peso 3] ¿Qué imprime?

```
var r = Stream.of(2, 4, 6)
    .noneMatch(n -> n % 2 != 0);
System.out.println(r);
```

- A) Optional[true]
- B) No compila
- C) true
- D) false

372. [peso 4] ¿Qué imprime?

```
record Prod(String cat, int qty) {}
var total = Stream.of(new Prod("A", 2), new Prod("B", 3), new Prod("A", 5))
    .filter(p -> p.cat().equals("A"))
    .mapToInt(Prod::qty)
    .sum();
System.out.println(total);
```

- A) 10
- B) 7
- C) 5
- D) 2

373. [peso 5] ¿Qué imprime?

```
var sb = Stream.of("a", "b", "c")
    .collect(StringBuilder::new,
        StringBuilder::append,
        StringBuilder::append);
System.out.println(sb);
```

- A) No compila
- B) cba
- C) [a, b, c]
- D) abc

374. [peso 4] ¿Qué imprime?

```
var r = Stream.of("aa", "ab", "ba")
    .filter(s -> s.startsWith("a"))
    .map(s -> s.charAt(1))
    .distinct()
    .count();
System.out.println(r);
```

- A) 3
- B) 1
- C) 2
- D) 0

375. [peso 4] ¿Qué imprime?

```
var r = Optional.ofNullable((String) null)
    .or(() -> Optional.of("backup"))
    .map(String::length)
    .orElse(-1);
System.out.println(r);
```

- A) NullPointerException
- B) -1
- C) 6
- D) 0

JDBC (32 preguntas)

376. [peso 3] ¿Qué problema tiene este código?

```
String user = request.get("user");
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(
    "SELECT * FROM accounts WHERE name = '" + user + "'");
```

- A) Ninguno, siempre que se use PreparedStatement después
- B) Falta hacer commit tras ejecutar la consulta
- C) Inyección SQL: la entrada se concatena sin sanitizar
- D) No compila por el uso de Statement en vez de PreparedStatement

377. [peso 3] ¿Cómo se asignan los parámetros de un PreparedStatement?

```
var ps = conn.prepareStatement(
    "SELECT * FROM emp WHERE dept = ? AND sal > ?");
```

- A) ps.setString(0, "IT"); ps.setDouble(1, 500)
- B) ps.bind(1, "IT")
- C) ps.setString(1, "IT")
- D) ps.set("dept", "IT")

378. [peso 4] ¿Qué imprime si la tabla tiene 3 filas?

```
ResultSet rs = ps.executeQuery();
rs.next();
int count = 0;
while (rs.next()) count++;
System.out.println(count);
```

- A) 2
- B) 3
- C) 4
- D) SQLException

379. [peso 3] ¿Qué método ejecuta un UPDATE y qué devuelve?

```
var ps = conn.prepareStatement(  
    "UPDATE emp SET sal = sal * 1.1 WHERE dept = ?");
```

- A) execute() → void, sin importar el tipo de sentencia
- B) run() → boolean, indicando éxito o fracaso
- C) executeUpdate() → int con filas afectadas
- D) executeQuery() → ResultSet, como cualquier consulta

380. [peso 4] ¿Cuál es el resultado?

```
var rs = st.executeQuery("SELECT id, name FROM emp");  
while (rs.next()) {  
    System.out.println(rs.getInt("ID") + rs.getString(2));  
}
```

- A) SQLException por mezclar el índice 2 con nombre de columna
- B) SQLException por usar "ID" en mayúsculas en el getInt
- C) No compila porque faltan los tipos explícitos
- D) Funciona: nombres de columna case-insensitive e índices desde 1

381. [peso 4] ¿Qué imprime si la columna BONUS es NULL en la fila actual?

```
int bonus = rs.getInt("bonus");  
System.out.println(bonus + " " + rs.wasNull());
```

- A) null true
- B) 0 true
- C) SQLException
- D) 0 false

382. [peso 4] ¿Qué ocurre con los datos?

```
conn.setAutoCommit(false);  
st.executeUpdate("INSERT INTO log VALUES (1)");  
st.executeUpdate("INSERT INTO log VALUES (2)");  
conn.rollback();  
st.executeUpdate("INSERT INTO log VALUES (3)");  
conn.commit();
```

- A) Solo 1 y 2
- B) No se guarda nada
- C) Se guardan 1, 2 y 3
- D) Solo se guarda 3

383. [peso 5] ¿Qué hace este código con savepoints?

```
conn.setAutoCommit(false);  
st.executeUpdate("INSERT INTO t VALUES (1)");  
Savepoint sp = conn.setSavepoint();  
st.executeUpdate("INSERT INTO t VALUES (2)");  
conn.rollback(sp);  
conn.commit();
```

- A) Guarda 1 y 2
- B) SQLException
- C) No guarda nada
- D) Guarda solo 1

384. [peso 3] ¿Qué pasa con el ResultSet al cerrar su Statement?

```
ResultSet rs;
try (var st = conn.createStatement()) {
    rs = st.executeQuery("SELECT 1 FROM dual");
}
rs.next();
```

- A) SQLException: el ResultSet se cierra junto con su Statement
- B) Devuelve false silenciosamente en cada next()
- C) No compila si el ResultSet se usa fuera del try
- D) Funciona con normalidad, el ResultSet sigue abierto

385. [peso 4] ¿Cómo se lee el parámetro OUT de este procedimiento?

```
var cs = conn.prepareCall("{call get_total(?, ?)}");
cs.setInt(1, 2024);
// el segundo parámetro es OUT de tipo NUMBER
```

- A) cs.setOut(2) con el valor esperado de retorno
- B) cs.getInt(2) directamente sin registrar el parámetro
- C) cs.registerOutParameter(2, Types.NUMERIC) antes de execute, luego cs.getInt(2)
- D) No es posible leer parámetros OUT en JDBC estándar

386. [peso 4] ¿Qué logra este código?

```
var ps = conn.prepareStatement("INSERT INTO emp(name) VALUES (?)");
for (String n : names) {
    ps.setString(1, n);
    ps.addBatch();
}
int[] results = ps.executeBatch();
```

- A) Agrupa los INSERT en un solo envío por lotes, reduciendo round-trips
- B) Solo inserta el último elemento del bucle
- C) Lanza SQLException por usar addBatch repetidamente
- D) Ejecuta un INSERT distinto por cada viaje a la BD

387. [peso 4] ¿Qué imprime con un ResultSet scrollable de 5 filas?

```
rs.absolute(3);
rs.relative(-1);
System.out.println(rs.getInt(1)); // columna con valores 10,20,30,40,50
```

- A) 20
- B) 40
- C) SQLException
- D) 30

388. [peso 3] ¿Qué información aporta getErrorCode() en una SQLException con Oracle?

- A) Nada útil
- B) El código HTTP
- C) El código de error del vendor
- D) La línea del error en Java

389. [peso 4] ¿Cómo se recuperan las claves generadas por un INSERT?

```
var ps = conn.prepareStatement(sql, ???);  
ps.executeUpdate();  
ResultSet keys = ps.getGeneratedKeys();
```

- A) Statement.KEEP_CURRENT_RESULT
- B) ResultSet.TYPE_KEYS
- C) No se puede
- D) Statement.RETURN_GENERATED_KEYS

390. [peso 4] ¿Qué imprime este metadata?

```
var rs = st.executeQuery("SELECT id, name FROM emp");  
var md = rs.getMetaData();  
System.out.println(md.getColumnCount() + " " + md.getColumnName(2));
```

- A) 2 ID
- B) SQLException
- C) 2 NAME
- D) 1 NAME

391. [peso 4] ¿Cuál es el error de este patrón?

```
var ps = conn.prepareStatement("SELECT * FROM emp WHERE id = ?");  
ps.setInt(1, 5);  
ps.executeQuery("SELECT * FROM emp"); // línea X
```

- A) Ninguno, el patrón es válido tal como está escrito
- B) Falta cerrar el PreparedStatement al finalizar
- C) La línea X: executeQuery(String) sobre PreparedStatement lanza SQLException
- D) El parámetro ps.setInt no se asignó correctamente

392. [peso 3] ¿Cómo se obtiene una conexión con DriverManager?

```
Connection conn = DriverManager.getConnection(???);
```

- A) Solo la URL, o URL + usuario + password
- B) Un DataSource
- C) Un objeto Properties únicamente
- D) El nombre de la clase del driver

393. [peso 4] ¿Qué imprime si emp está vacía?

```
var rs = st.executeQuery("SELECT COUNT(*) FROM emp");  
if (rs.next())  
    System.out.println(rs.getInt(1));  
else  
    System.out.println("sin filas");
```

- A) SQLException
- B) 0
- C) null
- D) sin filas

394. [peso 4] ¿Cuál es el riesgo de este código en un pool de conexiones?

```
Connection c = dataSource.getConnection();
c.setAutoCommit(false);
// ... operaciones y commit ...
// (no se restaura autoCommit ni se cierra en finally)
```

- A) Fuga de conexión si hay excepción, y estado sucio
- B) Ninguno: el pool siempre resetea el estado
- C) No compila al omitir el cierre en finally
- D) El commit falla si autoCommit quedó en false

395. [peso 4] ¿Qué imprime este código con getObject?

```
// columna HIRE_DATE de tipo DATE en Oracle
LocalDate d = rs.getObject("hire_date", LocalDate.class);
System.out.println(d.getClass().getSimpleName());
```

- A) Date
- B) LocalDate
- C) Timestamp
- D) SQLException siempre

396. [peso 4] En un batch, ¿qué indica Statement.EXECUTE_FAILED en el arreglo de resultados?

```
int[] r = ps.executeBatch();
```

- A) Que se necesita commit
- B) Que ese comando específico falló
- C) Que todo el batch se ejecutó
- D) Que devolvió filas

397. [peso 5] ¿Qué imprime?

```
conn.setAutoCommit(false);
try (var ps = conn.prepareStatement("UPDATE acc SET bal = bal - 100 WHERE id = 1")) {
    ps.executeUpdate();
    throw new RuntimeException("fallo");
} catch (Exception e) {
    conn.rollback();
    System.out.println("revertido");
}
```

- A) Nada, porque el bloque nunca llega a ejecutarse
- B) SQLException al intentar hacer rollback
- C) revertido, y el UPDATE queda deshecho
- D) revertido, pero el UPDATE persiste igualmente

398. [peso 4] ¿Qué devuelve executeUpdate para un DELETE que no coincide con ninguna fila?

```
int n = st.executeUpdate("DELETE FROM emp WHERE id = -999");
```

- A) null
- B) -1
- C) SQLException
- D) 0

399. [peso 4] ¿Qué hace este código con un ResultSet CONCUR_UPDATABLE?

```
rs.next();  
rs.updateDouble("sal", 5000);  
rs.updateRow();
```

- A) SQLException siempre, sin importar la concurrencia
- B) Modifica la columna en la fila actual y persiste el cambio en la BD
- C) Solo cambia la copia en memoria del ResultSet
- D) Nada: los ResultSet son siempre de solo lectura

400. [peso 3] ¿Qué falta para que este SELECT parametrizado funcione?

```
var ps = conn.prepareStatement("SELECT * FROM emp WHERE id = ?");  
var rs = ps.executeQuery();
```

- A) Nada, el SELECT ya está completo y listo
- B) Registrar el parámetro como OUT antes de ejecutar
- C) Llamar a un método ps.compile() antes de ejecutar
- D) Asignar el parámetro con ps.setInt(1, valor) antes de ejecutar

401. [peso 4] ¿Qué imprime?

```
try (var rs = st.executeQuery("SELECT 'A' FROM dual")) {  
    System.out.println(rs.isBeforeFirst());  
    rs.next();  
    System.out.println(rs.isFirst() + " " + rs.isLast());  
}
```

- A) SQLException
- B) true, luego true false
- C) true, luego true true
- D) false, luego true false

402. [peso 4] ¿Cuál es el resultado con una columna NUMBER(10,2)?

```
BigDecimal bd = rs.getBigDecimal("price");  
double d = rs.getDouble("price");
```

- A) Ambos lanzan excepción
- B) Ambos funcionan
- C) Solo getDouble funciona
- D) Solo getBigDecimal funciona

403. [peso 3] ¿Cómo se invoca una función almacenada que DEVUELVE un valor?

```
var cs = conn.prepareCall(???)
```

- A) "{call fn(?)}", igual que un procedimiento
- B) No es posible invocar funciones desde JDBC
- C) "select fn from dual" únicamente, sin bind
- D) "{? = call fn(?)}" registrando el parámetro 1 como OUT

404. [peso 5] ¿Qué problema de concurrencia tiene este código?

```
// campo compartido en un servlet/bean singleton
private Connection conn = ds.getConnection();

public List<Emp> find(int id) throws SQLException {
    var ps = conn.prepareStatement("SELECT * FROM emp WHERE id = ?");
    ...
}
```

- A) Connection no es segura para uso concurrente: múltiples hilos comparten transacción y estado, causando errores intermitentes
- B) No compila al intentar compartir la misma conexión entre varios hilos
- C) Solo es más lento por la contención, pero el resultado sigue siendo correcto
- D) Ninguno, Connection está diseñada específicamente para ese uso compartido

405. [peso 4] ¿Qué imprime?

```
var ps = conn.prepareStatement("SELECT ename FROM emp WHERE ename LIKE ?");
ps.setString(1, "S%");
// tabla: SMITH, SCOTT, ADAMS
var rs = ps.executeQuery();
int c = 0;
while (rs.next()) c++;
System.out.println(c);
```

- A) 1
- B) 2
- C) 0
- D) 3

406. [peso 4] ¿Qué imprime?

```
System.out.println(conn.getTransactionIsolation() ==
    Connection.TRANSACTION_READ_COMMITTED);
```

- A) false siempre, sin importar la configuración
- B) Depende del ResultSet abierto en ese momento
- C) SQLException al consultar el nivel de aislamiento
- D) true en Oracle con configuración por defecto

407. [peso 4] ¿Qué patrón muestra este código y qué previene?

```
var sql = "UPDATE emp SET sal = ? WHERE id = ? AND version = ?";
ps.setDouble(1, newSal);
ps.setInt(2, id);
ps.setInt(3, expectedVersion);
if (ps.executeUpdate() == 0)
    throw new OptimisticLockException();
```

- A) Un patrón de cache de segundo nivel para lecturas
- B) Bloqueo pesimista: bloquea la fila desde el inicio
- C) Bloqueo optimista: detecta modificaciones concurrentes comparando versión, sin mantener locks
- D) Un mecanismo de auditoría de cambios en la tabla

408. [peso 3] ¿Por qué falla esta consulta?

```
SELECT ename, sal * 12 AS annual
FROM emp
WHERE annual > 50000;
```

- A) La columna sal no puede multiplicarse por un número
- B) Falta agregar una cláusula GROUP BY a la consulta
- C) No falla, la consulta se ejecuta con normalidad
- D) El alias annual no existe aún cuando se evalúa WHERE

409. [peso 3] ¿Qué devuelve?

```
SELECT NVL(comm, 0) + sal FROM emp WHERE empno = 7369;
-- comm es NULL, sal = 800
```

- A) 800
- B) Error
- C) NULL
- D) 0

410. [peso 4] ¿Qué filas devuelve?

```
SELECT ename FROM emp
WHERE comm != NULL;
```

- A) Ninguna: la comparación con NULL nunca es verdadera
- B) Las filas que sí tienen una comisión asignada
- C) Error de sintaxis por comparar con NULL
- D) Las filas que no tienen ninguna comisión asignada

411. [peso 4] Si dept 40 no tiene empleados, ¿qué devuelve?

```
SELECT d.dname, COUNT(e.empno)
FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno
GROUP BY d.dname;
```

- A) Incluye dept 40 con COUNT = 0
- B) Incluye dept 40 con COUNT = NULL
- C) Error
- D) Omite el dept 40

412. [peso 3] ¿Qué devuelve con sal de 800 a 5000?

```
SELECT COUNT(*) FROM emp
WHERE sal BETWEEN 800 AND 1000;
-- salarios: 800, 950, 1000, 1100
```

- A) 4
- B) 1
- C) 2
- D) 3

413. [peso 4] ¿Qué error tiene esta consulta?

```
SELECT deptno, ename, AVG(sal)
FROM emp
GROUP BY deptno;
```

- A) Falta agregar una cláusula HAVING a la consulta
- B) AVG no acepta la columna sal como argumento
- C) ename no está en el GROUP BY ni es agregada: ORA-00979
- D) Ninguno, la consulta es válida tal cual

414. [peso 4] ¿Qué devuelve?

```
SELECT deptno, MAX(sal)
FROM emp
GROUP BY deptno
HAVING MAX(sal) > 2900
ORDER BY 2 DESC;
```

- A) Solo los departamentos cuyo salario máximo supera 2900, del mayor al menor
- B) Error: ORDER BY 2 no es una cláusula válida
- C) Los empleados individuales con salario mayor a 2900
- D) Todos los departamentos, sin excepción alguna

415. [peso 4] ¿Qué devuelve UNION frente a UNION ALL aquí?

```
SELECT deptno FROM emp      -- 14 filas, depts 10,20,30
UNION
SELECT deptno FROM dept;    -- 4 filas, depts 10,20,30,40
```

- A) 4 filas (elimina duplicados)
- B) Error de tipos al combinar ambas consultas
- C) 14 filas, sin considerar los duplicados
- D) 18 filas, sumando ambos orígenes

416. [peso 4] ¿Qué empleados devuelve?

```
SELECT ename FROM emp e
WHERE EXISTS (
    SELECT 1 FROM emp m
    WHERE m.mgr = e.empno);
```

- A) Los que tienen un jefe asignado
- B) Ninguno, la subconsulta nunca es verdadera
- C) Los que SON jefes de alguien
- D) Todos los empleados de la tabla

417. [peso 5] ¿Qué devuelve si algún deptno de la subconsulta es NULL?

```
SELECT ename FROM emp
WHERE deptno NOT IN (
    SELECT deptno FROM dept_cerrados);
```

- A) Ignora silenciosamente el valor NULL de la lista
- B) Error de sintaxis por usar NOT IN con subconsulta
- C) Los empleados de departamentos que no están cerrados
- D) NINGUNA fila: NOT IN con un NULL en la lista siempre es UNKNOWN

418. [peso 4] ¿Qué hace esta consulta?

```
SELECT ename, sal, d.avg_sal  
FROM emp e  
JOIN (SELECT deptno, AVG(sal) avg_sal  
      FROM emp GROUP BY deptno) d  
  ON e.deptno = d.deptno  
WHERE e.sal > d.avg_sal;
```

- A) Empleados bajo el promedio global
- B) Error: no se puede unir con subconsulta
- C) Empleados que ganan más que el promedio de SU departamento
- D) Promedios por departamento

419. [peso 4] ¿Qué devuelve ROWNUM aquí?

```
SELECT ename FROM (  
  SELECT ename FROM emp ORDER BY sal DESC)  
WHERE ROWNUM <= 3;
```

- A) Error
- B) 3 empleados aleatorios
- C) Los 3 peor pagados
- D) Los 3 mejor pagados

420. [peso 3] ¿Qué devuelve?

```
SELECT SUBSTR('ORACLE', -3, 2) FROM dual;
```

- A) CL
- B) ACL
- C) LE
- D) CLE

421. [peso 3] ¿Qué devuelve?

```
SELECT INSTR('MISSISSIPPI', 'S', 1, 3) FROM dual;
```

- A) 7
- B) Error
- C) 6
- D) 3

422. [peso 4] ¿Qué devuelve?

```
SELECT TO_CHAR(1234.5, '00000.00') FROM dual;
```

- A) 1234.50
- B) #####
- C) 01234.50
- D) 1234.5

423. [peso 4] ¿Cuál es el resultado?

```
SELECT TO_DATE('2024-02-30', 'YYYY-MM-DD') FROM dual;
```

- A) ORA-01839: fecha no válida para el mes
- B) NULL, porque TO_DATE tolera fechas inexistentes
- C) 29-FEB-2024, tomando el último día válido del mes
- D) 01-MAR-2024, ajustando automáticamente el día

424. [peso 3] ¿Qué devuelve?

```
SELECT MONTHS_BETWEEN(  
    DATE '2024-03-15', DATE '2024-01-15') FROM dual;
```

- A) 2
- B) -2
- C) 60
- D) 2.03

425. [peso 4] ¿Qué evalúa este CASE?

```
SELECT ename,  
    CASE  
        WHEN sal >= 3000 THEN 'ALTO'  
        WHEN sal >= 1500 THEN 'MEDIO'  
        ELSE 'BAJO'  
    END nivel  
FROM emp WHERE sal = 3000;
```

- A) ALTO
- B) BAJO
- C) MEDIO
- D) Ambos ALTO y MEDIO

426. [peso 4] ¿Qué permite ON DELETE CASCADE en una FK?

```
ALTER TABLE emp ADD CONSTRAINT fk_dept  
FOREIGN KEY (deptno) REFERENCES dept(deptno)  
ON DELETE CASCADE;
```

- A) Borra el departamento al borrar el último empleado
- B) Convierte los deptno en NULL
- C) Impide borrar departamentos
- D) Al borrar un departamento se borran automáticamente sus empleados

427. [peso 3] ¿Cuál es la diferencia entre DELETE y TRUNCATE?

- A) DELETE es más rápido en tablas grandes
- B) Ninguna
- C) DELETE es DML
- D) TRUNCATE acepta WHERE

428. [peso 4] ¿Qué calcula esta función analítica?

```
SELECT ename, deptno, sal,  
       RANK() OVER (PARTITION BY deptno ORDER BY sal DESC) rk  
FROM emp;
```

- A) El ranking global de todos los salarios sin partición
- B) Un número secuencial sin empates ni huecos
- C) El ranking de salario DENTRO de cada departamento, con huecos ante empates
- D) El promedio de salario calculado por departamento

429. [peso 4] Con salarios 5000, 3000, 3000, 2975: ¿qué devuelve DENSE_RANK para 2975?

```
SELECT sal, DENSE_RANK() OVER (ORDER BY sal DESC) dr FROM emp;
```

- A) 1
- B) 4
- C) 2
- D) 3

430. [peso 4] ¿Qué obtiene LAG aquí?

```
SELECT ename, hiredate,  
       LAG(ename) OVER (ORDER BY hiredate) anterior  
FROM emp;
```

- A) Error, porque LAG requiere PARTITION BY
- B) El empleado contratado inmediatamente antes
- C) El siguiente empleado contratado después
- D) El primer empleado contratado de todos

431. [peso 5] ¿Qué calcula esta ventana?

```
SELECT ename, sal,  
       SUM(sal) OVER (ORDER BY hiredate  
                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) acum  
FROM emp;
```

- A) El total global repetido
- B) La suma acumulada
- C) El promedio móvil
- D) La suma por departamento

432. [peso 4] ¿Por qué falla?

```
SELECT ename, sal,  
       RANK() OVER (ORDER BY sal DESC) rk  
FROM emp  
WHERE rk <= 3;
```

- A) Falta agregar PARTITION BY a la función RANK
- B) No falla, la consulta se ejecuta con normalidad
- C) RANK no acepta la cláusula DESC en su ORDER BY
- D) Las funciones analíticas se evalúan después del WHERE: rk no existe ahí

433. [peso 5] ¿Qué genera esta consulta jerárquica?

```
SELECT LPAD(' ', 2*(LEVEL-1)) || ename org
FROM emp
START WITH mgr IS NULL
CONNECT BY PRIOR empno = mgr;
```

- A) El organigrama: árbol jefe→subordinados con indentación por nivel
- B) Error de sintaxis por usar CONNECT BY PRIOR
- C) Solo la lista de gerentes de la compañía
- D) Una lista plana de todos los empleados sin orden

434. [peso 4] ¿Qué hace esta CTE recursiva?

```
WITH nums (n) AS (
  SELECT 1 FROM dual
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 5
)
SELECT SUM(n) FROM nums;
```

- A) 15
- B) 10
- C) 5
- D) Bucle infinito

435. [peso 3] ¿Qué devuelve?

```
SELECT COALESCE(NULL, NULL, 'A', 'B') FROM dual;
```

- A) B
- B) NULL
- C) Error
- D) A

436. [peso 4] ¿Qué devuelve?

```
SELECT NULLIF(10, 10), NULLIF(10, 20) FROM dual;
```

- A) 10 y NULL
- B) NULL y NULL
- C) NULL y 10
- D) 10 y 20

437. [peso 4] ¿Cuántas filas devuelve un CROSS JOIN entre emp (14 filas) y dept (4 filas)?

```
SELECT * FROM emp CROSS JOIN dept;
```

- A) 56
- B) 4
- C) 14
- D) 18

438. [peso 4] ¿Qué empleados lista este self-join?

```
SELECT e.ename, m.ename jefe
FROM emp e JOIN emp m
ON e.mgr = m.empno;
```

- A) Cada empleado con su jefe
- B) Solo los jefes
- C) Todos, incluido el presidente
- D) Error: no se puede unir una tabla consigo misma

439. [peso 4] ¿Qué ve la Sesión B?

```
-- Sesión A:
UPDATE emp SET sal = 9999 WHERE empno = 7369;
-- (sin commit)
-- Sesión B:
SELECT sal FROM emp WHERE empno = 7369;
```

- A) Se bloquea esperando a que la Sesión A confirme
- B) 9999, el valor recién actualizado por la Sesión A
- C) Error de lock al intentar leer la fila
- D) El valor original: Oracle da consistencia de lectura sin bloquear lecturas

440. [peso 4] ¿Qué hace FOR UPDATE en un SELECT?

```
SELECT * FROM emp
WHERE deptno = 10
FOR UPDATE NOWAIT;
```

- A) Actualiza directamente las filas seleccionadas
- B) Crea un trigger temporal sobre la tabla
- C) Bloquea las filas seleccionadas para la transacción
- D) No hace nada especial en la sesión actual

441. [peso 4] ¿Qué diferencia práctica muestra esto?

```
CREATE TABLE t (a CHAR(5), b VARCHAR2(5));
INSERT INTO t VALUES ('ab', 'ab');
SELECT LENGTH(a), LENGTH(b) FROM t;
```

- A) 2 y 5, invirtiendo el comportamiento esperado
- B) 5 y 2: CHAR rellena con espacios hasta la longitud fija
- C) Error de tipos al comparar CHAR con VARCHAR2
- D) 2 y 2, ya que ambos tipos truncan por igual

442. [peso 3] ¿Qué devuelve?

```
SELECT CEIL(10.1), FLOOR(10.9), ROUND(10.45, 1), TRUNC(10.45, 1)
FROM dual;
```

- A) 11, 10, 10.5, 10.4
- B) 11, 10, 10.4, 10.4
- C) 10, 11, 10.4, 10.5
- D) 11, 11, 10.5, 10.4

443. [peso 4] ¿Qué logra esta expresión regular?

```
SELECT ename FROM emp
WHERE REGEXP_LIKE(ename, '^[AEIOU].*$');
```

- A) Error de sintaxis
- B) Nombres que contienen vocales
- C) Nombres que EMPIEZAN con vocal y TERMINAN en S
- D) Nombres con exactamente 2 letras

444. [peso 5] ¿Cuántas filas de agrupación genera CUBE(deptno, job) con 3 deptnos y 5 jobs (todas las combinaciones pobladas)?

```
SELECT deptno, job, SUM(sal)
FROM emp GROUP BY CUBE(deptno, job);
```

- A) 16, una fila extra por cada valor de job
- B) 24: 15 combinaciones + 3 subtotales de dept + 5 de job + 1 total
- C) 8, contando únicamente los subtotales generados
- D) 15, solo las combinaciones de deptno y job

445. [peso 4] ¿Qué hace esta sentencia?

```
INSERT ALL
  INTO ventas_2023 VALUES (id, monto)
  INTO ventas_hist VALUES (id, monto)
SELECT id, monto FROM ventas_staging;
```

- A) Crea las tablas
- B) Inserta cada fila de la fuente en AMBAS tablas
- C) Da error: INSERT no acepta dos INTO
- D) Inserta solo en la primera tabla

446. [peso 4] ¿Qué devuelve CURRVAL si en la sesión aún no se llamó NEXTVAL?

```
SELECT seq_emp.CURRVAL FROM dual;
```

- A) NULL, sin lanzar ningún error
- B) ORA-08002: CURRVAL aún no está definido en esta sesión
- C) El último valor global usado por la secuencia
- D) 0, como valor inicial por defecto

447. [peso 4] ¿Qué muestra esta consulta de diccionario?

```
SELECT table_name, num_rows
FROM user_tables
WHERE table_name = 'EMP';
```

- A) Error: user_tables no existe en el diccionario
- B) Las columnas y tipos de datos de EMP
- C) El conteo exacto y en tiempo real de filas
- D) El número de filas según las ÚLTIMAS ESTADÍSTICAS recolectadas

448. [peso 3] ¿Qué privilegio necesita el usuario APP para leer la tabla HR.EMP?

```
GRANT ??? ON hr.emp TO app;
```

- A) CONNECT
- B) ALL PRIVILEGES obligatorio
- C) INSERT
- D) SELECT

449. [peso 4] ¿Qué resultado produce?

```
SELECT deptno,  
       LISTAGG(ename, ', ') WITHIN GROUP (ORDER BY ename) equipo  
FROM emp  
GROUP BY deptno;
```

- A) Error: LISTAGG no existe en Oracle SQL
- B) Una fila individual por cada empleado listado
- C) El conteo de empleados agrupado por departamento
- D) Una fila por departamento con los nombres concatenados y ordenados

PL/SQL (38 preguntas)

450. [peso 3] ¿Qué imprime?

```
DECLARE  
    v_num NUMBER := 10;  
BEGIN  
    v_num := v_num + 5;  
    DBMS_OUTPUT.PUT_LINE(v_num);  
END;
```

- A) 15
- B) 10
- C) Error: falta SERVEROUTPUT
- D) 5

451. [peso 3] ¿Qué contiene v_emp tras el fetch?

```
DECLARE  
    v_emp emp%ROWTYPE;  
BEGIN  
    SELECT * INTO v_emp FROM emp WHERE empno = 7369;  
    DBMS_OUTPUT.PUT_LINE(v_emp.ename);  
END;
```

- A) Error de tipos al usar %ROWTYPE con SELECT INTO
- B) Un registro con TODAS las columnas de la fila
- C) Solo el nombre del empleado consultado
- D) Un cursor abierto listo para hacer FETCH

452. [peso 4] ¿Qué excepción lanza si el empno no existe?

```
SELECT sal INTO v_sal FROM emp WHERE empno = -1;
```

- A) Ninguna: v_sal queda NULL
- B) NO_DATA_FOUND
- C) TOO_MANY_ROWS
- D) INVALID_NUMBER

453. [peso 4] ¿Qué imprime?

```
BEGIN
  BEGIN
    RAISE VALUE_ERROR;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('interno');
  END;
  DBMS_OUTPUT.PUT_LINE('sigue');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('externo');
END;
```

- A) interno y sigue
- B) interno y externo
- C) sigue
- D) externo

454. [peso 4] ¿Qué hacen SQLCODE y SQLERRM dentro de un manejador?

```
EXCEPTION
  WHEN OTHERS THEN
    log_error(SQLCODE, SQLERRM);
    RAISE;
```

- A) Limpian el error para que no se propague más
- B) Hacen un rollback automático de la transacción
- C) Devuelven el código numérico y el mensaje del error actual
- D) Detienen la ejecución del bloque PL/SQL inmediatamente

455. [peso 4] ¿Qué logra RAISE_APPLICATION_ERROR?

```
IF v_sal < 0 THEN
  RAISE_APPLICATION_ERROR(-20001, 'Salario inválido');
END IF;
```

- A) Lanza una excepción personalizada con código entre -20000 y -20999 que llega al cliente con ese mensaje
- B) Solo escribe el error en una tabla de log interna de la aplicación
- C) Reinicia por completo la sesión actual de la base de datos
- D) Lanza solamente un warning visible en el log de la sesión

456. [peso 3] ¿Cuántas iteraciones ejecuta?

```
BEGIN
  FOR i IN REVERSE 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
```

- A) 3, imprimiendo 1, 2, 3
- B) 3, imprimiendo 3, 2, 1
- C) Infinitas
- D) 0: REVERSE invierte los límites

457. [peso 4] ¿Qué imprime?

```
DECLARE
  v_i NUMBER := 0;
BEGIN
  LOOP
    v_i := v_i + 1;
    EXIT WHEN v_i >= 3;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_i);
END;
```

- A) 2
- B) Bucle infinito
- C) 3
- D) 4

458. [peso 4] ¿Qué imprime tras un UPDATE que afectó 3 filas?

```
UPDATE emp SET sal = sal * 1.05 WHERE deptno = 20;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
IF SQL%NOTFOUND THEN
  DBMS_OUTPUT.PUT_LINE('sin cambios');
END IF;
```

- A) NULL
- B) 3
- C) 0
- D) 3 y sin cambios

459. [peso 3] ¿Cuál es la secuencia correcta con un cursor explícito?

```
CURSOR c_emp IS SELECT ename FROM emp;
```

- A) DECLARE, CLOSE, FETCH
- B) OPEN, FETCH
- C) FETCH, OPEN, CLOSE
- D) EXECUTE, READ, END

460. [peso 4] ¿Qué ventaja tiene este cursor FOR loop?

```
FOR r IN (SELECT ename, sal FROM emp WHERE deptno = 10) LOOP
    DBMS_OUTPUT.PUT_LINE(r.ename || ': ' || r.sal);
END LOOP;
```

- A) Es más lento que un cursor explícito tradicional
- B) Requiere declarar %ROWTYPE manualmente antes de usarlo
- C) Permite ejecutar UPDATE directamente sobre el cursor
- D) OPEN, FETCH, EXIT y CLOSE son implícitos, y el registro r se declara automáticamente

461. [peso 4] ¿Qué hace WHERE CURRENT OF?

```
CURSOR c IS SELECT * FROM emp FOR UPDATE;
...
UPDATE emp SET sal = sal + 100
WHERE CURRENT OF c;
```

- A) Actualiza exactamente la fila que el cursor acaba de recuperar
- B) Actualiza únicamente la primera fila del cursor
- C) Error de sintaxis al combinarlo con FOR UPDATE
- D) Actualiza todas las filas de la tabla completa

462. [peso 4] ¿Qué imprime?

```
CREATE PROCEDURE swap(a IN OUT NUMBER, b IN OUT NUMBER) IS
    t NUMBER;
BEGIN
    t := a; a := b; b := t;
END;
-- llamada:
DECLARE x NUMBER := 1; y NUMBER := 2;
BEGIN swap(x, y); DBMS_OUTPUT.PUT_LINE(x || ',' || y); END;
```

- A) 1,1
- B) Error: IN OUT no existe
- C) 2,1
- D) 1,2

463. [peso 4] ¿Se puede llamar así en SQL?

```
CREATE FUNCTION tax(p_sal NUMBER) RETURN NUMBER IS
BEGIN
    RETURN p_sal * 0.16;
END;
-- luego:
SELECT ename, tax(sal) FROM emp;
```

- A) Sí: las funciones almacenadas que cumplen las reglas de pureza son invocables por fila en SELECT
- B) No: las funciones PL/SQL nunca se invocan desde SQL
- C) Solo es posible dentro de una cláusula WHERE
- D) Solo es posible usando PRAGMA RESTRICT_REFERENCES

464. [peso 4] ¿Qué particularidad tienen las variables declaradas en la spec de un package?

```
CREATE PACKAGE cfg IS
  g_contador NUMBER := 0;
END;
```

- A) Son compartidas entre sesiones
- B) Se reinician en cada llamada
- C) No se permiten variables en packages
- D) Mantienen su estado durante TODA la sesión

465. [peso 4] ¿Cuándo se dispara este trigger?

```
CREATE TRIGGER trg_emp
BEFORE INSERT OR UPDATE OF sal ON emp
FOR EACH ROW
BEGIN
  :NEW.sal := ROUND(:NEW.sal, 2);
END;
```

- A) Se dispara una sola vez por cada sentencia ejecutada
- B) Después de confirmarse la transacción con COMMIT
- C) Antes de cada FILA insertada o cuya columna sal se actualice
- D) Únicamente cuando se ejecuta un DELETE sobre emp

466. [peso 5] ¿Qué hace este bulk processing?

```
DECLARE
  TYPE t_ids IS TABLE OF emp.empno%TYPE;
  v_ids t_ids;
BEGIN
  SELECT empno BULK COLLECT INTO v_ids FROM emp;
  FORALL i IN v_ids.FIRST .. v_ids.LAST
    UPDATE emp SET sal = sal * 1.1 WHERE empno = v_ids(i);
END;
```

- A) Procesa cada fila de forma individual y completamente secuencial
- B) Da error de compilación: FORALL no admite sentencias UPDATE
- C) Es equivalente en rendimiento a recorrer un cursor con un FOR loop
- D) Carga todos los IDs en UNA operación y ejecuta los UPDATE en UN solo envío al motor SQL: minimiza cambios de contexto

467. [peso 4] ¿Qué diferencia hay entre estas colecciones?

```
TYPE t1 IS TABLE OF NUMBER; -- nested table
TYPE t2 IS TABLE OF NUMBER INDEX BY PLS_INTEGER; -- ?
TYPE t3 IS VARRAY(10) OF NUMBER;
```

- A) t1 solo acepta 10 elementos
- B) t2 es un associative array
- C) Son idénticas
- D) t3 es ilimitado

468. [peso 4] ¿Qué imprime?

```
DECLARE
  TYPE t_tab IS TABLE OF VARCHAR2(10) INDEX BY PLS_INTEGER;
  v t_tab;
BEGIN
  v(1) := 'a';
  v(5) := 'b';
  DBMS_OUTPUT.PUT_LINE(v.COUNT || ' ' || v.LAST);
END;
```

- A) 2 2
- B) 5 5
- C) Error de índice
- D) 2 5

469. [peso 4] ¿Qué hace SAVE EXCEPTIONS en FORALL?

```
FORALL i IN 1..v.COUNT SAVE EXCEPTIONS
  INSERT INTO destino VALUES v(i);
EXCEPTION
  WHEN OTHERS THEN
    FOR j IN 1..SQL%BULK_EXCEPTIONS.COUNT LOOP ... END LOOP;
```

- A) Continúa procesando pese a errores individuales y los acumula en SQL%BULK_EXCEPTIONS
- B) Guarda las excepciones en una tabla de auditoría
- C) Detiene el FORALL apenas ocurre el primer error
- D) Ignora silenciosamente todos los errores del lote

470. [peso 4] ¿Cómo consume Java este procedimiento?

```
CREATE PROCEDURE get_emps(p_cur OUT SYS_REFCURSOR) IS
BEGIN
  OPEN p_cur FOR SELECT * FROM emp;
END;
```

- A) Registrando el parámetro como OracleTypes.CURSOR y leyendo el resultado como ResultSet
- B) Con un executeQuery directo sobre el procedimiento
- C) Con el método getGeneratedKeys del Statement
- D) No es posible consumir cursores desde Java

471. [peso 4] ¿Qué riesgo tiene este SQL dinámico?

```
EXECUTE IMMEDIATE
  'DELETE FROM logs WHERE origen = ''' || p_origen || ''';
```

- A) Falta hacer commit tras el DELETE dinámico
- B) EXECUTE IMMEDIATE no admite sentencias DELETE
- C) Inyección SQL: p_origen se concatena
- D) Ninguno, el patrón es seguro tal cual

472. [peso 4] ¿Qué imprime?

```
DECLARE
  v_cnt NUMBER;
BEGIN
  EXECUTE IMMEDIATE
    'SELECT COUNT(*) FROM emp WHERE deptno = :d'
    INTO v_cnt USING 10;
  DBMS_OUTPUT.PUT_LINE(v_cnt);
END;
```

- A) El número de empleados del dept 10
- B) NULL
- C) 0 siempre
- D) Error: INTO no se usa con EXECUTE IMMEDIATE

473. [peso 4] ¿Qué imprime?

```
DECLARE
  v VARCHAR2(3);
BEGIN
  v := 'ABCD';
  DBMS_OUTPUT.PUT_LINE(v);
EXCEPTION
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('muy largo');
END;
```

- A) muy largo
- B) Error de compilación
- C) ABCD
- D) ABC (truncado)

474. [peso 4] ¿Qué valor tiene v tras el bloque?

```
DECLARE
  v NUMBER;
BEGIN
  IF NULL = NULL THEN v := 1;
  ELIF NULL IS NULL THEN v := 2;
  ELSE v := 3;
  END IF;
END;
```

- A) NULL
- B) 1
- C) 3
- D) 2

475. [peso 3] ¿Qué imprime?

```
DECLARE
  v_estado VARCHAR2(10) := 'B';
BEGIN
  CASE v_estado
    WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Activo');
    WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Baja');
    ELSE DBMS_OUTPUT.PUT_LINE('Otro');
  END CASE;
END;
```

- A) Baja
- B) Activo
- C) Otro
- D) CASE_NOT_FOUND

476. [peso 4] ¿Qué muestra este código para depurar errores de compilación?

```
SELECT line, text
FROM user_errors
WHERE name = 'MI_PROC'
ORDER BY sequence;
```

- A) El plan de ejecución de la última consulta
- B) Muestra el código fuente completo del procedimiento
- C) Los errores de compilación del procedimiento con su línea
- D) Los datos almacenados en la tabla relacionada

477. [peso 4] ¿Qué hace DETERMINISTIC en una función?

```
CREATE FUNCTION area(r NUMBER)
RETURN NUMBER DETERMINISTIC IS
BEGIN
  RETURN 3.14159 * r * r;
END;
```

- A) Declara que con las mismas entradas devuelve siempre lo mismo: habilita cachear resultados y usarla en índices basados en función
- B) Hace que la función se ejecute de forma autónoma en una transacción separada, sin afectar a la principal
- C) No tiene ningún efecto real sobre la función más allá de documentar su intención
- D) Impide que la función se use dentro de una sentencia SQL, solo permite llamadas PL/SQL

478. [peso 5] ¿Qué patrón muestra este compound trigger?

```
CREATE TRIGGER trg_audit
FOR UPDATE ON emp
COMPOUND TRIGGER
  TYPE t_ids IS TABLE OF NUMBER;
  g_ids t_ids := t_ids();
  AFTER EACH ROW IS BEGIN
    g_ids.EXTEND; g_ids(g_ids.COUNT) := :NEW.empno;
  END AFTER EACH ROW;
  AFTER STATEMENT IS BEGIN
    audit_pkg.log_bulk(g_ids);
  END AFTER STATEMENT;
END;
```

- A) Acumula IDs por fila y procesa en bloque al final de la sentencia: evita mutating table y N llamadas de log
- B) Error: no se pueden mezclar distintos timings en un mismo trigger
- C) Se comporta exactamente igual que un trigger de fila normal
- D) Declara en realidad dos triggers independientes sobre la tabla

479. [peso 3] ¿Qué imprime?

```
DECLARE
  v BOOLEAN := TRUE;
BEGIN
  IF v THEN
    DBMS_OUTPUT.PUT_LINE('sí');
  END IF;
END;
```

- A) Error: BOOLEAN no existe en PL/SQL
- B) sí
- C) Nada
- D) TRUE

480. [peso 4] ¿Qué hace este código de paquete UTL?

```
DBMS_OUTPUT.PUT_LINE(
  UTL_RAW.CAST_TO_VARCHAR2(
    UTL_RAW.CAST_TO_RAW('AB')));
```

- A) Imprime AB (ida y vuelta de conversión)
- B) Imprime el valor hexadecimal 4142
- C) Error al convertir la cadena a bytes crudos
- D) Imprime BA, invirtiendo el orden original

481. [peso 4] ¿Cuándo se ejecuta realmente el DML de este bloque?

```
BEGIN
  INSERT INTO t VALUES (1);
  -- sin COMMIT
END;
/
```

- A) El insert está hecho pero NO confirmado: otra sesión no lo ve hasta COMMIT
- B) El insert queda confirmado apenas termina el bloque
- C) Se revierte automáticamente al salir del bloque
- D) El bloque completo falla si no hay COMMIT explícito

482. [peso 4] ¿Qué devuelve v?

```
DECLARE
  v NUMBER;
  FUNCTION f RETURN NUMBER IS
  BEGIN
    RETURN 10;
    v := 99; -- línea X
  END;
BEGIN
  v := f;
  DBMS_OUTPUT.PUT_LINE(v);
END;
```

- A) 10: el código tras RETURN es inalcanzable
- B) Error de compilación por el código tras la línea X
- C) 99, porque la asignación sí llega a ejecutarse
- D) NULL, ya que la función nunca retorna valor

483. [peso 4] ¿Qué logra este uso de MERGE dentro de PL/SQL?

```
MERGE INTO stock s
USING (SELECT :id id, :qty qty FROM dual) src
ON (s.prod_id = src.id)
WHEN MATCHED THEN UPDATE SET s.qty = s.qty + src.qty
WHEN NOT MATCHED THEN INSERT VALUES (src.id, src.qty);
```

- A) Solo inserta filas nuevas, sin actualizar las que ya existen en la tabla destino
- B) Error: MERGE no admite una subconsulta USING basada en dual
- C) Upsert atómico de una fila: suma stock si existe, la crea si no, en una sola sentencia sin condición de carrera de SELECT previo
- D) Borra la fila existente e inserta una nueva en su lugar, perdiendo el historial

484. [peso 3] ¿Qué imprime?

```
BEGIN
  FOR i IN 1..3 LOOP
    CONTINUE WHEN i = 2;
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
```

- A) 1 2 3
- B) 3
- C) 1 3
- D) 1 2

485. [peso 4] ¿Qué imprime?

```
DECLARE
  x NUMBER := 5;
BEGIN
  DECLARE
    x NUMBER := 10;
  BEGIN
    DBMS_OUTPUT.PUT_LINE(x);
  END;
  DBMS_OUTPUT.PUT_LINE(x);
END;
```

- A) 10 y 5
- B) 10 y 10
- C) 5 y 10
- D) Error: x duplicada

486. [peso 4] ¿Qué evalúa correctamente cadenas nulas en Oracle?

```
DECLARE
  v VARCHAR2(10) := '';
BEGIN
  IF v IS NULL THEN DBMS_OUTPUT.PUT_LINE('nula');
  ELSE DBMS_OUTPUT.PUT_LINE('vacía');
  END IF;
END;
```

- A) Lanza un error al comparar la cadena vacía con NULL
- B) Devuelve 'vacía', distinguiéndola claramente de NULL
- C) No hace nada, la condición nunca se evalúa
- D) nula: en Oracle la cadena vacía " ES NULL

487. [peso 5] ¿Qué problema de rendimiento tiene este código?

```
FOR r IN (SELECT empno FROM emp) LOOP -- 1M filas
  UPDATE emp SET sal = sal * 1.1
  WHERE empno = r.empno;
END LOOP;
COMMIT;
```

- A) Falta índice en sal
- B) Ninguno
- C) Procesamiento fila a fila
- D) El COMMIT está mal ubicado

Optimización (13 preguntas)

488. [peso 4] ¿Cuál reescritura permite usar el índice de hiredate?

```
-- original:  
SELECT * FROM emp  
WHERE TO_CHAR(hiredate, 'YYYY') = '2024';
```

- A) WHERE EXTRACT(YEAR FROM hiredate) = 2024
- B) WHERE TRUNC(hiredate, 'YYYY') = DATE '2024-01-01'
- C) No hay forma de reescribir esta condición
- D) WHERE hiredate >= DATE '2024-01-01' AND hiredate < DATE '2025-01-01'

489. [peso 4] ¿Qué operación del plan indica un problema probable aquí?

```
SELECT e.*, d.*  
FROM emp e, dept d  
WHERE e.sal > 1000;  
-- Plan: MERGE JOIN CARTESIAN
```

- A) El filtro sobre la columna sal en el WHERE
- B) El producto cartesiano: falta la condición de join e.deptno = d.deptno
- C) El acceso por índice a la tabla dept
- D) Ninguno, el plan es completamente normal

490. [peso 4] ¿Por qué esta consulta con bind podría usar un plan subóptimo para 'RARO'?

```
SELECT * FROM pedidos WHERE estado = :b;  
-- 99% de filas 'PROCESADO', 1% 'RARO', histograma presente
```

- A) Los binds no usan índices
- B) El histograma rompe la consulta
- C) Bind peeking: el plan se fijó con el primer valor visto
- D) Los literales son siempre mejores

491. [peso 4] ¿Qué problema tiene SELECT * en esta consulta de reporte?

```
SELECT * FROM ventas v JOIN clientes c ON ...  
-- solo se usan 3 columnas de 40
```

- A) Bloquea la tabla mientras se ejecuta la consulta
- B) SELECT * simplemente no compila en Oracle
- C) Transfiere y procesa columnas innecesarias, impide índices covering y aumenta memoria/red
- D) Ninguno, SELECT * es igual de eficiente siempre

492. [peso 5] ¿Cuál de estas consultas evita la subconsulta correlacionada por fila?

```
-- original (lenta con emp grande):  
SELECT e.ename FROM emp e  
WHERE e.sal > (SELECT AVG(sal) FROM emp e2  
              WHERE e2.deptno = e.deptno);
```

- A) No se puede mejorar
- B) Convertirla en NOT IN
- C) SELECT ename FROM
- D) Agregar ROWNUM

493. [peso 4] ¿Qué delata este plan sobre el índice?

```
WHERE UPPER(ename) = 'SMITH'  
-- Plan: TABLE ACCESS FULL en EMP (existe índice en ename)
```

- A) Falta hacer commit tras crear el índice
- B) SMITH no existe en la tabla EMP
- C) UPPER() sobre la columna impide el índice normal
- D) El índice está corrupto y debe reconstruirse

494. [peso 4] ¿Cuál es el problema de rendimiento en Java?

```
for (Order o : orders) {           // 10.000 pedidos  
    var ps = conn.prepareStatement(  
        "SELECT * FROM items WHERE order_id = " + o.id());  
    ...  
}
```

- A) La lista de pedidos es demasiado grande para Java
- B) Solo el riesgo de inyección SQL en la consulta
- C) Nada, el patrón es aceptable en producción
- D) Problema N+1 + SQL literal: 10.000 consultas y hard parses en vez de un solo JOIN

495. [peso 4] ¿Por qué es mala idea COMMIT dentro del bucle?

```
FOR r IN c LOOP  
    UPDATE ...;  
    COMMIT; -- cada fila  
END LOOP;
```

- A) El commit simplemente falla dentro de un bucle
- B) Es obligatorio hacerlo tras cada fila procesada
- C) Solo consume más CPU, sin otro efecto
- D) Cada commit fuerza escritura síncrona de redo

496. [peso 4] ¿Qué mejora aporta el fetch size aquí?

```
st.setFetchSize(500);  
var rs = st.executeQuery("SELECT ... 100000 filas");
```

- A) Nada medible
- B) Aumenta el paralelismo del servidor
- C) Limita a 500 filas
- D) Reduce los round-trips de red de ~10.000

497. [peso 4] ¿Cuál LIKE puede usar un índice normal sobre ename?

```
A) WHERE ename LIKE '%SON'  
B) WHERE ename LIKE 'JO%'  
C) WHERE ename LIKE '%MI%'
```

- A) Ninguno de los tres puede usar un índice normal
- B) El patrón C: '%MI%', buscando en cualquier posición
- C) El patrón A: '%SON', buscando un sufijo fijo
- D) El patrón B: el prefijo fijo 'JO%' permite un range scan

498. [peso 5] ¿Qué transformación reduce este DELETE masivo de horas a minutos?

```
DELETE FROM logs WHERE fecha < ADD_MONTHS(SYSDATE, -12);  
-- borra 200M de 250M filas
```

- A) CTAS: crear tabla nueva con el 20% a conservar, índices, y reemplazar (o particionar y hacer DROP PARTITION)
- B) Aumentar el tamaño del tablespace de undo para soportar la transacción
- C) Agregar un índice sobre la columna fecha antes de ejecutar el DELETE
- D) Ejecutarlo de noche, fuera del horario laboral, sin cambiar la estrategia

499. [peso 4] ¿Qué patrón de caché muestra y qué riesgo tiene?

```
// Java  
Map<Integer, Product> cache = new ConcurrentHashMap<>();  
Product get(int id) {  
    return cache.computeIfAbsent(id, this::loadFromDb);  
}
```

- A) Es incorrecto: computeIfAbsent no es atómico
- B) Cache-aside thread-safe
- C) Write-through sin riesgos
- D) No es un caché

500. [peso 5] ¿Por qué la versión B es preferible para paginación profunda?

```
-- A:  
OFFSET 100000 ROWS FETCH NEXT 20 ROWS ONLY  
-- B (keyset):  
WHERE id > :ultimo_id ORDER BY id  
FETCH FIRST 20 ROWS ONLY
```

- A) B ordena los resultados de forma incorrecta
- B) No hay diferencia real entre ambos enfoques
- C) A debe LEER y descartar 100.000 filas antes de llegar a la página deseada
- D) A usa menos memoria que la versión con keyset

HOJA DE RESPUESTAS Y RETROALIMENTACIÓN

1. Respuesta correcta: D — ¿Cuál es el rango de un byte en Java?...

Retroalimentación: byte ocupa 8 bits con signo en complemento a dos: -2^7 a 2^7-1 , es decir -128 a 127.

2. Respuesta correcta: D — ¿Qué tipo de dato NO puede inferirse con var?...

Retroalimentación: var requiere que el compilador infiera un tipo concreto; null por sí solo no tiene tipo. Las lambdas tampoco compilan con var salvo cast explícito, pero null es el caso directo: `var x = null` no compila.

3. Respuesta correcta: D — ¿Qué ocurre al asignar un int a un long sin cast?...

Retroalimentación: `int`→`long` es una conversión de ampliación (widening primitive conversion) que Java realiza automáticamente sin cast.

4. Respuesta correcta: C — ¿Por qué String es inmutable en Java?...

Retroalimentación: La inmutabilidad permite compartir instancias en el pool, cachear el hash (clave en HashMap) y usar Strings de forma segura entre hilos y en APIs sensibles (class loading, rutas).

5. Respuesta correcta: D — En un text block ("""), ¿qué determina la indentación incidental que s...

Retroalimentación: El compilador calcula el mínimo de espacios iniciales entre todas las líneas de contenido y el delimitador de cierre; ese margen común se considera incidental y se elimina.

6. Respuesta correcta: C — ¿Qué operador realiza desplazamiento a la derecha sin signo?...

Retroalimentación: `>>>` rellena con ceros a la izquierda independientemente del signo; `>>` preserva el bit de signo (desplazamiento aritmético).

7. Respuesta correcta: A — ¿Cuál de estos es un identificador válido para una variable en Java 17?...

Retroalimentación: `precio$total` es válido (`$` y `_` se permiten dentro del nombre). No puede empezar con dígito, `'_'` como identificador único es ilegal desde Java 9, y `new` es palabra reservada.

8. Respuesta correcta: B — ¿Cuándo es obligatorio yield en un switch expression?...

Retroalimentación: Con sintaxis de flecha y bloque, o con sintaxis de dos puntos, el valor del caso se devuelve con yield; una expresión simple tras `->` lo devuelve implícitamente.

9. Respuesta correcta: D — ¿Qué clase debe usarse para cálculos monetarios exactos?...

Retroalimentación: BigDecimal ofrece aritmética decimal de precisión arbitraria; los tipos binarios de punto flotante introducen errores de redondeo inaceptables en finanzas.

10. Respuesta correcta: D — ¿Cuál es la diferencia clave entre Period y Duration?...

Retroalimentación: Period opera sobre conceptos de calendario (LocalDate); Duration sobre cantidades exactas de tiempo (LocalTime, Instant). Sumar un Period de 1 día puede diferir de 24 horas con cambios de horario.

11. Respuesta correcta: B — ¿Qué método de String elimina espacios en blanco Unicode al inicio y f...

Retroalimentación: `strip()` (Java 11) usa `Character.isWhitespace` y cubre espacios Unicode; `trim()` solo elimina caracteres $\leq U+0020$.

12. Respuesta correcta: D — ¿Qué garantiza el contrato entre equals() y hashCode()?...

Retroalimentación: Si `a.equals(b)` es true, `a.hashCode() == b.hashCode()` es obligatorio. Lo inverso no aplica: colisiones de hash entre objetos distintos son válidas. Romperlo corrompe HashMap/HashSet.

13. Respuesta correcta: A — ¿Cuál es el valor por defecto de una variable local int no inicializad...

Retroalimentación: Solo los campos de instancia/estáticos reciben valores por defecto; las variables locales deben asignarse definitivamente antes de leerse o el código no compila.

14. Respuesta correcta: A — ¿Qué palabra clave impide que una variable local sea reasignada?...

Retroalimentación: final impide la reasignación de la referencia o valor; const está reservada pero no se usa en Java. Ojo: final no hace immutable al objeto referenciado.

15. Respuesta correcta: C — ¿Cuál es el orden de precedencia correcto (de mayor a menor)?...

Retroalimentación: La negación unaria tiene mayor precedencia que AND lógico, que a su vez precede al OR lógico. Ante la duda, use paréntesis.

16. Respuesta correcta: B — ¿Qué estructura garantiza al menos una ejecución del cuerpo?...

Retroalimentación: do-while evalúa la condición después del cuerpo, garantizando una iteración mínima.

17. Respuesta correcta: B — ¿Qué imprime System.out.printf("%.2f", 1234567.891)?...

Retroalimentación: El flag `'f'` agrupa miles y .2f redondea a dos decimales según el Locale por defecto (aquí en formato en-US: 1,234,567.89).

18. Respuesta correcta: C — ¿Qué método convierte "42" en un int primitivo?...

Retroalimentación: `parseInt` devuelve int primitivo; `valueOf` devuelve un Integer (objeto, posiblemente cacheado).

19. Respuesta correcta: B — ¿Qué literal numérico NO es válido en Java 17?...

Retroalimentación: El guion bajo no puede ir al inicio, al final, ni junto al punto decimal o prefijos; 0b denota binario y los separadores internos son legales.

20. Respuesta correcta: D — ¿Cuál es la diferencia entre `StringBuilder` y `StringBuffer`?...

Retroalimentación: StringBuffer sincroniza sus métodos (herencia de Java 1.0); StringBuilder (Java 5) elimina esa sincronización y es preferible en contextos de un solo hilo.

21. Respuesta correcta: D — ¿Qué devuelve `"banana".indexOf('a', 2)`?...

Retroalimentación: La búsqueda inicia en el índice 2; la siguiente 'a' está en la posición 3.

22. Respuesta correcta: C — ¿Qué pasa si dos casos de un `switch` clásico (con `:`) coinciden y falta ...

Retroalimentación: El switch clásico continúa ejecutando los casos siguientes (fall-through) hasta encontrar break; el switch con `->` no tiene este comportamiento.

23. Respuesta correcta: B — ¿Qué tipos NO son válidos en la expresión de un `switch` en Java 17 (sin...)

Retroalimentación: switch acepta byte/short/char/int (y wrappers), String y enums; long, float, double y boolean quedan excluidos.

24. Respuesta correcta: B — ¿Qué clase representa un instante en la línea de tiempo UTC?...

Retroalimentación: Instant modela un punto absoluto en UTC con precisión de nanosegundos; LocalDateTime carece de zona y no identifica un instante único.

25. Respuesta correcta: D — ¿Qué formato produce `DateTimeFormatter.ofPattern("MM/dd/yyyy")` con `Loc...`

Retroalimentación: LocalTime no tiene campos de fecha; formatear con símbolos de fecha lanza UnsupportedOperationException.

26. Respuesta correcta: C — ¿Cuál es el resultado de `(int) 9.99`?...

Retroalimentación: El cast de double a int trunca la parte decimal (no redondea): 9.

27. Respuesta correcta: C — ¿Dónde puede usarse `var` en Java 17?...

Retroalimentación: var es inferencia de tipo local (LVTI): solo variables locales, índices de for y recursos try-with-resources, siempre con inicializador.

28. Respuesta correcta: B — ¿Qué método de `Arrays` busca en un arreglo ORDENADO?...

Retroalimentación: binarySearch requiere el arreglo previamente ordenado; sobre datos desordenados el resultado es indefinido.

29. Respuesta correcta: A — ¿Qué imprime `System.out.println(new int[]{1,2}.equals(new int[]{1,2}))`...

Retroalimentación: Los arreglos no redefinen equals/hashCode; comparan identidad de referencia. Use Arrays.equals o deepEquals para contenido.

30. Respuesta correcta: B — Con pattern matching, ¿cuándo NO está disponible la variable de patrón...

Retroalimentación: El alcance sigue el flujo: con if (o instanceof Tipo t) la variable existe solo en el then; con la negación `!(...)` existe en el else, porque ahí el match está garantizado.

31. Respuesta correcta: B — ¿Qué expresión evalúa el operador ternario?...

Retroalimentación: El ternario es el único operador de tres operandos: evalúa la condición y solo la rama seleccionada.

32. Respuesta correcta: D — ¿Qué tipo resulta de la expresión `(true ? 1 : 2.0)`?...

Retroalimentación: En el ternario numérico se aplica promoción binaria: el tipo común de int y double es double, así que la expresión vale 1.0.

33. Respuesta correcta: C — En un text block, ¿qué hace la secuencia `\` al final de una línea?...

Retroalimentación: La barra invertida final une la línea con la siguiente sin salto, útil para mantener líneas largas legibles en el código.

34. Respuesta correcta: C — ¿Qué paquete se importa implícitamente en todo archivo Java?...

Retroalimentación: java.lang (String, Math, System, Object...) se importa automáticamente; java.base es un módulo, no un paquete.

35. Respuesta correcta: A — ¿Cuál es el orden correcto en un archivo .java?...

Retroalimentación: Declaración de paquete (opcional, una sola), luego imports, luego tipos. Invertir el orden es error de compilación.

36. Respuesta correcta: C — ¿Qué sucede con `int max = Integer.MAX_VALUE; max++`?...

Retroalimentación: La aritmética entera en Java desborda silenciosamente en complemento a dos; use Math.addExact para detectar overflow con excepción.

37. Respuesta correcta: B — ¿Qué método de `String` verifica contenido ignorando mayúsculas?...

Retroalimentación: equalsIgnoreCase compara contenido sin distinguir mayúsculas; compareTo distingue y devuelve un int de orden.

38. Respuesta correcta: B — ¿Qué devuelve `"abc".repeat(0)`?...

Retroalimentación: repeat(0) devuelve ""; solo un argumento negativo lanza IllegalArgumentException.

39. Respuesta correcta: B — ¿Cuál es el tipo de retorno de `String.charAt(int)`?...

Retroalimentación: charAt devuelve el primitivo char en la posición dada; lanza StringIndexOutOfBoundsException si el índice es inválido.

40. Respuesta correcta: A — ¿Qué hace `String.valueOf(null)` directamente en una llamada?...

Retroalimentación: El literal null hace que el compilador elija la sobrecarga más específica, valueOf(char[]), que lanza NPE al desreferenciar. Con Object o = null; valueOf(o) sí devuelve "null".

41. Respuesta correcta: D — ¿Qué pilar de la OOP oculta el estado interno tras métodos de acceso?...

Retroalimentación: El encapsulamiento restringe el acceso directo a los campos (private) y expone comportamiento controlado; protege invariantes de la clase.

42. Respuesta correcta: A — ¿Qué regla aplica al tipo de retorno al sobrescribir un método?...

Retroalimentación: Java permite retornos covariantes: si el padre devuelve Number, el hijo puede devolver Integer. Ampliar el tipo (contravariante) no compila.

43. Respuesta correcta: A — Al sobrescribir, ¿qué puede hacer el método hijo con las excepciones ...

Retroalimentación: El hijo no puede ampliar el contrato de excepciones checked; puede reducirlas o eliminarlas. Las unchecked se declaran libremente porque no forman parte del contrato.

44. Respuesta correcta: C — ¿Qué modificador de acceso permite visibilidad en el mismo paquete y s...

Retroalimentación: protected = paquete + subclases (aun en otros paquetes, vía herencia); default se limita al paquete.

45. Respuesta correcta: A — ¿Qué debe ser la PRIMERA sentencia de un constructor?...

Retroalimentación: Todo constructor inicia con una llamada a otro constructor: this(...) al de la misma clase o super(...) al del padre; si no se escribe, el compilador inserta super().

46. Respuesta correcta: D — ¿Puede una clase abstracta tener constructor?...

Retroalimentación: Las clases abstractas no se instancian directamente, pero sus constructores se ejecutan en la cadena de construcción de las subclases.

47. Respuesta correcta: C — ¿Qué combinación de modificadores es ilegal en un método?...

Retroalimentación: abstract exige ser sobrescrito y final lo prohíbe: contradicción. Igualmente ilegales: abstract private y abstract static.

48. Respuesta correcta: D — En Java 17, ¿qué tipos de miembros puede tener una interfaz?...

Retroalimentación: Desde Java 9 las interfaces admiten métodos private (helpers para defaults); los campos son siempre public static final implícitos.

49. Respuesta correcta: D — Si una clase hereda un método concreto de una superclase y un default ...

Retroalimentación: Regla de resolución: la clase siempre gana sobre la interfaz; los defaults solo aplican cuando no hay implementación heredada de clase.

50. Respuesta correcta: C — ¿Qué genera automáticamente un record?...

Retroalimentación: El record deriva todo de sus componentes; no genera setters porque sus campos son final: es una clase portadora de datos inmutable.

51. Respuesta correcta: B — ¿Qué NO puede hacer un record?...

Retroalimentación: Todo record extiende implícitamente java.lang.Record, y Java no permite herencia múltiple de clases. Sí puede implementar interfaces y añadir métodos.

52. Respuesta correcta: A — ¿Qué exige sealed sobre sus subclases permitidas?...

Retroalimentación: Cada subtipo permitido debe declarar cómo continúa la jerarquía: la cierra (final), la restringe (sealed) o la reabre (non-sealed).

53. Respuesta correcta: A — Si las subclases de una clase sealed están en el mismo archivo, ¿es ob...

Retroalimentación: Cuando todos los subtipos residen en el mismo archivo fuente, permits puede omitirse y se infiere.

54. Respuesta correcta: B — ¿Cuándo se ejecuta el constructor de un enum?...

Retroalimentación: Las constantes se instancian una única vez durante la inicialización de la clase; el constructor es implícitamente private.

55. Respuesta correcta: A — ¿Qué variables locales puede capturar una clase anónima o lambda?...

Retroalimentación: La captura copia el valor; permitir reasignación crearía inconsistencias entre la copia y la variable. Por eso deben ser efectivamente finales.

56. Respuesta correcta: B — ¿Qué relación modela mejor la composición frente a la herencia?...

Retroalimentación: La composición (has-a) delega en objetos contenidos y reduce el acoplamiento; la herencia modela is-a y expone al hijo a los cambios del padre.

57. Respuesta correcta: D — ¿Qué imprime getClass() en una referencia polimórfica Padre p = new Hi...

Retroalimentación: getClass() es resuelto en runtime y devuelve la clase real del objeto instanciado, no el tipo declarado de la referencia.

58. Respuesta correcta: D — ¿Qué diferencia una sobrecarga (overload) de una sobrescritura (overr...

Retroalimentación: La sobrecarga se resuelve en compilación por los tipos de argumentos; la sobrescritura en runtime por el tipo real del objeto.

59. Respuesta correcta: D — ¿Cuál es el orden de inicialización al instanciar una subclase?...

Retroalimentación: Los bloques/campos estáticos corren una sola vez (padre antes que hijo); luego, por instancia: inicializadores+constructor del padre y después los del hijo.

60. Respuesta correcta: D — ¿Qué método de Object debe sobrescribirse junto con equals()?...

Retroalimentación: Romper la consistencia equals/hashCode hace que las colecciones hash no encuentren objetos lógicamente iguales.

61. Respuesta correcta: A — ¿Qué garantiza la anotación @Override?...

Retroalimentación: Es una verificación de compilación contra errores de firma (tipos, parámetros incorrectos); no cambia el comportamiento en runtime.

62. Respuesta correcta: D — ¿Qué es una clase final?...

Retroalimentación: final en la clase prohíbe subclases (como String); no implica inmutabilidad de su estado por sí solo.

63. Respuesta correcta: B — ¿Qué requisitos definen una clase inmutable correcta?...

Retroalimentación: Además de final y sin setters, los campos mutables (List, Date, arrays) deben copiarse defensivamente al entrar y salir, o exponerse como vistas inmutables.

64. Respuesta correcta: D — ¿Qué hace @FunctionalInterface?...

Retroalimentación: La anotación es un contrato verificado: si alguien agrega un segundo método abstracto, el código deja de compilar. Cualquier SAM interface es funcional aunque no la lleve.

65. Respuesta correcta: C — En genéricos, ¿qué significa List<? extends Number>?...

Retroalimentación: El wildcard acotado superior es de solo lectura (producer): el tipo real podría ser List<Integer> y agregar un Double rompería la seguridad de tipos. Regla PECS: producer-extends.

66. Respuesta correcta: A — Según PECS, ¿qué firma es correcta para copiar de src a dst?...

Retroalimentación: Producer Extends (se lee de src), Consumer Super (se escribe en dst). Es la firma real de Collections.copy.

67. Respuesta correcta: B — ¿Qué es el type erasure?...

Retroalimentación: Por erasure no se puede hacer new T(), T.class ni instanceof List<String>; el bytecode trabaja con el tipo crudo más los casts insertados por el compilador.

68. Respuesta correcta: A — ¿Qué categoría de excepción NO exige try/catch ni throws?...

Retroalimentación: Solo las checked (subclases de Exception que no son RuntimeException) participan del chequeo del compilador; las unchecked pueden propagarse libremente.

69. Respuesta correcta: A — En try-with-resources con dos recursos, ¿en qué orden se cierran?...

Retroalimentación: Los recursos se cierran en orden LIFO (inverso), garantizando que los dependientes se cierren antes que sus fuentes.

70. Respuesta correcta: B — Si el try lanza una excepción y close() lanza otra, ¿qué recibe el cat...?

Retroalimentación: En try-with-resources la excepción primaria es la del bloque; las de close() se adjuntan vía addSuppressed y se leen con getSuppressed(). (En try/finally clásico ocurría lo contrario).

71. Respuesta correcta: B — ¿Qué permite el multi-catch catch (IOException | SQLException e)?...

Retroalimentación: Los tipos del multi-catch no pueden ser subtipos entre sí; la variable e no puede reasignarse dentro del bloque.

72. Respuesta correcta: B — ¿Cuál de estas es una excepción checked?...

Retroalimentación: FileNotFoundException hereda de IOException (checked); las otras tres son RuntimeException.

73. Respuesta correcta: C — ¿Cuál es el propósito del módulo (module-info.java)?...

Retroalimentación: JPMS aporta encapsulamiento fuerte a nivel de módulo: solo los paquetes exportados son accesibles, y las dependencias se declaran explícitamente.

74. Respuesta correcta: B — En module-info.java, ¿qué diferencia hay entre requires y requires tra...?

Retroalimentación: Con requires transitive B, cualquier módulo que requiera el tuyo también lee B (implied readability), útil cuando tu API expone tipos de B.

75. Respuesta correcta: A — ¿Qué directiva permite reflexión profunda sobre un paquete sin exporta...?

Retroalimentación: opens habilita acceso reflexivo (setAccessible) en runtime sin exponer el paquete en compilación; clave para frameworks como Hibernate o Jackson.

76. Respuesta correcta: C — ¿Puede un método static acceder directamente a un campo de instancia?...

Retroalimentación: Los métodos static pertenecen a la clase, no a una instancia; necesitan una referencia explícita a un objeto para leer campos de instancia.

77. Respuesta correcta: D — ¿Qué interfaz debe implementar una clase para ser usada en for-each?...

Retroalimentación: El for-each se compila a iterable.iterator(); basta implementar Iterable<T> aunque no sea Collection.

78. Respuesta correcta: C — ¿Qué colección mantiene el orden de INSERCIÓN al iterar?...

Retroalimentación: LinkedHashMap enlaza las entradas en orden de inserción (o de acceso, configurable); TreeMap ordena por clave; HashMap no garantiza orden.

79. Respuesta correcta: C — ¿Qué estructura ofrece búsqueda ordenada con métodos como floorKey y c...?

Retroalimentación: TreeMap implementa NavigableMap sobre un árbol rojo-negro: búsquedas por proximidad en O(log n).

80. Respuesta correcta: A — ¿Qué diferencia a Comparable de Comparator?...

Retroalimentación: Una clase tiene UN orden natural (Comparable) pero puede ordenarse con N Comparators externos sin modificarla.

81. Respuesta correcta: C — ¿Qué lanza Collections.unmodifiableList(list).add(x)?...

Retroalimentación: Las vistas y colecciones inmutables rechazan mutaciones con UnsupportedOperationException. Ojo: la lista original subyacente sí puede seguir mutando la vista.

82. Respuesta correcta: B — ¿Qué estructura es más adecuada para una cola LIFO (pila) moderna?...

Retroalimentación: ArrayDeque supera a Stack (legada y sincronizada) con push/pop/peek sin bloqueos; PriorityQueue ordena por prioridad, no LIFO.

83. Respuesta correcta: B — ¿Por qué es peligroso mutar un objeto usado como clave en un HashMap?...

Retroalimentación: El bucket se calculó con el hash de inserción; tras mutar, get() busca en otro bucket y no encuentra la entrada, aunque esté en el mapa. Use claves inmutables.

84. Respuesta correcta: A — ¿Qué patrón de diseño usa un constructor privado y una instancia única?...

Retroalimentación: Singleton controla que exista una sola instancia; en Java moderno un enum de una constante es la implementación más robusta.

85. Respuesta correcta: D — ¿Qué problema resuelve el patrón Builder?...

Retroalimentación: Evita constructores telescópicos: métodos encadenados con nombre por parámetro y un build() final, ideal para objetos inmutables complejos.

86. Respuesta correcta: D — ¿Qué aporta una jerarquía sealed a un switch por patrones (pattern mat...)

Retroalimentación: Al conocer todos los subtipos posibles, el compilador valida que el switch los cubra; agregar un nuevo subtipo rompe la compilación donde falte manejarlo.

87. Respuesta correcta: B — ¿Cuál de las cuatro formas de referencia a método NO existe?...

Retroalimentación: Las cuatro formas reales: estático, instancia acotada (obj::m), instancia no acotada (Clase::m sobre el primer parámetro) y constructor (Clase::new).

88. Respuesta correcta: A — ¿Qué imprime Object.toString() por defecto?...

Retroalimentación: El toString heredado devuelve getClass().getName() + "@" + Integer.toHexString(hashCode()); por eso conviene sobrescribirlo.

89. Respuesta correcta: C — ¿Puede un constructor llamar a this() y super() a la vez?...

Retroalimentación: Ambas deben ser la primera sentencia, así que son mutuamente excluyentes; la cadena de this() terminará eventualmente en un constructor que llame a super().

90. Respuesta correcta: A — ¿Cuál es la característica principal de las operaciones intermedias de...

Retroalimentación: map, filter, sorted, etc. solo construyen el pipeline; nada se procesa hasta invocar una terminal como collect, forEach o count.

91. Respuesta correcta: D — ¿Qué diferencia hay entre map y flatMap?...

Retroalimentación: flatMap es la herramienta para estructuras anidadas (List<List<T>>, Optional dentro de stream): cada elemento produce un stream que se concatena.

92. Respuesta correcta: C — ¿Qué devuelve findFirst() sobre un stream vacío?...

Retroalimentación: Las terminales que pueden no producir resultado (findFirst, findAny, min, max, reduce sin identidad) devuelven Optional, evitando null.

93. Respuesta correcta: A — ¿Qué lanza Optional.get() sobre un Optional vacío?...

Retroalimentación: get() sin verificar isPresent() lanza NoSuchElementException; prefiera orElse/orElseThrow/ifPresent.

94. Respuesta correcta: C — ¿Qué devuelve allMatch sobre un stream VACÍO?...

Retroalimentación: allMatch y noneMatch devuelven true en stream vacío (vacuous truth); anyMatch devuelve false. Pregunta clásica de examen.

95. Respuesta correcta: C — ¿Qué requisito tiene reduce(identity, accumulator) para paralelismo co...

Retroalimentación: En paralelo el stream parte los datos y combina parciales: sin asociatividad ((a+b)+c = a+(b+c)) los resultados varían entre ejecuciones.

96. Respuesta correcta: C — ¿Qué colector cuenta elementos por grupo?...

Retroalimentación: El colector downstream counting() produce Map<K, Long>; es la forma idiomática de contar por categoría.

97. Respuesta correcta: A — ¿Qué diferencia a IntStream de Stream<Integer>?...

Retroalimentación: Los streams primitivos evitan el costo de autoboxing y agregan métodos numéricos; se convierte con mapToInt/boxed.

98. Respuesta correcta: B — ¿Qué interfaz funcional recibe un argumento y devuelve boolean?...

Retroalimentación: Predicate<T> declara boolean test(T t) y ofrece composición con and(), or() y negate().

99. Respuesta correcta: A — ¿Qué interfaz funcional NO recibe argumentos y devuelve un valor?...

Retroalimentación: Supplier<T> declara T get(); es la base de la evaluación perezosa (orElseGet, Optional.or, factorías).

100. Respuesta correcta: D — ¿Qué interfaz funcional corresponde a (String, String) -> Integer?...

Retroalimentación: BiFunction<T,U,R> recibe dos argumentos de tipos posiblemente distintos y devuelve R; BinaryOperator exige que los tres tipos coincidan.

101. Respuesta correcta: A — ¿Qué interfaz funcional usa forEach de un Stream?...

Retroalimentación: forEach(Consumer<? super T>) ejecuta un efecto por elemento y no devuelve nada.

102. Respuesta correcta: D — ¿Cuándo conviene usar parallelStream()?...

Retroalimentación: El paralelismo tiene costo de partición y coordinación en el ForkJoinPool común; con pocos datos u operaciones triviales suele ser más lento que el secuencial.

103. Respuesta correcta: D — ¿Qué operación intermedia es 'stateful' y puede penalizar el paralelismo...

Retroalimentación: sorted (también distinct, limit, skip) necesita ver todos o parte de los elementos antes de emitir, forzando barreras y buffering.

104. Respuesta correcta: D — ¿Qué hace Collectors.teeing (Java 12)?...

Retroalimentación: teeing(c1, c2, merger) procesa cada elemento en ambos colectores en una sola pasada; útil para min y max simultáneos, o suma y conteo.

105. Respuesta correcta: C — ¿Qué diferencia hay entre Stream.generate y Stream.iterate?...

Retroalimentación: generate(Supplier) produce valores independientes (aleatorios, constantes); iterate(seed, f) construye una secuencia dependiente. Ambos son infinitos sin limit.

106. Respuesta correcta: D — ¿Qué operación terminal devuelve siempre long?...

Retroalimentación: `count()` devuelve long en cualquier tipo de stream; `sum()` existe solo en streams primitivos.

107. Respuesta correcta: A — ¿Por qué `peek()` no debe usarse para lógica de negocio?...

Retroalimentación: Las optimizaciones del pipeline pueden eludir `peek` cuando el recuento se deriva del tamaño de la fuente; sus efectos secundarios no están garantizados.

108. Respuesta correcta: A — ¿Qué colector calcula el promedio de un campo numérico?...

Retroalimentación: `averagingInt/averagingDouble` devuelven `Double` como colector (usable como downstream en `groupingBy`); `IntStream.average` es método de stream, no colector.

109. Respuesta correcta: C — ¿Qué método crea un Stream desde un arreglo?...

Retroalimentación: `Arrays.stream` funciona con arreglos de objetos y primitivos (devuelve `IntStream` para `int[]`); los arreglos no tienen método `stream()` propio.

110. Respuesta correcta: C — ¿Qué característica pierde un stream generado desde un HashSet?...

Retroalimentación: `HashSet` no tiene orden definido; `findFirst`, `limit` y `skip` pierden significado predecible. `LinkedHashSet` o `List` sí conservan orden de encuentro.

111. Respuesta correcta: B — ¿Cuál es la firma funcional de `Consumer<T>.andThen?`...

Retroalimentación: `andThen` compone efectos: `c1.andThen(c2)` ejecuta `accept` de `c1` y luego de `c2` sobre el mismo argumento.

112. Respuesta correcta: C — ¿Qué interfaz representa un operador binario del mismo tipo, como `Inte...`

Retroalimentación: `BinaryOperator<Integer>` (especialización de `BiFunction`) e `IntBinaryOperator` (primitiva) encajan; la primitiva evita boxing.

113. Respuesta correcta: B — ¿Cuándo usar `Collectors.toUnmodifiableList()` frente a `toList()`?...

Retroalimentación: `Collectors.toList()` no promete mutabilidad ni tipo concreto (aunque suele dar `ArrayList`); `toUnmodifiableList` y `Stream.toList()` devuelven listas no modificables que además rechazan null.

114. Respuesta correcta: A — ¿Qué operación transforma `Stream<T>` en `Stream<R>`?...

Retroalimentación: `map` aplica una `Function<T,R>` a cada elemento; `filter`, `peek` y `distinct` conservan el tipo.

115. Respuesta correcta: D — ¿Qué hace `mapMulti` (Java 16) frente a `flatMap`?...

Retroalimentación: `mapMulti((elem, consumer) -> ...)` emite cero o más valores por elemento sin instanciar streams intermedios: más eficiente cuando se emiten pocos elementos.

116. Respuesta correcta: B — En un `Comparator`, ¿qué hace `nullsFirst?`...

Retroalimentación: `Comparator.nullsFirst(natural)` ordena nulls al inicio sin lanzar NPE; `nullsLast` los coloca al final.

117. Respuesta correcta: C — ¿Qué sucede al llamar `stream()` sobre una colección y luego modificar l...

Retroalimentación: Los streams vinculan la fuente tarde: los datos se leen al ejecutar la terminal. Modificar DURANTE la ejecución sí puede lanzar CME en fuentes fail-fast.

118. Respuesta correcta: A — ¿Qué método de `Optional` ejecuta una acción solo si hay valor?...

Retroalimentación: `ifPresent(Consumer)` evita el patrón `if (opt.isPresent()) { opt.get()... }` y no hace nada en vacío.

119. Respuesta correcta: D — ¿Qué interfaz de JDBC representa la conexión física a la base de datos?...

Retroalimentación: `Connection` encapsula la sesión con la BD; se obtiene de `DriverManager` o (preferible en producción) de un `DataSource` con pool.

120. Respuesta correcta: D — ¿Por qué se prefiere `DataSource` sobre `DriverManager` en producción?...

Retroalimentación: Crear conexiones físicas es costoso; el pool del `DataSource` las reutiliza. Además se integra con JNDI y con gestores de transacciones.

121. Respuesta correcta: C — ¿Cuál es el formato correcto de una URL JDBC?...

Retroalimentación: Siempre inicia con `jdbc:`, sigue el subprotocolo del vendor y los detalles específicos. 1521 es el puerto por defecto del listener de Oracle.

122. Respuesta correcta: A — ¿Dónde apunta el cursor de un `ResultSet` recién creado?...

Retroalimentación: El cursor comienza before-first: siempre se necesita un `next()` antes de leer con `getXxx`, incluso para una sola fila.

123. Respuesta correcta: B — ¿Qué devuelve el método `execute()` de `PreparedStatement`?...

Retroalimentación: `execute()` es genérico: con true se recupera `getResultSet()`, con false `getUpdateCount()`. Útil cuando no se conoce el tipo de sentencia.

124. Respuesta correcta: D — ¿Cuál es el estado por defecto de auto-commit en una `Connection` JDBC?...

Retroalimentación: Por defecto cada sentencia es su propia transacción. Para agrupar operaciones: `setAutoCommit(false)`, luego `commit()` o `rollback()`.

125. Respuesta correcta: C — ¿Qué garantiza try-with-resources con `Connection`, `PreparedStatement` y ...

Retroalimentación: Los tres implementan `AutoCloseable`; no cerrarlos fuga conexiones del pool. Cerrar la `Connection` cierra en cascada, pero el cierre explícito de los tres es la práctica correcta.

126. Respuesta correcta: B — ¿Qué interfaz JDBC sirve para invocar procedimientos almacenados?...

Retroalimentación: `CallableStatement` usa la sintaxis `{call nombre(?, ?)}` y soporta parámetros IN, OUT e INOUT.

127. Respuesta correcta: B — ¿Qué tipo de `ResultSet` permite moverse hacia atrás con `previous()`?...

Retroalimentación: El tipo por defecto (FORWARD_ONLY) solo avanza; los scrollable admiten previous, absolute(n), first, last. SENSITIVE además refleja cambios externos.

128. Respuesta correcta: C — ¿Qué excepción lanzan casi todas las operaciones JDBC?...

Retroalimentación: SQLException es checked: obliga a manejarla o declararla. Ofrece getSQLState, getErrorCode y encadenamiento con getNextException.

129. Respuesta correcta: C — ¿Qué hace Connection.setReadOnly(true)?...

Retroalimentación: Es una sugerencia de optimización (p. ej. enrutar a réplicas de lectura); el comportamiento exacto depende del driver.

130. Respuesta correcta: B — ¿Qué nivel de aislamiento evita lecturas sucias pero permite lecturas ...

Retroalimentación: READ_COMMITTED (el default de Oracle) solo lee datos confirmados, pero una misma consulta puede ver valores distintos si otra transacción confirma entre lecturas.

131. Respuesta correcta: A — ¿Para qué sirve DatabaseMetaData?...

Retroalimentación: conn.getMetaData() expone getTables, getColumns, versión del motor, soporte de transacciones, etc.: base de herramientas genéricas de BD.

132. Respuesta correcta: B — ¿Qué ventaja de rendimiento aporta PreparedStatement frente a Statemen...

Retroalimentación: Además de la seguridad, el SQL parametrizado permite a Oracle reutilizar el cursor compartido en la shared pool, evitando hard parses repetidos.

133. Respuesta correcta: D — ¿Sigue siendo necesario Class.forName("oracle.jdbc.OracleDriver") en J...

Retroalimentación: El mecanismo de service provider carga automáticamente cualquier driver presente en el classpath al usar DriverManager.

134. Respuesta correcta: A — ¿Qué método de PreparedStatement asigna NULL a un parámetro?...

Retroalimentación: setNull requiere el tipo SQL explícito. Muchos drivers aceptan setString(i, null), pero setNull es la forma portable y correcta para tipos no String.

135. Respuesta correcta: D — ¿Qué clase de java.sql corresponde a un TIMESTAMP con precisión de nan...

Retroalimentación: java.sql.Timestamp extiende java.util.Date agregando nanos; java.sql.Date solo conserva la parte de fecha y Time solo la hora.

136. Respuesta correcta: B — ¿Qué hace setFetchSize(100) en un Statement?...

Retroalimentación: fetchSize controla el tamaño del buffer de red por viaje, no el total de filas. En Oracle el default histórico es 10: subirlo acelera lecturas masivas.

137. Respuesta correcta: C — ¿Qué diferencia hay entre setMaxRows(n) y setFetchSize(n)?...

Retroalimentación: maxRows descarta silenciosamente el excedente (límite duro del resultado); fetchSize es una optimización de red sin afectar el contenido.

138. Respuesta correcta: C — ¿Qué representa un RowSet frente a un ResultSet?...

Retroalimentación: CachedRowSet mantiene los datos en memoria sin conexión activa y puede sincronizar cambios después; útil para capas desconectadas.

139. Respuesta correcta: C — ¿Qué patrón de diseño implementa DriverManager al elegir el driver seg...

Retroalimentación: DriverManager actúa como factoría: recorre los drivers registrados y delega en el que acepta la URL para fabricar la Connection.

140. Respuesta correcta: D — En Oracle JDBC, ¿qué logra la propiedad oracle.jdbc.implicitStatementC...

Retroalimentación: El statement caching implícito conserva los statements 'cerrados' listos para reuso: menos soft parses y mejor latencia en OLTP.

141. Respuesta correcta: A — ¿Qué método permite comprobar si una conexión sigue válida?...

Retroalimentación: isValid ejecuta una verificación real contra la BD con timeout; isClosed solo informa si close() fue llamado localmente, no detecta conexiones caídas.

142. Respuesta correcta: A — ¿Qué ocurre al llamar dos veces executeQuery() sobre el mismo Prepared...

Retroalimentación: Cada Statement mantiene UN ResultSet abierto por defecto; re-ejecutar cierra el anterior. Para varios resultados simultáneos se requieren statements distintos.

143. Respuesta correcta: D — ¿Qué aporta un pool como HikariCP frente a abrir conexiones directas?...

Retroalimentación: Establecer una conexión Oracle cuesta decenas de ms (red + autenticación + proceso servidor); el pool amortiza ese costo y evita agotar sesiones.

144. Respuesta correcta: D — ¿Por qué un parámetro ? NO puede sustituir el nombre de una tabla?...

Retroalimentación: El binding ocurre después del parseo y optimización: la estructura de la consulta (tablas, columnas) debe ser fija. SQL dinámico de identificadores exige construir el texto (con listas blancas).

145. Respuesta correcta: A — ¿Qué significa que el driver Oracle 'thin' sea tipo 4?...

Retroalimentación: Los drivers tipo 4 (pure Java, direct-to-database) simplifican el despliegue; el tipo 2 (OCI) necesita bibliotecas nativas del cliente Oracle.

146. Respuesta correcta: C — ¿Qué paquete contiene las interfaces principales de JDBC?...

Retroalimentación: java.sql define Connection, Statement, ResultSet, etc.; javax.sql agrega DataSource, RowSet y pooling.

147. Respuesta correcta: B — ¿En qué orden lógico procesa Oracle las cláusulas de un SELECT?...

Retroalimentación: El orden lógico explica por qué los alias del SELECT no son visibles en WHERE (se evalúa antes) pero sí en ORDER BY (se evalúa después).

148. Respuesta correcta: B — ¿Qué diferencia a NVL2(expr, a, b) de NVL(expr, a)?...

Retroalimentación: NVL2 elige entre dos valores según la nulidad; NVL solo reemplaza el caso NULL. COALESCE es la variante estándar de N argumentos.

149. Respuesta correcta: B — ¿Qué diferencia hay entre COUNT(*) y COUNT(columna)?...

Retroalimentación: COUNT(comm) sobre emp con 4 comisiones no nulas devuelve 4 aunque haya 14 filas; COUNT() devuelve 14.*

150. Respuesta correcta: D — ¿Cuál es la diferencia entre WHERE y HAVING?...

Retroalimentación: Filtrar en WHERE lo que no requiere agregación reduce filas antes de agrupar y es más eficiente; HAVING AVG(sal) > x solo es posible tras agrupar.

151. Respuesta correcta: D — ¿Qué operador devuelve las filas de la primera consulta que NO están e...

Retroalimentación: MINUS es el operador propio de Oracle (EXCEPT llegó como sinónimo en 21c); INTERSECT devuelve las comunes.

152. Respuesta correcta: B — ¿Qué tipo de subconsulta puede referirse a columnas de la consulta ext...

Retroalimentación: La subconsulta correlacionada se evalúa lógicamente por cada fila externa; las inline views del FROM son independientes.

153. Respuesta correcta: B — ¿Qué cláusula moderna de Oracle 12c+ reemplaza el patrón ROWNUM para p...

Retroalimentación: La row limiting clause estándar se aplica después del ORDER BY: OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY página de forma legible.

154. Respuesta correcta: C — ¿Qué diferencia hay entre TRUNC(fecha, 'MM') y ROUND(fecha, 'MM')?...

Retroalimentación: TRUNC(DATE '2024-05-20', 'MM') = 01-may; ROUND con día 20 salta al 01-jun. Ambas ponen la hora en 00:00.

155. Respuesta correcta: D — ¿Qué constraint garantiza que una columna referencie una clave de otra...

Retroalimentación: La foreign key impone integridad referencial: el valor debe existir en la clave primaria/única referenciada (o ser NULL si la columna lo permite).

156. Respuesta correcta: D — ¿Qué sentencia combina INSERT y UPDATE según exista o no la fila?...

Retroalimentación: MERGE INTO destino USING fuente ON (condición) WHEN MATCHED THEN UPDATE... WHEN NOT MATCHED THEN INSERT...: sincronización idiomática en Oracle.

157. Respuesta correcta: A — ¿Qué ventaja tienen las funciones analíticas sobre GROUP BY?...

Retroalimentación: OVER() calcula sobre un conjunto de filas manteniendo la granularidad original: detalle y agregado en la misma fila, sin self-joins.

158. Respuesta correcta: B — ¿Qué es una CTE (WITH clause) en Oracle?...

Retroalimentación: WITH nombre AS (SELECT...) mejora legibilidad y permite reuso; el optimizador decide si la materializa (o con hints /+ MATERIALIZE */).*

159. Respuesta correcta: A — ¿Qué diferencia hay entre una vista y una vista materializada?...

Retroalimentación: Las vistas materializadas precálculan agregaciones/joins costosos y admiten índices y query rewrite; el costo es la gestión del refresco (COMPLETE/FAST/ON COMMIT).

160. Respuesta correcta: D — ¿Qué diferencia hay entre NATURAL JOIN y JOIN ... USING?...

Retroalimentación: NATURAL JOIN es frágil: agregar una columna homónima cambia silenciosamente el join. USING(col) es explícito y en ambos la columna común no admite prefijo de tabla.

161. Respuesta correcta: C — ¿Qué comando revierte una transacción no confirmada?...

Retroalimentación: ROLLBACK deshace los cambios desde el último COMMIT (o hasta un SAVEPOINT). Recuerde que en Oracle todo DDL confirma implícitamente.

162. Respuesta correcta: A — ¿Qué tipo de dato de Oracle almacena texto de longitud variable hasta ...

Retroalimentación: VARCHAR2 es el estándar de facto; CHAR rellena con espacios a longitud fija y CLOB almacena hasta gigabytes para texto grande.

163. Respuesta correcta: D — ¿Qué hace GROUP BY ROLLUP(a, b)?...

Retroalimentación: ROLLUP produce la jerarquía de subtotales de derecha a izquierda; CUBE generaría TODAS las combinaciones. GROUPING() distingue los NULL de subtotal.

164. Respuesta correcta: D — ¿Qué objeto genera valores numéricos únicos y secuenciales?...

Retroalimentación: CREATE SEQUENCE con NEXTVAL/CURRVAL; desde 12c también columnas IDENTITY y DEFAULT seq.NEXTVAL.

165. Respuesta correcta: B — ¿Cuál es la estructura mínima de un bloque anónimo PL/SQL?...

Retroalimentación: Solo BEGIN...END; es obligatorio; DECLARE y EXCEPTION son secciones opcionales del bloque.

166. Respuesta correcta: A — ¿Qué declara v_sal emp.sal%TYPE?...

Retroalimentación: %TYPE ancla el tipo a la columna; %ROWTYPE anclaría un registro a la estructura completa de la tabla o cursor. Reduce mantenimiento.

167. Respuesta correcta: A — ¿Qué regla aplica a WHEN OTHERS en un manejador de excepciones?...

Retroalimentación: WHEN OTHERS captura cualquier excepción no manejada antes, por eso el compilador exige que sea el último (PLS-00370 sí no).

168. Respuesta correcta: D — ¿Qué hace PRAGMA EXCEPTION_INIT?...

Retroalimentación: Ejemplo: `e_fk EXCEPTION; PRAGMA EXCEPTION_INIT(e_fk, -2292);` permite `WHEN e_fk THEN` en lugar de comparar `SQLCODE` dentro de `WHEN OTHERS`.

169. Respuesta correcta: C — ¿Qué diferencia hay entre un cursor implícito y uno explícito?...

Retroalimentación: `SQL%ROWCOUNT`, `SQL%FOUND` informan del último implícito; los explícitos dan control fino y sus propios atributos (`cur%FOUND`, `cur%ROWCOUNT`).

170. Respuesta correcta: C — ¿Qué diferencia un procedimiento de una función en PL/SQL?...

Retroalimentación: Solo las funciones (con restricciones de pureza) son invocables dentro de un `SELECT`; los procedimientos se llaman como sentencias.

171. Respuesta correcta: D — ¿Qué característica tiene un parámetro OUT al entrar al procedimiento?...

Retroalimentación: OUT inicializa en NULL dentro del procedimiento; IN OUT sí trae el valor original. Con NOCOPY se pasa por referencia (sin copia), con implicaciones ante excepciones.

172. Respuesta correcta: D — ¿Qué contiene la especificación (spec) de un package frente a su body?...

Retroalimentación: Separar spec y body permite recompilar implementaciones sin invalidar dependientes, y ocultar procedimientos privados declarados solo en el body.

173. Respuesta correcta: A — ¿Qué provoca el error ORA-04091 (mutating table)?...

Retroalimentación: La tabla está en estado inconsistente durante el DML; soluciones: compound trigger, mover la lógica a nivel de sentencia, o rediseñar.

174. Respuesta correcta: C — ¿Qué son :OLD y :NEW en un trigger de fila?...

Retroalimentación: En INSERT solo :NEW tiene datos; en DELETE solo :OLD; en UPDATE ambos. Se referencian sin '.' dentro de la cláusula WHEN.

175. Respuesta correcta: C — ¿Qué hace la cláusula LIMIT en BULK COLLECT?...

Retroalimentación: FETCH c BULK COLLECT INTO v LIMIT 1000 en un bucle procesa por lotes: sin LIMIT, una tabla enorme puede agotar memoria.

176. Respuesta correcta: C — ¿Qué es un REF CURSOR y para qué se usa?...

Retroalimentación: SYS_REFCURSOR es el tipo débil predefinido; es el mecanismo estándar para devolver conjuntos de resultados desde procedimientos a JDBC u otros lenguajes.

177. Respuesta correcta: B — ¿Qué hace PRAGMA AUTONOMOUS_TRANSACTION?...

Retroalimentación: Típico en logging de errores: el log se confirma aunque la transacción principal haga rollback. Debe terminar en COMMIT o ROLLBACK propio o lanza ORA-06519.

178. Respuesta correcta: D — ¿Qué diferencia hay entre NUMBER y PLS_INTEGER para contadores?...

Retroalimentación: PLS_INTEGER (y SIMPLE_INTEGER con NOT NULL y wrap) opera en registros de máquina; NUMBER está optimizado para exactitud, no velocidad.

179. Respuesta correcta: C — ¿Cuándo cambia un objeto PL/SQL a estado INVALID?...

Retroalimentación: La invalidación en cascada es normal tras ALTER TABLE o recompilar dependencias; USER_OBJECTS.status y USER_ERRORS muestran el estado y los errores.

180. Respuesta correcta: D — ¿Qué aporta RESULT_CACHE a una función?...

Retroalimentación: El result cache de funciones acelera lookups repetidos de datos casi estáticos; RELIES_ON se detecta automáticamente desde 11gR2.

181. Respuesta correcta: A — ¿Puede una columna de tabla ser BOOLEAN en Oracle 19c?...

Retroalimentación: El tipo BOOLEAN en SQL llegó en Oracle 23c; en 19c y anteriores se usa la convención 'Y'/'N' o 0/1 con CHECK.

182. Respuesta correcta: C — ¿Qué hace el paquete DBMS_SCHEDULER?...

Retroalimentación: CREATE_JOB con repeat_interval (sintaxis calendario: `FREQ=DAILY;BYHOUR=2`) reemplaza al antiguo DBMS_JOB.

183. Respuesta correcta: C — ¿Qué significan los niveles de PRAGMA en funciones para SQL: WNDS, RND?...

Retroalimentación: RESTRICT_REFERENCES declaraba garantías de pureza (hoy Oracle las verifica automáticamente en la mayoría de casos); WNDS es obligatoria para funciones llamadas desde SQL.

184. Respuesta correcta: B — ¿Qué privilegio se necesita para ejecutar un procedimiento de otro esq?...

Retroalimentación: GRANT EXECUTE ON hr.mi_proc TO app; por defecto el procedimiento corre con los derechos del DEFINIDOR (definir's rights) salvo AUTHID CURRENT_USER.

185. Respuesta correcta: B — ¿Qué cambia AUTHID CURRENT_USER en un procedimiento?...

Retroalimentación: Invoker's rights: el mismo código opera sobre los objetos del usuario que lo llama; el default (DEFINER) usa los del propietario, útil para encapsular seguridad.

186. Respuesta correcta: D — ¿Para qué sirve DBMS_UTILITY.FORMAT_ERROR_BACKTRACE?...

Retroalimentación: SQLERRM da el mensaje pero no el origen; FORMAT_ERROR_BACKTRACE reconstruye la pila de líneas: esencial en manejadores WHEN OTHERS de bloques grandes.

187. Respuesta correcta: A — ¿Por qué esta condición impide usar el índice sobre hiredate?...

Retroalimentación: El índice almacena hiredate, no TRUNC(hiredate). Soluciones: reescribir como rango (`hiredate >= x AND hiredate < y`) o crear un function-based index.

188. Respuesta correcta: C — ¿Qué muestra EXPLAIN PLAN / DBMS_XPLAN.DISPLAY?...

Retroalimentación: Es una estimación del optimizador; para el plan REAL con estadísticas de ejecución use DBMS_XPLAN.DISPLAY_CURSOR con el hint GATHER_PLAN_STATISTICS.

189. Respuesta correcta: C — ¿Cuándo un FULL TABLE SCAN es MEJOR que un acceso por índice?...

Retroalimentación: El índice implica lectura del índice + salto a la tabla por fila; con alta selectividad de filas devueltas, el escaneo completo con multiblock reads es más barato. El optimizador decide por costo.

190. Respuesta correcta: A — ¿Qué método de join conviene para unir dos tablas GRANDES sin índices ...

Retroalimentación: HASH JOIN construye una tabla hash con la entrada menor y sondea con la mayor: ideal para equijoins masivos. NESTED LOOPS brilla con pocas filas dirigidas por índice.

191. Respuesta correcta: B — ¿Qué recolecta DBMS_STATS.GATHER_TABLE_STATS y por qué importa?...

Retroalimentación: El optimizador basado en costos decide TODO a partir de estadísticas; tras cargas masivas, recolectarlas evita estimaciones erróneas de cardinalidad.

192. Respuesta correcta: C — ¿Qué índice soporta esta consulta: WHERE deptno = 10 ORDER BY sal?...

Retroalimentación: La columna de igualdad primero y la de orden después permiten INDEX RANGE SCAN que ya entrega el orden requerido, eliminando el paso de ordenación.

193. Respuesta correcta: B — ¿Puede usarse un índice en (a, b) para WHERE b = 5 sin condición sobre...

Retroalimentación: El skip scan recorre el índice por cada valor de la columna líder: viable con baja cardinalidad de a, pero raramente óptimo. El orden de columnas del índice compuesto importa.

194. Respuesta correcta: A — ¿Qué es un índice bitmap y dónde conviene?...

Retroalimentación: Los bitmaps se combinan con AND/OR a nivel de bits (estrella de DWH), pero un UPDATE bloquea rangos de filas: contraindicados en OLTP concurrente.

195. Respuesta correcta: A — ¿Qué diferencia EXISTS de IN en subconsultas, en términos de optimizac...

Retroalimentación: Oracle convierte ambas en semi-joins cuando puede. La regla práctica: prefiera NOT EXISTS sobre NOT IN por el comportamiento con NULLs, no por velocidad.

196. Respuesta correcta: A — ¿Qué logra el particionamiento por rango de fechas en una tabla de mil...

Retroalimentación: WHERE fecha >= ... permite podar particiones enteras del plan; DROP PARTITION purga meses antiguos en segundos frente a un DELETE masivo.

197. Respuesta correcta: D — ¿Qué mide la 'selectividad' de un predicado?...

Retroalimentación: El optimizador estima la selectividad con estadísticas para elegir índice vs full scan y el orden de joins; por eso los histogramas importan en datos sesgados.

198. Respuesta correcta: A — ¿Qué es un covering index (índice cubridor)?...

Retroalimentación: Eliminar el TABLE ACCESS BY INDEX ROWID ahorra los accesos aleatorios a la tabla; se logra agregando al índice las columnas del SELECT.

199. Respuesta correcta: A — ¿Qué caracteriza a una consulta 'sargable'?...

Retroalimentación: Search ARGument ABLE: col = valor, col > valor, col LIKE 'x%' son sargables; NVL(col,0) = 0, col + 1 = 10 y LIKE '%x' no lo son.

200. Respuesta correcta: A — ¿Qué indica una espera alta en 'db file sequential read'?...

Retroalimentación: Contraintuitivo: sequential read = single block (índices); scattered read = multiblock (full scans). Son los eventos base del diagnóstico de I/O en AWR.

201. Respuesta correcta: A — ¿Qué hace el hint /*+ INDEX(e ix_emp_dept) */?...

Retroalimentación: Un hint parchea un plan puntual pero congela decisiones: si los datos cambian, puede volverse dañino. Primero: estadísticas correctas, predicados sargables e índices adecuados.

202. Respuesta correcta: B — ¿Qué causa el error ORA-01555 (snapshot too old)?...

Retroalimentación: La lectura consistente reconstruye datos desde undo; si la consulta dura más que la retención de undo (o hay commits en su propio bucle), las versiones desaparecen. Solución: undo_retention adecuado y evitar fetch-across-commit.

203. Respuesta correcta: D — ¿Por qué conviene paginar resultados grandes hacia la aplicación?...

Retroalimentación: Con OFFSET/FETCH (o mejor, paginación por keyset: WHERE id > :ultimo ORDER BY id) la aplicación consume solo lo necesario por página.

204. Respuesta correcta: C — ¿Qué imprime este código?...

Retroalimentación: x++ devuelve 5 (x pasa a 6); ++x incrementa primero y devuelve 7. 5+7=12.

205. Respuesta correcta: B — ¿Cuál es el resultado?...

Retroalimentación: La evaluación es de izquierda a derecha: 1+2=3 (aritmética), luego concatenación: "3"+"3"+"4"+"5" = 3345.

206. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: La aritmética binaria de punto flotante IEEE 754 no representa 0.1 ni 0.2 exactamente; la suma da 0.30000000000000004. Para dinero use BigDecimal.

207. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: "Ja" + "va" es una expresión constante que el compilador resuelve en tiempo de compilación e interna en el String pool, por lo que ambas referencias apuntan al mismo objeto.

208. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `intern()` devuelve la referencia canónica del String pool; como "Java" ya está internado, a y b son la misma referencia.

209. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `insert(2,"io")` sobre "radar" da "raiodar"; `delete(0,1)` elimina el índice 0 (fin exclusivo): "aiodar". `StringBuilder` es mutable y encadena sobre la misma instancia.

210. Respuesta correcta: B — ¿Cuál es el resultado?...

Retroalimentación: `List.of()` crea una lista inmutable; `add()` lanza `UnsupportedOperationException`. Compila porque el tipo inferido es `List<Integer>`, que sí declara `add()`.

211. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: La asignación copia la referencia, no el arreglo; ambos nombres apuntan al mismo objeto en el heap.

212. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: En Java el resultado del módulo toma el signo del dividendo: $-7 \% 3 = -1$ (porque $-7 = 3*(-2) + (-1)$).

213. Respuesta correcta: C — ¿Cuál es el resultado?...

Retroalimentación: `Integer.valueOf` cachea los valores de -128 a 127; dentro del rango las referencias coinciden, fuera se crean objetos distintos. Compare wrappers siempre con `equals()`.

214. Respuesta correcta: D — ¿Qué imprime este switch expression?...

Retroalimentación: Las switch expressions con flecha permiten múltiples etiquetas por caso y no tienen fall-through; `day=3` coincide con el segundo caso.

215. Respuesta correcta: B — ¿Cuántas veces se imprime "Hi"?...

Retroalimentación: `i=0: j=3,2,1` (3 veces); `i=1: j=3,2` (2); `i=2: j=3` (1). Total $3+2+1=6$.

216. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `continue` con etiqueta salta a la siguiente iteración del bucle externo; solo se imprime `j=1` en cada pasada de `i`.

217. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `equals()` de `BigDecimal` compara valor Y escala (1.0 tiene escala 1, 1.00 escala 2); `compareTo` compara solo el valor numérico. Por eso en `TreeSet/TreeMap` se comportan como iguales.

218. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `plusMonths` ajusta al último día válido del mes destino; febrero de 2024 (bisiesto) termina el 29.

219. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: El 10/03/2024 a las 2:00 comienza el horario de verano en Nueva York; las 2:30 no existe y la API la desplaza a las 3:30 (gap). No lanza excepción.

220. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: String es inmutable: `toUpperCase` y `concat` devuelven nuevos objetos que aquí se descartan; s no cambia.

221. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: `substring(inicio, fin)` incluye el índice inicial y excluye el final: índices 2 y 3 → "cd".

222. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `c += 2` aplica cast implícito de vuelta a char ('c'); en la suma explícita 'a' se promueve a int (97) y el resultado 99 se imprime como número.

223. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `b = true` es una asignación (no comparación) cuyo valor es true; el if la acepta porque el resultado es boolean. Error clásico de examen: `=` vs `==`.

224. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `s + 5` se promueve a int y asignarlo a short requiere cast explícito; `+=` incluye un cast implícito al tipo de la variable, por eso la línea B sí compilaría sola.

225. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `x++` devuelve el valor original (10) antes de incrementar; la asignación posterior sobrescribe el 11 con ese 10 guardado.

226. Respuesta correcta: B — ¿Cuál es el resultado?...

Retroalimentación: final protege la referencia, no el contenido: modificar elementos es legal, reasignar el arreglo no.

227. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: La división entre ints trunca (2); si un operando es double, el otro se promueve y el resultado es 2.5.

228. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: En punto flotante $10/0.0 = \text{Infinity}$ (no lanza excepción); $10 \% 0.0 = \text{NaN}$, y NaN nunca es igual a nada, ni a sí mismo.

229. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `&&` es cortocircuito: `i++ > 0` evalúa `0 > 0 = false` (i pasa a 1) y el segundo operando nunca se ejecuta.

230. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: | sin cortocircuito evalúa ambos lados siempre: `i++ (i=1)` y `++i (i=2)`.

231. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: La variable del for-each es una copia de la referencia; reasignarla no modifica el arreglo original.

232. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Modificar estructuralmente una colección durante un for-each dispara `ConcurrentModificationException` del iterador fail-fast; use `Iterator.remove()` o `removeIf()`.

233. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: La concatenación convierte null en la cadena literal "null" mediante `String.valueOf()`; no lanza NPE.

234. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El unboxing de null (`i.intValue()`) lanza `NullPointerException` en tiempo de ejecución; el código compila sin errores.

235. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: El segundo argumento es la base: 1010 en binario es 10 decimal.

236. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `0x1F` es hexadecimal (31); un literal con cero inicial es octal: `017` = 15 decimal. Fuente clásica de bugs.

237. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Cada `+=` crea internamente un nuevo `String` (vía `StringBuilder` por iteración); en bucles largos conviene un `StringBuilder` externo para evitar basura intermedia.

238. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `chars()` devuelve un `IntStream` con los code points UTF-16 de la cadena; `count()` da su longitud, 5.

239. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `reverse()` muta a "cba" y devuelve this; `length()` en ese momento es 3, que se anexa: "cba3".

240. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Entra por case 2 y por fall-through ejecuta también case 3, donde el break detiene la cascada antes del default.

241. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `Period` normaliza: del 01-ene al 15-mar hay 2 meses completos y 14 días. `getDays()` devuelve solo el componente de días, no el total.

242. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: 2025 no es bisiesto; la API ajusta al último día válido del mes: 2025-02-28.

243. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El desbordamiento en punto flotante produce Infinity silenciosamente (no excepción); `3.6e38` excede el máximo de float (~`3.4e38`).

244. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `Math.round` suma 0.5 y aplica floor: `-2.5+0.5=-2` → -2; `2.5+0.5=3` → 3. El redondeo es hacia arriba en el empate, no simétrico.

245. Respuesta correcta: B — ¿Cuál de estas declaraciones NO compila?...

Retroalimentación: `var` no puede inferir tipo de null aislado; las demás infieren `int`, `int[]` y `String` respectivamente.

246. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Si no encuentra el valor, devuelve `-(punto_de_inserción) - 1`; 8 iría en el índice 1, por lo tanto `-(1)-1 = -2`.

247. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Los arreglos multidimensionales pueden ser irregulares (jagged): `m[1][2]=5` y `m[0].length=2`, total 7.

248. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `==` compara referencias (objetos distintos); `Arrays.equals` compara elemento a elemento.

249. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Pattern matching de `instanceof` (Java 16) declara la variable de patrón `n`, disponible donde el flujo garantiza el match — incluido el lado derecho del `&&`.

250. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `*` tiene mayor precedencia que `+`: `2*3=6`, luego `"1"+6` concatena a "16".

251. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: El cast trunca a 8 bits: `130` = `1000_0010` en binario, que en complemento a dos es -126.

252. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: El ternario solo evalúa la rama seleccionada: `a++` devuelve 5 y luego `a` vale 6; `a--` nunca se ejecuta.

253. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El text block contiene dos líneas de contenido; el delimitador de cierre en la misma línea que "Mundo" no agrega salto final.

254. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `split` descarta las cadenas vacías finales (con `limit 0` por defecto): `["A","B","", "C"]` → longitud 4.

255. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: y se autoboxea a Long, e Integer.equals devuelve false si el argumento no es Integer, aunque el valor numérico coincida.

256. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: parseBoolean es case-insensitive para "true"; cualquier otra cadena (incluida "yes") devuelve false sin excepción.

257. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: 'A' es 65; el desplazamiento $<< 1$ multiplica por 2 $\rightarrow 130$. El resultado de operar un char es int.

258. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: floorDiv redondea hacia -infinito (-4); la división entera / trunca hacia cero (-3).

259. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: compareTo es lexicográfico por valores char: 'a'(97) < 'b'(98) $\rightarrow$ resultado negativo $\rightarrow$ true.

260. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: strip() quita espacios exteriores; indent(2) antepone 2 espacios a cada línea Y agrega salto de línea final si falta: "java\n".

261. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: append está sobrecargado para todos los primitivos y convierte cada valor a su representación textual: "1true2.0".

262. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: La multiplicación se hace en int (todos los literales lo son) y desborda ANTES de asignarse al long. Solución: hacer long alguno de los factores (24L * ...).

263. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Duration.ofDays(1) equivale exactamente a 24 horas; Period.ofDays(1) es un concepto de calendario que conserva el componente días.

264. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El despacho de métodos de instancia es dinámico: se ejecuta la versión del tipo real del objeto (B), no del tipo de la referencia.

265. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Los métodos static no se sobrescriben, se ocultan (hiding): la resolución es estática por el tipo de la referencia (A). Invocar estáticos vía instancia compila pero es mala práctica.

266. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Los campos no son polimórficos: se resuelven en compilación por el tipo de la referencia (A.x = 1).

267. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: El constructor del padre llama al init() sobrescrito ANTES de inicializar los campos del hijo: x aún vale 0. Por eso no se deben invocar métodos sobrescribibles desde constructores.

268. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: id = id asigna el parámetro a sí mismo (shadowing); el campo queda en su valor por defecto 0. Falta this.id = id.

269. Respuesta correcta: B — ¿Por qué NO compila?...

Retroalimentación: Al definir Base(int), se pierde el constructor por defecto; Derived() intenta super() implícito y falla. Debe llamarse super(algúnInt) explícitamente.

270. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Patrón template method: la clase abstracta invoca el método abstracto que la subclase concreta implementa.

271. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Ante el conflicto de defaults, la clase DEBE sobrescribir; la sintaxis Interfaz.super.metodo() invoca la versión específica de cada interfaz.

272. Respuesta correcta: B — ¿Cuál es el resultado?...

Retroalimentación: Todo campo de interfaz es public static final implícito; reasignarlo es error de compilación.

273. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El constructor compacto permite validar/normalizar los parámetros ANTES de la asignación implícita a los campos; reasignar el parámetro x es legal ahí.

274. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Los records generan equals por valor de componentes, pero siguen siendo objetos distintos en memoria: == compara referencias.

275. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: Solo los tipos listados en permits pueden implementar la interfaz sellada. Los records son implícitamente final, por lo que encajan sin modificador extra.

276. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: ordinal() devuelve la posición de declaración empezando en 0: LOW=0, MID=1. No dependa de ordinal para lógica de negocio.

277. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Cada constante puede aportar su propio cuerpo (constant-specific body) implementando el método abstracto del enum: MUL multiplica.

278. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: La inner class (no estática) guarda una referencia implícita a su instancia externa y accede a sus miembros privados; se instancia con la sintaxis o.new Inner().

279. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El downcast compila (la relación existe en la jerarquía) pero falla en runtime porque el objeto real es Animal, no Dog. Proteja con instanceof.

280. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: instanceof con patrón devuelve false sin lanzar excepción cuando el tipo real no coincide; String no es Integer.

281. Respuesta correcta: D — ¿Qué método se invoca?...

Retroalimentación: Con null el compilador elige la sobrecarga MÁS específica; String e Integer no se relacionan entre sí, así que hay ambigüedad y no compila. Con solo Object y String, ganaría String.

282. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Orden de resolución: widening primitivo (short→int) gana sobre widening a long, autoboxing y varargs. La opción exacta o la promoción más cercana siempre precede al boxing.

283. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Primera instancia: estáticos 1,4 (una sola vez), luego 2,3 (padre) y 5,6 (hijo) = 142356. Segunda: solo 2356.

284. Respuesta correcta: A — ¿Este equals es una sobreescritura válida?...

Retroalimentación: La firma correcta es equals(Object); equals(Point) es un método distinto (overload) y las colecciones seguirán usando el equals de identidad. @Override habría detectado el error.

285. Respuesta correcta: C — ¿Cuál es el resultado?...

Retroalimentación: La sobreescritura debe mantener o ampliar el acceso (protected → protected/public); reducirlo rompería el polimorfismo.

286. Respuesta correcta: C — ¿Por qué esta clase NO es inmutable?...

Retroalimentación: final protege la referencia, no el contenido: quien pasó la lista o quien la obtiene con getKeys() puede hacer add/remove. Se necesita List.copyOf en constructor y getter.

287. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Greeter es una interfaz funcional (un único método abstracto); la lambda provee esa implementación inline.

288. Respuesta correcta: D — ¿Compila esta interfaz funcional?...

Retroalimentación: Los métodos abstractos que redeclaran métodos públicos de Object (equals, hashCode, toString) no cuentan para el SAM; work() es el único método funcional.

289. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: List<Integer> NO es subtipo de List<Number> (invarianza); si compilara, podría agregarse un Double a una lista de Integers. Los arrays, en cambio, sí son covariantes (y fallan en runtime).

290. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Los arrays son covariantes en compilación pero verifican el tipo real en runtime: guardar un String en un Integer[] lanza ArrayStoreException. Este agujero es lo que la invarianza de genéricos evita.

291. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: El método genérico acota T a tipos comparables consigo mismos; "pear" > "apple" lexicográficamente.

292. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: finally se ejecuta después de evaluar la expresión del return pero antes de devolver el control: primero imprime finally-, luego se imprime el valor retornado.

293. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Un return en finally descarta silenciosamente el valor (¡y hasta excepciones!) del try: gana el 2. Nunca haga return en finally.

294. Respuesta correcta: A — ¿Qué requisito debe cumplir un recurso de try-with-resources?...

Retroalimentación: AutoCloseable (padre de Closeable) define close(); el compilador genera el bloque de cierre automático. La variable es implícitamente final.

295. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Se ejecuta el primer catch compatible en orden; IllegalArgumentException ES una RuntimeException. El orden inverso (Exception antes) no compilaría por catch inalcanzable.

296. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: Capturar una checked exception que el try no puede lanzar es error de compilación; con RuntimeException o Exception genérica sí compilaría.

297. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El cuerpo imprime T; los recursos se cierran en orden inverso: primero B, luego A.

298. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: El campo static es compartido por todas las instancias de la clase; cada constructor lo incrementa.

299. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El raw type desactiva la verificación genérica: `set("text")` compila (con warning) y el cast a `Integer` explota en runtime. Evite raw types.

300. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `merge` aplica la función si la clave existe (`1+10=11`) y simplemente inserta el valor si no existe (`b=10`). Si la función devolviera `null`, la entrada se eliminaría.

301. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `getOrDefault` evita el `null` en claves ausentes; `putIfAbsent` solo escribe la primera vez (5) y no sobrescribe con 9.

302. Respuesta correcta: B — ¿Cuál es el resultado?...

Retroalimentación: `TreeSet` ordena comparando elementos: `null` no puede compararse y lanza `NPE`. `HashSet` sí admite un `null`.

303. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Los records generan `equals` y `hashCode` por componentes, por lo que dos `Item("pen")` son iguales para el `HashSet`. Con una clase normal sin sobrescribir `equals` daría `false`.

304. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Ordena por nombre `asc` y, en empate, edad descendente: Ana-30 queda antes que Ana-25.

305. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `List.copyOf` crea una copia independiente e inmutable (queda en 2); `unmodifiableList` es una VISTA del original y refleja el nuevo tamaño (3).

306. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `PriorityQueue` extrae según la prioridad del comparador (aquí, mayor primero): 3 y luego 2. El orden interno de iteración NO está garantizado, pero `poll` sí respeta la prioridad.

307. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `push` agrega al frente (2 queda primero), `offer` agrega al final; `peek` mira el frente: 2.

308. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Con `int` se invoca `remove(int index)`, no `remove(Object)`: elimina el índice 1 (el 20). Para eliminar el valor 1 habría que usar `remove(Integer.valueOf(1))`.

309. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: La inicialización perezosa construye la instancia solo en la primera llamada. Nota: sin sincronización este patrón no es `thread-safe`.

310. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `instanceof` evalúa el tipo real en runtime; a referencia un `Dog`. Las jerarquías selladas habilitan además `switch` exhaustivos por tipo.

311. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: Una `static nested class` no requiere instancia de la clase externa; se instancia directamente con `Outer.Nested`.

312. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `count` se reasigna después de capturarse en la `lambda`, dejando de ser efectivamente final: error de compilación en la `lambda`.

313. Respuesta correcta: B — ¿Qué referencia a método equivale a `(s1, s2) -> s1.compareTo(s2)`?...

Retroalimentación: Es una referencia de instancia no acotada: el primer argumento (`s1`) actúa como receptor y el segundo como parámetro.

314. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `this(5)` encadena al constructor con parámetro primero (imprime `b`) y luego continúa el cuerpo del constructor sin argumentos (`a`).

315. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Sin operación terminal el pipeline nunca se ejecuta: la lazy evaluation hace que `map` no procese ningún elemento.

316. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `filter` deja `"bb"` y `"ccc"`; `map` los convierte a mayúsculas; `toList()` (Java 16) devuelve una lista inmutable.

317. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Un stream se consume una sola vez; reutilizarlo tras una terminal lanza `IllegalStateException`. Hay que recrearlo.

318. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `flatMap` aplana a 1,2,3,4 y reduce con identidad 0 los suma: 10.

319. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `rangeClosed` incluye ambos extremos (1..5); los pares son 2 y 4, suma 6. Los streams primitivos ofrecen `sum()` directo.

320. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `orElseGet` evalúa el supplier solo si está vacío; `map` sobre empty devuelve empty y `orElse` aporta el valor por defecto B.

321. Respuesta correcta: A — ¿Cuál es la diferencia de comportamiento?...

Retroalimentación: `orElse` recibe un valor ya evaluado (evaluación estricta de argumentos); si construir el default es costoso, use `orElseGet` con `supplier`.

322. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El orden inverso coloca "c" primero; `findFirst` lo envuelve en `Optional` y `orElse` no aplica porque hay valor.

323. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `anyMatch` es cortocircuito: procesa 1, 2 y 3; al encontrar $3 > 2$ detiene el pipeline y devuelve `true`. `peek` muestra solo los elementos que fluyeron.

324. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `iterate` genera un stream infinito desde la semilla; `limit(5)` lo corta. Sin `limit`, la terminal nunca terminaría.

325. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: La sobrecarga de tres argumentos (Java 9) incluye predicado de parada: genera 1, 3, 9 y se detiene porque $27 \geq 20$.

326. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El acumulador antepone cada elemento nuevo al acumulado: ""→"a"→"ba"→"cba".

327. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `joining(delimitador, prefijo, sufijo)` concatena con separador y envuelve el total una sola vez.

328. Respuesta correcta: C — ¿Qué contiene el mapa resultante?...

Retroalimentación: `groupingBy` agrupa por la clave del clasificador en un `Map<Character, List<String>>`; los valores son listas de coincidencias.

329. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `partitioningBy` siempre produce un mapa con exactamente dos claves booleanas; `get(true)` devuelve los que cumplen el predicado (pares).

330. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: `toMap` sin función de fusión lanza `IllegalStateException` ante claves repetidas; hay que pasar un `merge`: `toMap(k, v, (v1, v2) -> v1)`.

331. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `mapping` transforma cada elemento antes del colector final dentro de cada grupo: agrupa por longitud y guarda las versiones en mayúsculas.

332. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `average()` devuelve `OptionalDouble`; `getAsDouble` extrae el primitivo 4.0. Con stream vacío lanzaría `NoSuchElementException`.

333. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `summaryStatistics` calcula en una pasada `count`, `sum`, `min`, `max` y `average`: máx 4, suma $1+2+3+4=10$.

334. Respuesta correcta: C — ¿Qué método convierte `Stream<String>` en `IntStream` de longitudes?...

Retroalimentación: `mapToInt` produce `IntStream` aplicando un `ToIntFunction`, habilitando `sum()` y evitando `boxing`.

335. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: 3 es positivo (`true`) y NO es par (`negate` de `false = true`): `true && true = true`.

336. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `andThen` encadena: primero `up` ("JAVA"), luego `len` (4). `compose` invertiría el orden.

337. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `compose` aplica `g` primero: $(3*2)+1=7$; `andThen` aplica `f` primero: $(3+1)*2=8$.

338. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `distinct→[3,1,4,5]`; `sorted→[1,3,4,5]`; `skip(1)` descarta el 1; `limit(2)` toma 3 y 4.

339. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: `chars()` emite los 10 code units; el filtro descarta el espacio: 9 letras.

340. Respuesta correcta: B — ¿Qué problema tiene este código?...

Retroalimentación: Mutar estado compartido desde un stream paralelo es una condición de carrera; el tamaño final puede ser menor a 10000. Lo correcto: `.boxed().collect(Collectors.toList())`.

341. Respuesta correcta: D — ¿Qué garantiza `forEachOrdered` en un stream paralelo?...

Retroalimentación: `forEachOrdered` sacrifica parte del paralelismo para respetar el orden original de la fuente; `forEach` en paralelo emite en orden impredecible.

342. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `concat` es asociativa y "" es identidad válida, así que el resultado paralelo es determinista: `abc`. Si la identidad fuera "X", aparecería varias veces y el resultado sí variaría.

343. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `concat` une ambos streams en orden; `max()` sobre `IntStream` devuelve `OptionalInt` con 3.

344. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Se ordenan las entradas por valor descendente (b=2 primero) y se concatenan las claves: "ba". Map.of no garantiza orden, por eso se ordena explícitamente.

345. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: El primer colector cuenta (4), el segundo suma (10) y el merger produce "10/4" en una sola pasada del stream.

346. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: repeat(2) duplica cada cadena; toArray con generador crea un String[] tipado: índice 1 es "yy".

347. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: takeWhile (Java 9) toma elementos MIENTRAS el predicado sea true y corta en el primer fallo: "tres" (4 letras) detiene el flujo. No es un filter.

348. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: dropWhile descarta el prefijo que cumple el predicado y desde el primer fallo (5) deja pasar TODO, incluidos los 2 y 1 posteriores.

349. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: min con orden natural encuentra 3; ifPresentOrElse (Java 9) ejecuta la primera acción cuando hay valor.

350. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: skip(2) descarta a! y b!; en stream secuencial findAny devuelve típicamente el primero disponible: c!.

351. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: La sobrecarga de toMap con Supplier controla la implementación del mapa resultante; sin él, la implementación es indefinida (usualmente HashMap).

352. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Si la fuente tiene tamaño conocido (SIZED) y no hay filtros, count() puede resolverse sin recorrer elementos y peek no ejecutarse. Comportamiento documentado en la API.

353. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Pares: 2, 4, 6; sus cuadrados 4+16+36 = 56.

354. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El downstream averagingDouble promedia por grupo: (1000+3000)/2 = 2000.0.

355. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: collectingAndThen aplica una transformación final al resultado del colector: la lista de 3 elementos se convierte en su tamaño.

356. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: boxed() convierte IntStream en Stream<Integer>, habilitando colectores de objetos como toList().

357. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Cada cadena se divide en 2 elementos y flatMap los aplana: a, b, c, d = 4.

358. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Optional.filter devuelve empty si el valor no cumple el predicado; "data" tiene 4 caracteres.

359. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: flatMap desanida el Optional interno; map habría devuelto Optional<Optional<String>>.

360. Respuesta correcta: D — ¿Qué imprime?...

*Retroalimentación: BiFunction.andThen aplica la función unaria al resultado: 3*4=12 → "R:12".*

361. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: stream y filter tienen longitud 6: el merge concatena en orden de encuentro: "stream|filter".

362. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El stream vacío produce Optional.empty y orElse suministra el valor alternativo.

363. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: sorted ordena a,b,c pero limit(2) hace que solo a y b fluyan por peek: imprime ab.

364. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: Stream.toList() Sí admite elementos null (a diferencia de List.of y Collectors.toUnmodifiableList, que lanzan NPE).

365. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: range(0,5) genera 0..4; los pares (0,2,4) dan P y los impares (1,3) dan I: PIPIP.

366. Respuesta correcta: D — ¿Qué imprime este pipeline con infinito ordenado?...

Retroalimentación: El stream infinito 5,4,3,2,1... encuentra el primer n<3 (el 2) y el cortocircuito de findFirst termina el pipeline. Si el filtro fuera imposible (n>10 con secuencia descendente), sí sería bucle infinito.

367. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El Supplier TreeMap ordena las claves (longitudes 3, 4, 5); firstKey devuelve la menor: 3 (de "fig").

368. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: `max` con comparador por longitud selecciona "three" (5 caracteres).

369. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `counting()` devuelve `Long`: la clave "a" aparece 3 veces. Es el idiom estándar para frecuencias.

370. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: `nullsFirst` delega en el orden natural para los no nulos y antepone los null: `[null, a, b]`.

371. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `noneMatch` devuelve `true` si NINGÚN elemento cumple: ninguno es impar → `true`.

372. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: Filtra categoría A (`qty 2 y 5`) y suma: 7. Patrón típico de agregación con streams primitivos.

373. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: La forma de tres argumentos de `collect (supplier, accumulator, combiner)` es la base mutable de los `Collectors`; en secuencial acumula en orden: `abc`.

374. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Pasan `aa` y `ab`; sus segundos caracteres son 'a' y 'b'; `distinct` los mantiene ambos: 2.

375. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: `ofNullable(null)` da `empty`; `or()` (Java 9) sustituye por `Optional[backup]`; `map` calcula su longitud: 6.

376. Respuesta correcta: C — ¿Qué problema tiene este código?...

Retroalimentación: Concatenar entrada del usuario permite inyectar SQL (' OR '1'='1'). Use `PreparedStatement` con parámetros `?`, que separa código de datos.

377. Respuesta correcta: C — ¿Cómo se asignan los parámetros de un `PreparedStatement`?...

Retroalimentación: Los índices de parámetros JDBC empiezan en 1, no en 0. Error clásico de examen.

378. Respuesta correcta: A — ¿Qué imprime si la tabla tiene 3 filas?...

Retroalimentación: El `rs.next()` inicial ya consumió la primera fila (el cursor empieza ANTES de la fila 1); el `while` cuenta solo las 2 restantes.

379. Respuesta correcta: C — ¿Qué método ejecuta un `UPDATE` y qué devuelve?...

Retroalimentación: `executeUpdate` sirve para `INSERT/UPDATE/DELETE` (y `DDL`) y devuelve el número de filas afectadas; `executeQuery` con `DML` lanza `SQLException`.

380. Respuesta correcta: D — ¿Cuál es el resultado?...

Retroalimentación: `getXxx` acepta etiqueta de columna (insensible a mayúsculas) o índice basado en 1; ambas formas son válidas y mezclables.

381. Respuesta correcta: B — ¿Qué imprime si la columna `BONUS` es `NULL` en la fila actual?...

Retroalimentación: `getInt` convierte `NULL` de SQL en 0 (los primitivos no admiten null); `wasNull()` indica si el último valor leído era `NULL` real.

382. Respuesta correcta: D — ¿Qué ocurre con los datos?...

Retroalimentación: `rollback` deshace los `inserts 1 y 2` (no confirmados); el `insert 3` posterior sí queda dentro de la transacción confirmada por `commit`.

383. Respuesta correcta: D — ¿Qué hace este código con savepoints?...

Retroalimentación: `rollback(sp)` deshace solo lo posterior al savepoint (el 2); el `commit` confirma lo restante (el 1). Los savepoints permiten `rollback` parcial.

384. Respuesta correcta: A — ¿Qué pasa con el `ResultSet` al cerrar su `Statement`?...

Retroalimentación: El ciclo de vida del `ResultSet` depende de su `Statement`: al cerrarse este (fin del `try`), el `ResultSet` queda cerrado y usarlo lanza `SQLException`.

385. Respuesta correcta: C — ¿Cómo se lee el parámetro `OUT` de este procedimiento?...

Retroalimentación: Todo parámetro `OUT` debe registrarse con su tipo SQL ANTES de ejecutar; después se lee con el getter correspondiente.

386. Respuesta correcta: A — ¿Qué logra este código?...

Retroalimentación: El `batching` reduce drásticamente los viajes de red; `results` contiene las filas afectadas por cada comando. Clave de rendimiento en cargas masivas.

387. Respuesta correcta: A — ¿Qué imprime con un `ResultSet` scrollable de 5 filas?...

Retroalimentación: `absolute(3)` va a la fila 3 (valor 30); `relative(-1)` retrocede a la fila 2 (valor 20).

388. Respuesta correcta: C — ¿Qué información aporta `getErrorCode()` en una `SQLException` con Oracle?...

Retroalimentación: `getErrorCode` devuelve el código numérico específico del vendor (ORA-xxxxx); `getSQLState` devuelve el código estándar de 5 caracteres.

389. Respuesta correcta: D — ¿Cómo se recuperan las claves generadas por un `INSERT`?...

Retroalimentación: El flag `RETURN_GENERATED_KEYS` en `prepareStatement` habilita `getGeneratedKeys()`, que devuelve un `ResultSet` con los valores generados (identity/secuencia).

390. Respuesta correcta: C — ¿Qué imprime este metadata?...

Retroalimentación: `ResultSetMetaData` describe la estructura del resultado: 2 columnas, y la de índice 2 es `NAME` (Oracle reporta identificadores en mayúsculas por defecto).

391. Respuesta correcta: C — ¿Cuál es el error de este patrón?...

Retroalimentación: En PreparedStatement, las variantes executeQuery/executeUpdate CON argumento String están prohibidas (el SQL se fijó al preparar); lanzan SQLException.

392. Respuesta correcta: A — ¿Cómo se obtiene una conexión con DriverManager?...

Retroalimentación: getConnection acepta (url), (url, user, pass) o (url, Properties). Desde JDBC 4 el driver se carga automáticamente vía ServiceLoader: Class.forName ya no es necesario.

393. Respuesta correcta: B — ¿Qué imprime si emp está vacía?...

Retroalimentación: COUNT() siempre devuelve exactamente una fila (con valor 0 si la tabla está vacía); next() es true y getInt(1) da 0.*

394. Respuesta correcta: A — ¿Cuál es el riesgo de este código en un pool de conexiones?...

Retroalimentación: Sin try-with-resources/finally la conexión puede no volver al pool (fuga). Buenas prácticas: cerrar siempre y restaurar el estado modificado.

395. Respuesta correcta: B — ¿Qué imprime este código con getObject?...

Retroalimentación: JDBC 4.2 soporta java.time: getObject con el tipo destino convierte DATE→LocalDate y TIMESTAMP→LocalDateTime sin pasar por java.sql.Date.

396. Respuesta correcta: B — En un batch, ¿qué indica Statement.EXECUTE_FAILED en el arreglo de res...

Retroalimentación: Ante fallo parcial se lanza BatchUpdateException; getUpdateCounts() muestra el estado por comando. El comportamiento de continuar o detener depende del driver.

397. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Con auto-commit desactivado el UPDATE no se confirmó; rollback lo revierte. Es el patrón transaccional básico: commit al final feliz, rollback en el catch.

398. Respuesta correcta: D — ¿Qué devuelve executeUpdate para un DELETE que no coincide con ninguna...

Retroalimentación: Cero filas afectadas es un resultado válido, no un error; verificar el conteo es la forma de detectar 'no encontrado' en DML.

399. Respuesta correcta: B — ¿Qué hace este código con un ResultSet CONCUR_UPDATABLE?...

Retroalimentación: Con concurrencia UPDATABLE, updateXxx + updateRow escribe directamente en la base; insertRow y deleteRow también están disponibles.

400. Respuesta correcta: D — ¿Qué falta para que este SELECT parametrizado funcione?...

Retroalimentación: Ejecutar con parámetros sin asignar lanza SQLException; cada ? debe recibir valor en cada ejecución (o conservar el de la anterior según el driver, lo cual no es confiable).

401. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: Antes de next() el cursor está before-first (true); con una sola fila, tras next() es simultáneamente primera y última.

402. Respuesta correcta: B — ¿Cuál es el resultado con una columna NUMBER(10,2)?...

Retroalimentación: JDBC convierte tipos numéricos flexiblemente; para dinero use siempre getBigDecimal, pues double no representa exactamente muchos decimales.

403. Respuesta correcta: D — ¿Cómo se invoca una función almacenada que DEVUELVE un valor?...

Retroalimentación: La sintaxis de escape {? = call ...} coloca el retorno como primer parámetro, que debe registrarse con registerOutParameter(1, tipo).

404. Respuesta correcta: A — ¿Qué problema de concurrencia tiene este código?...

Retroalimentación: Una Connection debe usarse por un hilo a la vez; en componentes compartidos hay que obtenerla del pool POR petición y cerrarla al terminar.

405. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: El comodín % viaja DENTRO del valor del parámetro (no en el SQL): coinciden SMITH y SCOTT. Los parámetros no pueden reemplazar identificadores ni palabras clave, pero sí valores con comodines.

406. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: Oracle opera por defecto en READ COMMITTED (y solo soporta además SERIALIZABLE); el driver refleja ese nivel.

407. Respuesta correcta: C — ¿Qué patrón muestra este código y qué previene?...

Retroalimentación: Si otro proceso cambió la fila, la versión ya no coincide y el UPDATE afecta 0 filas: la colisión se detecta sin bloquear lecturas.

408. Respuesta correcta: D — ¿Por qué falla esta consulta?...

*Retroalimentación: WHERE se procesa antes que la lista SELECT; hay que repetir la expresión (WHERE sal * 12 > 50000) o usar una subconsulta/CTE.*

409. Respuesta correcta: A — ¿Qué devuelve?...

Retroalimentación: NVL sustituye NULL por 0; sin NVL, NULL + 800 sería NULL porque cualquier aritmética con NULL da NULL.

410. Respuesta correcta: A — ¿Qué filas devuelve?...

Retroalimentación: NULL no es comparable con = o !=; el resultado es UNKNOWN y la fila se descarta. Lo correcto es IS NULL / IS NOT NULL.

411. Respuesta correcta: A — Si dept 40 no tiene empleados, ¿qué devuelve?...

Retroalimentación: El outer join produce la fila con e.empno NULL, y COUNT(columna) no cuenta NULLs: 0. Con COUNT() contaría 1, error sutil frecuente.*

412. Respuesta correcta: D — ¿Qué devuelve con sal de 800 a 5000?...

Retroalimentación: BETWEEN es inclusivo en ambos extremos: 800, 950 y 1000 cuentan; 1100 no.

413. Respuesta correcta: C — ¿Qué error tiene esta consulta?...

Retroalimentación: Toda columna del SELECT debe estar agrupada o agregada; ename varía dentro de cada grupo y Oracle rechaza la ambigüedad con ORA-00979.

414. Respuesta correcta: A — ¿Qué devuelve?...

Retroalimentación: HAVING filtra grupos por la agregada; ORDER BY 2 ordena por la segunda columna de la lista SELECT (posicional, válido).

415. Respuesta correcta: A — ¿Qué devuelve UNION frente a UNION ALL aquí?...

Retroalimentación: UNION deduplica (con costo de sort/hash); UNION ALL concatena sin eliminar repetidos y es más barato cuando los duplicados no importan.

416. Respuesta correcta: C — ¿Qué empleados devuelve?...

Retroalimentación: EXISTS es verdadero si la subconsulta correlacionada halla al menos una fila: empleados cuyo empno aparece como mgr de otro.

417. Respuesta correcta: D — ¿Qué devuelve si algún deptno de la subconsulta es NULL?...

Retroalimentación: NOT IN se traduce a deptno != v1 AND deptno != v2...; una comparación con NULL da UNKNOWN y anula todo el predicado. Use NOT EXISTS, que es null-safe.

418. Respuesta correcta: C — ¿Qué hace esta consulta?...

Retroalimentación: La inline view precalcula el promedio por departamento y el join lo compara fila a fila: patrón clásico resoluble también con funciones analíticas.

419. Respuesta correcta: D — ¿Qué devuelve ROWNUM aquí?...

Retroalimentación: ROWNUM se asigna ANTES del ORDER BY en la misma consulta, por eso hay que ordenar en la subconsulta y filtrar afuera. Desde 12c: FETCH FIRST 3 ROWS ONLY.

420. Respuesta correcta: A — ¿Qué devuelve?...

Retroalimentación: Posición negativa cuenta desde el final: -3 apunta a la 'C' de CLE; longitud 2 → 'CL'.

421. Respuesta correcta: C — ¿Qué devuelve?...

Retroalimentación: INSTR(cadena, subcadena, desde, ocurrencia): la tercera 'S' está en la posición 6 (S en 3, 4, 6, 7).

422. Respuesta correcta: C — ¿Qué devuelve?...

Retroalimentación: El modelo 0 fuerza dígitos con relleno de ceros; 9 dejaría espacios. Si el número no cabe en la máscara, Oracle muestra #.

423. Respuesta correcta: A — ¿Cuál es el resultado?...

Retroalimentación: TO_DATE valida el calendario: el 30 de febrero no existe y lanza ORA-01839, no ajusta la fecha.

424. Respuesta correcta: A — ¿Qué devuelve?...

Retroalimentación: Con días del mes iguales, MONTHS_BETWEEN devuelve un entero exacto; el primer argumento mayor da resultado positivo.

425. Respuesta correcta: A — ¿Qué evalúa este CASE?...

Retroalimentación: El CASE evalúa las ramas EN ORDEN y devuelve la primera verdadera: 3000 >= 3000 entra en ALTO y no evalúa el resto.

426. Respuesta correcta: D — ¿Qué permite ON DELETE CASCADE en una FK?...

Retroalimentación: CASCADE propaga el DELETE a las filas hijas; ON DELETE SET NULL las conservaría anulando la FK. Sin cláusula, el borrado del padre con hijos falla (ORA-02292).

427. Respuesta correcta: C — ¿Cuál es la diferencia entre DELETE y TRUNCATE?...

Retroalimentación: TRUNCATE desasigna extents y resetea el HWM sin generar undo por fila: rapidísimo pero irreversible y dispara commit implícito como todo DDL.

428. Respuesta correcta: C — ¿Qué calcula esta función analítica?...

Retroalimentación: PARTITION BY reinicia el ranking por departamento; RANK deja huecos tras empates (1,1,3). DENSE_RANK no los deja (1,1,2) y ROW_NUMBER nunca empata.

429. Respuesta correcta: D — Con salarios 5000, 3000, 3000, 2975: ¿qué devuelve DENSE_RANK para 297...?

Retroalimentación: DENSE_RANK: 5000→1, 3000→2 (ambos), 2975→3 sin salto. RANK habría dado 4.

430. Respuesta correcta: B — ¿Qué obtiene LAG aquí?...

Retroalimentación: LAG accede a la fila previa según el orden de la ventana; LEAD mira hacia adelante. El tercer argumento define el default en los bordes.

431. Respuesta correcta: B — ¿Qué calcula esta ventana?...

Retroalimentación: El frame desde el inicio hasta la fila actual produce el acumulado clásico. Con solo ORDER BY el frame por defecto es RANGE ... CURRENT ROW, que agrupa empates: ROWS es más preciso.

432. Respuesta correcta: D — ¿Por qué falla?...

Retroalimentación: Las analíticas se calculan casi al final (tras WHERE/GROUP BY/HAVING); para filtrar por ellas hay que envolver en subconsulta o CTE.

433. Respuesta correcta: A — ¿Qué genera esta consulta jerárquica?...

Retroalimentación: START WITH define la raíz (sin jefe); CONNECT BY PRIOR empno = mgr une cada padre con sus hijos; LEVEL es la profundidad. La alternativa estándar es la CTE recursiva.

434. Respuesta correcta: A — ¿Qué hace esta CTE recursiva?...

Retroalimentación: El miembro ancla genera 1 y el recursivo agrega 2..5; la suma 1+2+3+4+5 = 15. La condición WHERE n < 5 detiene la recursión.

435. Respuesta correcta: D — ¿Qué devuelve?...

Retroalimentación: COALESCE devuelve el primer argumento no NULL, con evaluación en cortocircuito: 'A'.

436. Respuesta correcta: C — ¿Qué devuelve?...

Retroalimentación: NULLIF(a, b) devuelve NULL si a = b, y a en caso contrario; útil para evitar divisiones por cero: x / NULLIF(y, 0).

437. Respuesta correcta: A — ¿Cuántas filas devuelve un CROSS JOIN entre emp (14 filas) y dept (4 f...)

Retroalimentación: El producto cartesiano multiplica: 14 × 4 = 56. Suele indicar un JOIN sin condición ON accidental.

438. Respuesta correcta: A — ¿Qué empleados lista este self-join?...

Retroalimentación: El inner join descarta la fila con mgr NULL (no matchea); para incluir al presidente se necesita LEFT JOIN.

439. Respuesta correcta: D — ¿Qué ve la Sesión B?...

Retroalimentación: El modelo multiversión de Oracle (undo) reconstruye la versión confirmada: las lecturas nunca se bloquean por escrituras ni ven datos sucios.

440. Respuesta correcta: C — ¿Qué hace FOR UPDATE en un SELECT?...

Retroalimentación: FOR UPDATE toma locks de fila (bloqueo pesimista); NOWAIT evita esperar, y WAIT n o SKIP LOCKED son alternativas para colas de trabajo.

441. Respuesta correcta: B — ¿Qué diferencia práctica muestra esto?...

Retroalimentación: CHAR(5) almacena 'ab ' (relleno); esto también afecta comparaciones con VARCHAR2 y joins. Prefiera VARCHAR2 salvo códigos de longitud fija real.

442. Respuesta correcta: A — ¿Qué devuelve?...

Retroalimentación: CEIL sube, FLOOR baja, ROUND(x,1) redondea al décimo (10.45→10.5 por redondeo del 5) y TRUNC corta sin redondear.

443. Respuesta correcta: C — ¿Qué logra esta expresión regular?...

Retroalimentación: ^ ancla el inicio, [AEIOU] exige vocal inicial, .* cualquier contenido y S\$ ancla la S final: p. ej. ADAMS.

444. Respuesta correcta: B — ¿Cuántas filas de agrupación genera CUBE(deptno, job) con 3 deptnos y ...

Retroalimentación: CUBE genera 2^n conjuntos de agrupación: (deptno,job)=15, (deptno)=3, (job)=5 y ()=1: 24 filas.

445. Respuesta correcta: B — ¿Qué hace esta sentencia?...

Retroalimentación: INSERT ALL sin condiciones inserta en todas las tablas listadas; con WHEN...THEN se convierte en insert condicional multitable.

446. Respuesta correcta: B — ¿Qué devuelve CURRVAL si en la sesión aún no se llamó NEXTVAL?...

Retroalimentación: CURRVAL es POR SESIÓN y requiere un NEXTVAL previo en la misma sesión; además las secuencias pueden tener huecos (cache, rollbacks) y no garantizan continuidad.

447. Respuesta correcta: D — ¿Qué muestra esta consulta de diccionario?...

Retroalimentación: num_rows proviene de DBMS_STATS, no de contar la tabla; USER_/ALL_/DBA_ son los tres niveles de vistas del diccionario según alcance de privilegios.

448. Respuesta correcta: D — ¿Qué privilegio necesita el usuario APP para leer la tabla HR.EMP?...

Retroalimentación: SELECT sobre el objeto basta para consultar; WITH GRANT OPTION le permitiría además re-otorgarlo. CONNECT es un rol de sistema, no un privilegio de objeto.

449. Respuesta correcta: D — ¿Qué resultado produce?...

Retroalimentación: LISTAGG agrega cadenas dentro del grupo con separador y orden propios; desde 12.2 admite ON OVERFLOW TRUNCATE para resultados que exceden 4000 bytes.

450. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: La asignación usa := (no =); PUT_LINE muestra 15 (visible con SET SERVEROUTPUT ON en el cliente, pero el bloque ejecuta igual).

451. Respuesta correcta: B — ¿Qué contiene v_emp tras el fetch?...

Retroalimentación: %ROWTYPE crea un record espejo de la tabla; SELECT INTO llena todos los campos de la fila única recuperada.

452. Respuesta correcta: B — ¿Qué excepción lanza si el empno no existe?...

Retroalimentación: SELECT INTO exige exactamente una fila: cero filas lanza NO_DATA_FOUND y más de una TOO_MANY_ROWS. No deja la variable en NULL.

453. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El bloque interno no maneja VALUE_ERROR; la excepción se propaga al bloque externo (saltándose 'sigue') donde WHEN OTHERS la captura.

454. Respuesta correcta: C — ¿Qué hacen SQLCODE y SQLERRM dentro de un manejador?...

Retroalimentación: Registrar y relanzar es el patrón correcto: WHEN OTHERS que 'traga' errores sin relanzar oculta fallos (y el warning PLW-06009 lo advierte).

455. Respuesta correcta: A — ¿Qué logra RAISE_APPLICATION_ERROR?...

Retroalimentación: Es la vía estándar para errores de negocio: el rango -20000..-20999 está reservado a aplicaciones y el mensaje viaja como ORA-20001 al llamador (visible en SQLException de JDBC).

456. Respuesta correcta: B — ¿Cuántas iteraciones ejecuta?...

Retroalimentación: REVERSE itera del límite superior al inferior; los límites se escriben SIEMPRE menor..mayor. El contador i es implícito y de solo lectura.

457. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: El LOOP básico requiere EXIT explícito; la condición se cumple cuando v_i llega a 3 y el bucle termina con ese valor.

458. Respuesta correcta: B — ¿Qué imprime tras un UPDATE que afectó 3 filas?...

Retroalimentación: SQL%ROWCOUNT reporta las filas del último DML (3); SQL%NOTFOUND es false porque hubo filas afectadas.

459. Respuesta correcta: B — ¿Cuál es la secuencia correcta con un cursor explícito?...

Retroalimentación: OPEN ejecuta la consulta y posiciona; cada FETCH trae una fila; CLOSE libera recursos. Fetch sin open lanza INVALID_CURSOR.

460. Respuesta correcta: D — ¿Qué ventaja tiene este cursor FOR loop?...

Retroalimentación: El cursor FOR loop elimina el código repetitivo y cierra el cursor incluso ante excepciones: la forma idiomática de recorrer consultas.

461. Respuesta correcta: A — ¿Qué hace WHERE CURRENT OF?...

Retroalimentación: FOR UPDATE bloquea las filas del cursor y WHERE CURRENT OF referencia la fila actual por ROWID: actualización posicionada segura.

462. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: IN OUT permite leer y modificar el argumento: los valores se intercambian y la llamada refleja 2,1.

463. Respuesta correcta: A — ¿Se puede llamar así en SQL?...

Retroalimentación: Es un patrón común, con costo: el cambio de contexto SQL↔PL/SQL por fila. DETERMINISTIC o la cláusula WITH FUNCTION pueden mitigar.

464. Respuesta correcta: D — ¿Qué particularidad tienen las variables declaradas en la spec de un p...?

Retroalimentación: El estado de package persiste entre llamadas de la misma sesión (útil como caché); recompilar el package puede invalidarlo (ORA-04068).

465. Respuesta correcta: C — ¿Cuándo se dispara este trigger?...

Retroalimentación: FOR EACH ROW lo hace de nivel fila; solo los triggers BEFORE de fila pueden modificar :NEW. En DELETE solo existe :OLD.

466. Respuesta correcta: D — ¿Qué hace este bulk processing?...

Retroalimentación: BULK COLLECT y FORALL reducen los context switches PL/SQL↔SQL: en volúmenes grandes puede ser un orden de magnitud más rápido que el bucle fila a fila.

467. Respuesta correcta: B — ¿Qué diferencia hay entre estas colecciones?...

Retroalimentación: Associative arrays viven solo en memoria PL/SQL y admiten índices no consecutivos (o VARCHAR2); nested tables y varrays son tipos SQL almacenables.

468. Respuesta correcta: D — ¿Qué imprime?...

Retroalimentación: El array asociativo es disperso: COUNT cuenta elementos existentes (2) y LAST devuelve el mayor índice usado (5).

469. Respuesta correcta: A — ¿Qué hace SAVE EXCEPTIONS en FORALL?...

Retroalimentación: Sin SAVE EXCEPTIONS el primer fallo aborta el lote; con ella se completa lo posible y luego se lanza ORA-24381 con el detalle por índice.

470. Respuesta correcta: A — ¿Cómo consume Java este procedimiento?...

Retroalimentación: CallableStatement + registerOutParameter(1, OracleTypes.CURSOR) permite castear el OUT a ResultSet: puente clásico PL/SQL→JDBC.

471. Respuesta correcta: C — ¿Qué riesgo tiene este SQL dinámico?...

Retroalimentación: Igual que en JDBC: concatenar datos en SQL dinámico permite inyección y fuerza hard parses; USING liga variables de forma segura y eficiente.

472. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: EXECUTE IMMEDIATE admite INTO para consultas de una fila y USING para binds posicionales: SQL dinámico con parámetros seguros.

473. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: Asignar 4 caracteres a VARCHAR2(3) lanza ORA-06502 (VALUE_ERROR) en runtime; PL/SQL no trunca implícitamente en asignaciones.

474. Respuesta correcta: D — ¿Qué valor tiene v tras el bloque?...

Retroalimentación: NULL = NULL evalúa a UNKNOWN (no verdadero); la rama correcta es IS NULL: v = 2. La lógica trivaluada aplica igual en PL/SQL.

475. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: El CASE sentencia (simple) compara contra cada WHEN; sin ELSE y sin coincidencia lanzaría CASE_NOT_FOUND (ORA-06592), a diferencia de la expresión CASE que daría NULL.

476. Respuesta correcta: C — ¿Qué muestra este código para depurar errores de compilación?...

Retroalimentación: USER_ERRORS es el equivalente a SHOW ERRORS: línea, posición y mensaje de cada error del objeto almacenado.

477. Respuesta correcta: A — ¿Qué hace DETERMINISTIC en una función?...

Retroalimentación: Es un contrato del programador (Oracle no lo verifica): permite function-based indexes y evita reevaluaciones. Mentir en él produce resultados incorrectos.

478. Respuesta correcta: A — ¿Qué patrón muestra este compound trigger?...

Retroalimentación: El compound trigger comparte estado entre los timing points de UNA sentencia: patrón estándar contra ORA-04091 y para auditoría eficiente.

479. Respuesta correcta: B — ¿Qué imprime?...

Retroalimentación: BOOLEAN existe en PL/SQL (no como tipo de columna SQL hasta 23c); el IF lo evalúa directamente.

480. Respuesta correcta: A — ¿Qué hace este código de paquete UTL?...

Retroalimentación: CAST_TO_RAW convierte a bytes (4142 hex) y CAST_TO_VARCHAR2 los reinterpreta como texto: la ida y vuelta restaura 'AB'.

481. Respuesta correcta: A — ¿Cuándo se ejecuta realmente el DML de este bloque?...

Retroalimentación: Los bloques PL/SQL no confirman automáticamente; la transacción sigue abierta en la sesión. Salir del cliente puede confirmar o revertir según la herramienta.

482. Respuesta correcta: A — ¿Qué devuelve v?...

Retroalimentación: RETURN termina la función inmediatamente; la asignación posterior nunca se ejecuta (PL/SQL puede advertirlo con PLW-06002 pero compila).

483. Respuesta correcta: C — ¿Qué logra este uso de MERGE dentro de PL/SQL?...

Retroalimentación: El patrón SELECT-luego-INSERT/UPDATE tiene ventana de carrera; MERGE resuelve la decisión dentro del motor en una sentencia.

484. Respuesta correcta: C — ¿Qué imprime?...

Retroalimentación: CONTINUE WHEN (11g+) salta a la siguiente iteración cuando la condición se cumple: omite el 2.

485. Respuesta correcta: A — ¿Qué imprime?...

Retroalimentación: La declaración interna oculta (shadowing) a la externa dentro de su bloque; al salir, la x externa recupera visibilidad. Un label <<outer>> permitiría outer.x.

486. Respuesta correcta: D — ¿Qué evalúa correctamente cadenas nulas en Oracle?...

Retroalimentación: Peculiaridad de Oracle: " y NULL son indistinguibles en VARCHAR2 (a diferencia del estándar SQL y de otras BD). Afecta comparaciones y constraints.

487. Respuesta correcta: C — ¿Qué problema de rendimiento tiene este código?...

*Retroalimentación: Regla de oro: si se puede hacer en un solo SQL, hágalo en SQL. El bucle genera 1M de round-trips lógicos; UPDATE emp SET sal = sal*1.1 lo hace en una operación.*

488. Respuesta correcta: D — ¿Cuál reescritura permite usar el índice de hiredate?...

Retroalimentación: El predicado de rango (sargable) sobre la columna desnuda habilita un INDEX RANGE SCAN; las tres variantes con función sobre la columna lo impiden.

489. Respuesta correcta: B — ¿Qué operación del plan indica un problema probable aquí?...

Retroalimentación: MERGE JOIN CARTESIAN con tablas medianas suele delatar un join olvidado: el resultado crece multiplicativamente. A veces es legítimo con tablas diminutas.

490. Respuesta correcta: C — ¿Por qué esta consulta con bind podría usar un plan subóptimo para 'RA'...

Retroalimentación: Con datos sesgados, el plan óptimo depende del valor: Oracle 'espía' el primer bind y desde 11g el adaptive cursor sharing genera planes por rango de selectividad.

491. Respuesta correcta: C — ¿Qué problema tiene SELECT * en esta consulta de reporte?...

*Retroalimentación: Si las columnas pedidas están todas en un índice, Oracle resuelve sin tocar la tabla (index-only). SELECT * lo imposibilita y acopla el código a cambios de esquema.*

492. Respuesta correcta: C — ¿Cuál de estas consultas evita la subconsulta correlacionada por fila?...

Retroalimentación: La función analítica calcula el promedio por departamento en UNA pasada; la correlacionada lógicamente re-agrega por cada fila (aunque el optimizador a veces la reescribe solo).

493. Respuesta correcta: C — ¿Qué delata este plan sobre el índice?...

Retroalimentación: CREATE INDEX ix ON emp(UPPER(ename)) materializa la expresión y el mismo predicado pasa a usar INDEX RANGE SCAN.

494. Respuesta correcta: D — ¿Cuál es el problema de rendimiento en Java?...

Retroalimentación: El patrón N+1 multiplica la latencia de red; además cada literal distinto genera un hard parse que contamina la shared pool. Un JOIN único resuelve en un round-trip.

495. Respuesta correcta: D — ¿Por qué es mala idea COMMIT dentro del bucle?...

Retroalimentación: Commits masivos por fila degradan el rendimiento y arriesgan snapshot too old sobre el cursor abierto. Confirme por lotes o al final de la unidad lógica.

496. Respuesta correcta: D — ¿Qué mejora aporta el fetch size aquí?...

Retroalimentación: Cada viaje de red tiene latencia fija; traer 500 filas por viaje amortiza ese costo. El valor óptimo balancea memoria y latencia.

497. Respuesta correcta: D — ¿Cuál LIKE puede usar un índice normal sobre ename?...

Retroalimentación: El índice B-tree ordena por el inicio de la cadena: solo el patrón con prefijo literal delimita un rango. Para sufijos: índice sobre REVERSE(ename) o Oracle Text.

498. Respuesta correcta: A — ¿Qué transformación reduce este DELETE masivo de horas a minutos?...

Retroalimentación: Borrar el 80% genera undo/redo gigantescos; recrear con CREATE TABLE AS SELECT lo que se conserva (o particionar y soltar particiones) es el enfoque estándar para purgas masivas.

499. Respuesta correcta: B — ¿Qué patrón de caché muestra y qué riesgo tiene?...

Retroalimentación: computeIfAbsent es atómico por clave en ConcurrentHashMap. El riesgo real es de consistencia: sin TTL o invalidación, el caché sirve datos viejos.

500. Respuesta correcta: C — ¿Por qué la versión B es preferible para paginación profunda?...

Retroalimentación: El OFFSET grande escala linealmente con la profundidad; la paginación por keyset (seek method) usa el índice para posicionarse y cada página cuesta lo mismo.